# 分布与并行数据库

## 分布式数据库管理

主讲教师：鄂金龙
中国人民大学信息学院

# 1.分布式事务管理

- 事务的基本概念
- 分布式事务
- 分布式事务的提交协议
- 分布式事务管理的实现
- 两段提交协议(2PC)的实现方法
- 非阻塞分布式事务提交协议

# 事务的基本概念

## 事务的定义

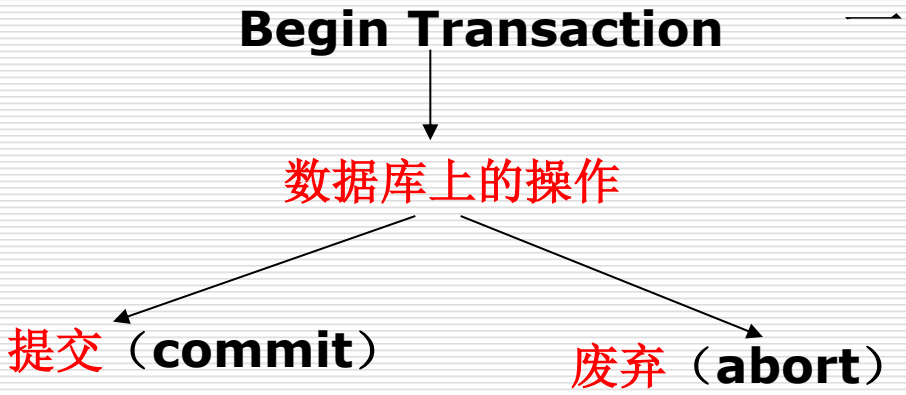
- 任何数据库应用最终都将转换为一系列对数据库进行存取的操作系列；
- 为了保证数据库的正确性及操作的有效性，将数据库应用中全部或部分操作序列的执行定义为事务；
- 一个事务所包含的所有操作，要么全做，要么全不做，是一个不可分割的整体；

**事务是由若干个为完成某一任务而逻辑相关的操作组成的操作序列，是保证数据库正确性的基本逻辑单元。**

- **显式声明**：应用程序通过事务命令显式地划分事务；
- **隐式声明**：系统按照默认规定自动地划分事务；

# 事务的基本概念

## □ 事务的基本模型




一个事务由三部分组成：

- 开始标识begin_transaction
- 数据库操作
- 结束标识commit或abort

- commit：表示提交，即成功完成事务中的所有数据库操作，将操作结果永久化，使数据库状态从事务执行前的状态改变到事务执行后的状态；
- abort：表示废弃，即在事务执行过程中发生了某种故障，使得事务中的操作不能继续执行，系统需要将该事务中已完成的操作全部撤销，使数据库保持事务执行前的状态；

# 事务的基本概念

例1：航班订票系统。设航班订票数据库中有：

航班信息表**FLIGHT(Fno，Date，Src，Dest，StSold，Capacity)**

顾客订座表**FC(Fno，Date，Cname，Special)**，

其中，**Fno**为航班号，**Date**为航班日期，**Src**为出发地，**Dest**为目的地，**StSold**为卖出席位，**Capacity**为座席数；**Cname**为客户姓名，**Special**为客户信息。

# 事务的基本概念

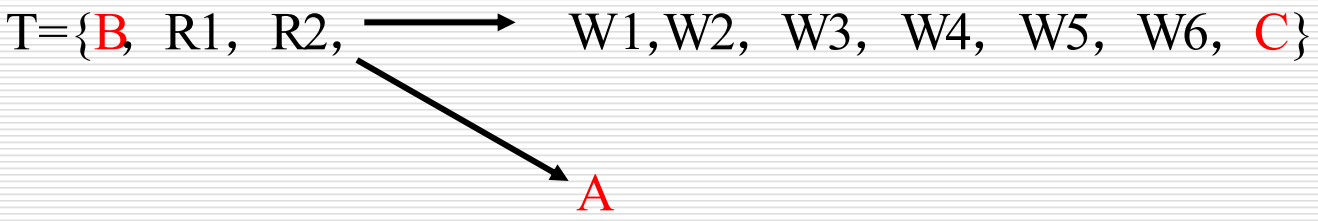
```
begin_transaction reservation  /*事务开始*/
   input (Flight_no，Cdate，Customer_name)；   /*输入预订信息*/
   EXEC SQL SELECT StSold，Capacity  /*查询预订航班的卖出席位和坐席数*/
                INTO temp1，temp2
                FROM FLIGHT   WHERE Fno= Flight_no AND Date= Cdate；
    If temp1==temp2 then  /*若无空座*/
       Output(“no free seats”)；
       Abort；  /*事务废弃*/
    Else/*更新航班信息表和顾客订座表*/
          EXEC SQL UPDATE FLIGHT SET StSold= StSold+1
                     WHERE Fno= Flight_no AND Date= Cdate；
          EXEC SQL INSERT INTO FC(Fno，Date，Cname，Special)
                     VALUES (Flight_no，Cdate，Customer_name，null)；
          Commit；  /*事务提交*/
       Output(“reservation complete”)；
     Endif
End;
```

# 事务的基本概念

```
B：Begin_transaction reservation  /*事务开始*/
R1：input (Flight_no，Cdate，Customer_name)；   /*读取预订信息*/
R2：temp←read(FLIGHT(Flight_no，Cdate). StSold)；/*读取卖出席位*/
      If temp==FLIGHT(Flight_no，Cdate). Capacity then
         Output(“no free seats”)；
A：       Abort；  /*事务废弃*/
      Else
W1：      Write (FLIGHT.StSold，temp+1)；   /*更新航班信息表*/
W2：      Insert (FC，fc)；   /*更新顾客订座表*/
W3：      Write(FC.Fno，Flight_no)；
W4：      Write(FC. Date，Cdate)；
W5：      Write(FC. Cname，Customer_name)；
W6：      Write(FC. Special，null)；
C：       Commit；   /*事务提交*/
          Output(“reservation  complete”)；
    End;
```

# 事务的基本概念

从上面事务具体操作描述，可得到订票事务的偏序集T




B：事务开始;R：读操作；W：写操作；C：事务提交；A：事务废弃

# 事务的基本概念

## 事务的基本性质

### 原子性(atomicity)

事务所包含的操作要么全做，要么全不做。

由于输入错误、系统过载、死锁等导致的事务废弃而需要进行的事务的原子性维护，称为事务恢复。

由于系统崩溃（死机、掉电）而导致的事务废弃或者提交结果的丢失而需要进行的原子性维护处理，称为故障恢复。

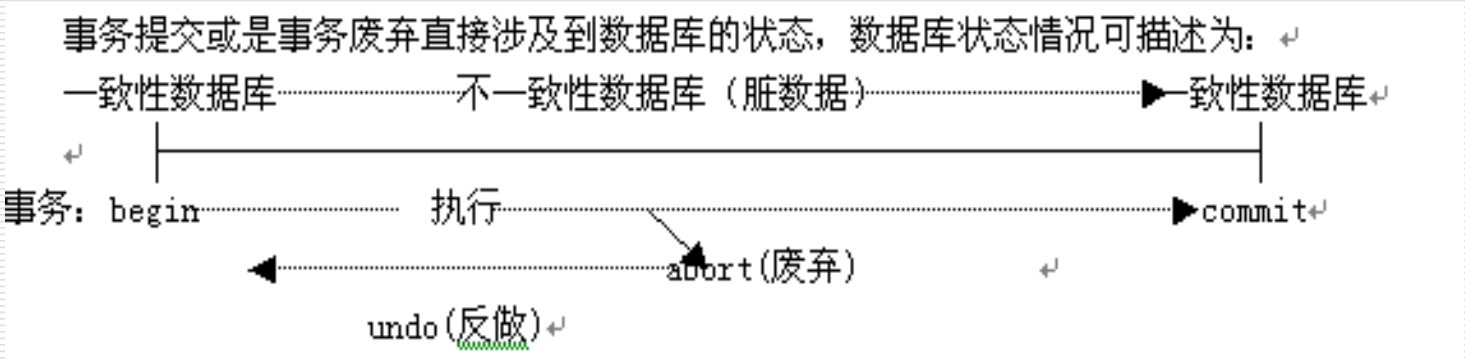
- 对于废弃的事务，需要撤销事务已经完成的部分操作，使得数据库恢复到事务执行前的状态，称为反做(undo)；
- 对于已经提交的事务，由于不能保证事务所作的修改已经被更新到外存数据库，因此需要重新执行事务的全部操作，称为重做(redo)；

# 事务的基本概念

## 事务的基本性质

### 一致性(consistency)

假如数据库的状态满足所有的完整性约束，则称该数据库是一致的。事务的一致性，是事务执行的结果必须是使数据库从一个一致性状态变化到另一个一致性状态，而不会停留在某种不一致的中间状态上。



# 事务的基本概念

## 事务的基本性质

### 隔离性(isolation)

当多个事务的操作交叉执行时，若不加控制，一个事务的操作及所使用的数据可能会对其它事务造成影响。事务的隔离性是指：一个事务的执行既不能被其它事务所干扰，同时也不能干扰其它事务。

具体来讲，一个没有结束的事务在提交之前不允许将其结果暴露给其它事务。因为未提交的事务有可能在以后的执行中废弃。若存在其它事务使用了这种无效的数据，则这些事务同样也要进行废弃。这种因一个事务的废弃而导致其它事务被牵连地进行废弃的情况称为“级联废弃”。

# 事务的基本概念

## 事务的基本性质

### 隔离性(isolation)

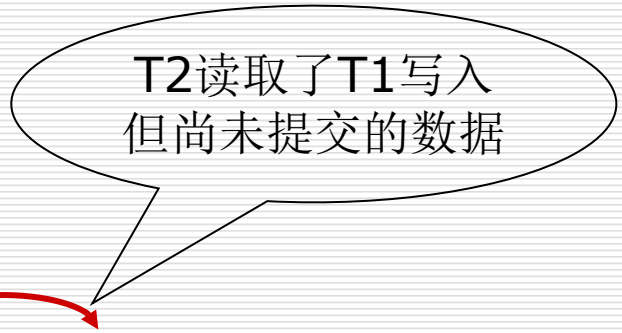
例如，

T1：{R1(X)，W1(X)}

T2：{R2(Y)， R2(X)，W2(X)}

设T1与T2的操作交叉执行过程为

{R1(X)，R2(Y)，W1(X)，R2(X)，W2(X)}




在执行中，T2引用了没提交的事务T1的结果X，则当T1提交失败时，T1需反做，T2也必须反做（“级联废弃”）。

如果T1的操作及使用的数据对T2是隔离的，也就是当T1提交后，T2再使用T1的结果，那么就不会出现级联废弃问题了。

# 事务的基本概念

## 事务的基本性质

### 耐久性(durability)

事务的耐久性体现在：当一个事务提交后，系统保证该事务的结果不会因以后的故障而丢失。也就是说，事务一旦被提交，它对数据库的更改将是永久性的。即使发生了故障，系统应具备有效的恢复能力，将已提交事务的操作结果恢复过来，即重做（REDO）处理，使这些事务的执行结果不受任何影响。

# 事务的概念

- 例子
  - 从银行账户A转帐200到帐户B

```
T:  read(A);
    A: = A – 200;
    write(A);
    read(B);
    B: = B + 200;
    write(B);
```

```
T’:  read(A),read(B);
     Sum=A+B;
```

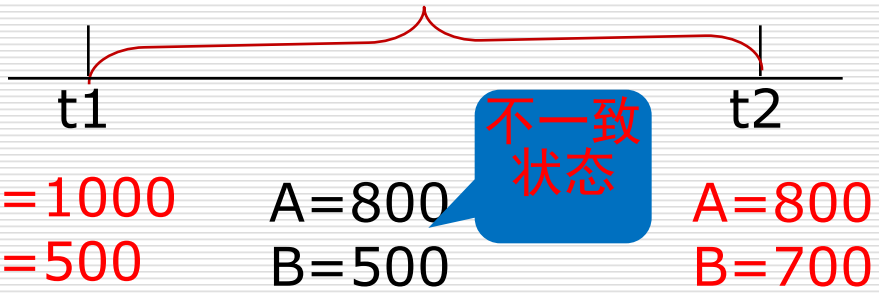
原子性：“转出操作+转入操作”要么都做，要么都不做

一致性：事务执行前后 A、B之和不变

持久性：转帐结束后，数据不会丢失

隔离性：并发执行事务必须保持互不干扰

如：转帐事务＋求和事务



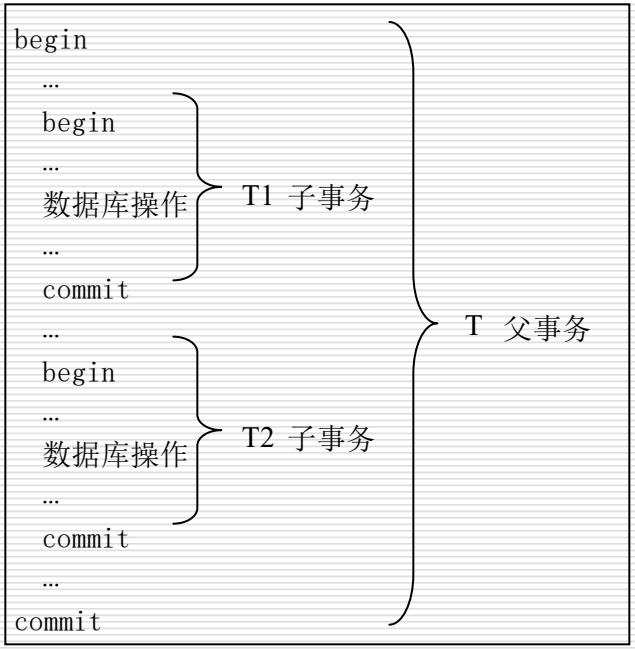
# 事务的基本概念

## □ 事务的种类

按照组成结构的不同，可以将事务划分为两类：平面事务（flat）和嵌套事务(nest)。

平面事务是指每个事务都与系统中其它事务相分离，并独立于其它事务。平面事务是用begin和end括起来的自治执行方式。

嵌套事务是指一个事务的执行包括另一个事务。其中，内部事务称为外部事务的子事务，外部事务称为子事务的父事务。



# 事务的基本概念

## □ 事务的种类

按照组成结构的不同，可以将事务划分为两类：平面事务（flat）和嵌套事务(nest)。

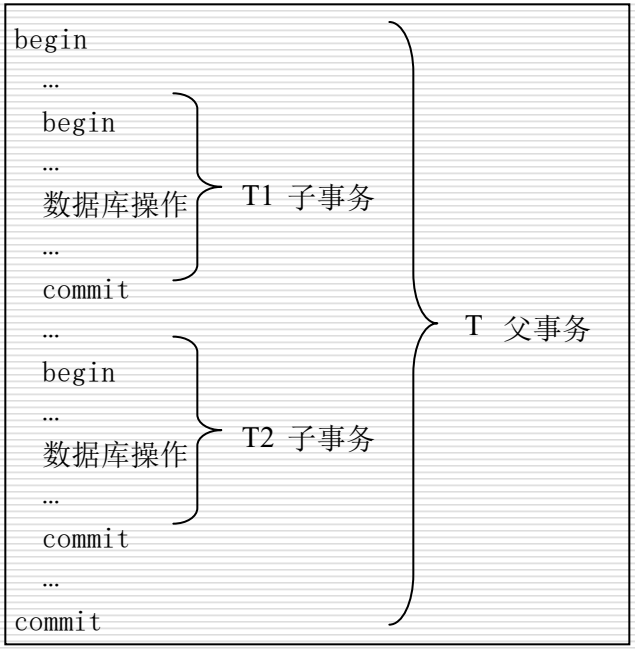
嵌套事务具有：

- 提交依赖性（commit_dependency）

  子事务提交，必须等待父事务提交。

- 废弃依赖性(abort_dependency)

  父事务废弃，则子事务必须废弃。



# 分布式事务

## □ 分布式事务的定义

从宏观上来看，分布式事务是由一系列分布在多个场地上执行的数据库操作所组成的。

分布式事务：是指分布式数据库应用中的事务，也称为全局事务。

子事务：一个分布式事务在执行时将被分解为若干个场地上独立执行的操作序列，即一个分布式事务在某个场地上操作的集合。

显然，分布式事务是典型的嵌套事务。

# 分布式事务

## □ 分布式事务的定义

分布式事务同样具有ACID四个特性。由于子事务的分布式执行，因此保证分布式数据库系统的ACID特性更加复杂。

例如，为了保证全局事务的原子性，必须保证组成该事务的所有子事务要么全部提交，要么全部撤销，不允许出现部分场地上的子事务提交，部分场地上的子事务撤销的情况发生。因此，在分布式事务执行过程中，需要对子事务进行协调，从而决定事务的提交与撤销。

另外，在分布式事务中，除了要考虑对数据的存取操作序列之外，还需要涉及大量的通信原语和控制报文。通信原语负责在进程间进行数据传送，控制报文负责协调各子事务的操作。

# 分布式事务

## 分布式事务的实现模型

一个分布式事务所要完成的任务是由分布于各个场地上的子事务相互协调合作完成的。有两种实现模型：

- 进程模型
- 服务器模型

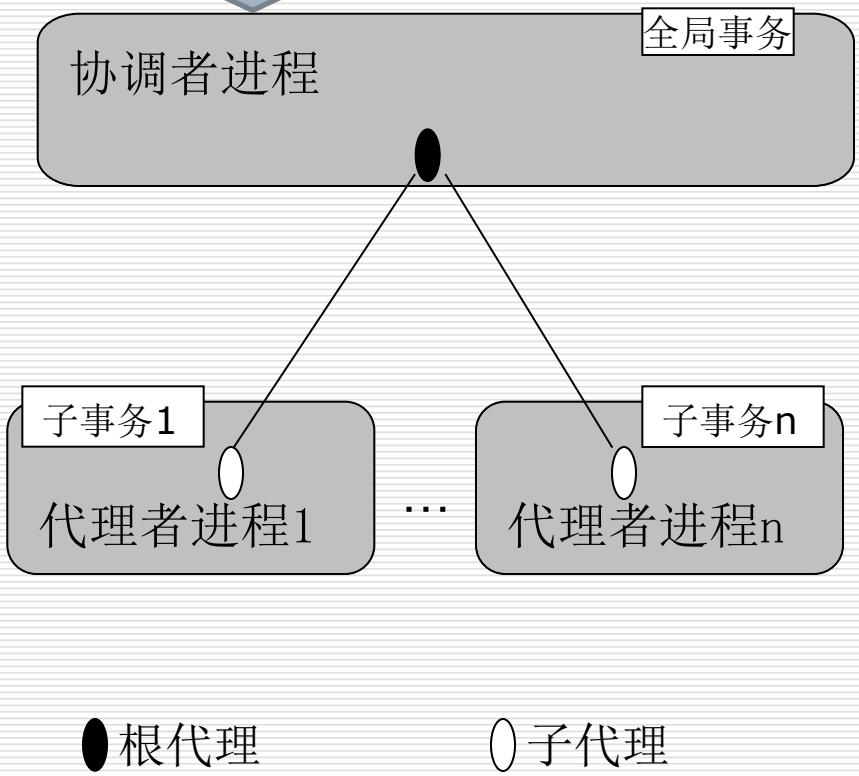
- 每个应用均有一个根代理，负责执行全局事务的开始、提交或废弃等命令；
- 只有根代理才可以请求创建新的子代理；
- 只有当各个子事务均被成功提交后，根代理才能决定在全局上提交该事务，否则根代理将决定废弃该事务。

# 分布式事务

## □ 分布式事务的实现模型

**进程模型：** 全局事务为每一个子事务在相应的场地上创建一个代理者进程（也称局部进程或子进程），由代理者进程执行该场地上的有关操作。同时，为协调各子事务的操作，全局事务还要启动一个协调者进程，来进行代理者进程间的通讯，控制和协调各代理者进程的操作。

发出分布式事务的场地（根代理所在的场地）称为该事务的**源场地**。



# 分布式事务

## □ 分布式事务的实现模型

**例2**：实现两个银行帐户间的转帐操作的分布式事务。

设有一银行帐户关系ACCOUNT{Acc_No，Amount}，其中Acc_No、Amount分别为帐号和账面余额。

要求实现从贷方帐号（from_acc）到借方帐号（to_acc）的转帐操作，转帐金额为transfer_amount。

假设贷方帐户和借方帐户分别存在不同场地上。

# 分布式事务

## 分布式事务的实现模型

```
begin_transaction
        input (transfer_amount,from_acc,to_acc);
        EXEC SQL SELECT Amount
                INTO temp
                FROM ACCOUNT
                WHERE Acc_No=from_acc;
        If temp< transfer_amount then
                Output("transfer failure");
                Abort;
        Else
                EXEC SQL UPDATE ACCOUNT
                SET Amount=Amount- transfer_amount
                WHERE Acc_No=from_acc;
                EXEC SQL UPDATE ACCOUNT
                SET Amount=Amount+ transfer_amount
                WHERE Acc_No=to_acc;
                Commit;
                Output("transfer complete");
end
```

# 分布式事务

## □ 分布式事务的实现模型

由于贷借双方的帐户分别存放于不同场地上，因此转帐操作涉及两个场地。整个转帐应用被分为两个进程（协调者进程和代理者进程），其操作分别由根代理和子代理协作完成。

场地1：根代理(ROOT)

```
begin_transaction;
  input (transfer_amount,from_acc,to_acc);
  EXEC SQL SELECT Amount INTO temp
           FROM ACCOUNT
           WHERE Acc_No=from_acc;
  If temp< transfer_amount then
     Outout("transfer failure");
     Abort;
  Else
      EXEC SQL UPDATE ACCOUNT
          SET Amount=Amount- transfer_amount
          WHERE Acc_No=from_acc;
      Create AGENT1;
      Send (AGENT1,transfer_amount,to_acc);
      Commit;
End
```

场地2：子代理（AGENT1）

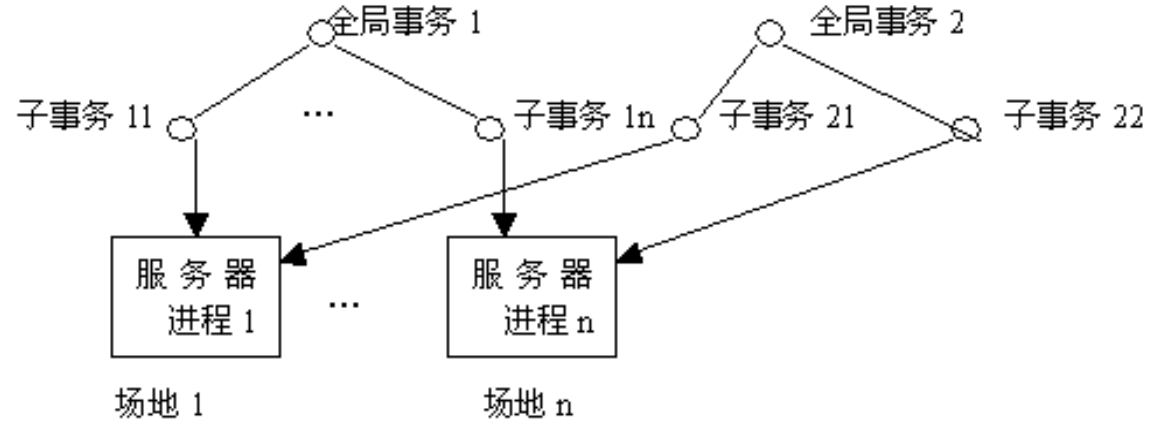
```
Receive (ROOT, transfer_amount, to_acc);
EXEC SQL UPDATE ACCOUNT
    SET Amount=Amount + transfer_amount
    WHERE Acc_No=to_acc;
```

# 分布式事务

## □ 分布式事务的实现模型

**服务器模型：** 服务器模型要求在事务的每个执行场地上创建一个服务器进程，用于执行发生在该场地上的所有子事务。每个服务器进程可以交替地为多个事务的子事务服务。也就是说，不同全局事务中的子事务在同一个场地上共用一个服务器进程。

服务器模型可表示为：



# 分布式事务

## □ 分布式事务管理的目标

分布式事务管理的目标是使事务的执行具有较高的执行效率，具有较高的可靠性和并发性。与事务执行效率有关的因素还包括：

- CPU和内存利用率
- 控制报文开销
- 分布式执行计划
- 系统可用性

为此，提出分布式事务管理的目标是：

- 维护分布式事务的ACID性质；
- 提高系统的性能，包括CPU、内存等资源的使用效率和数据资源的使用效率，尽量减少控制报文的长度及传送次数，加速事务的响应速度；
- 提高系统可靠性和可用性，当系统的一部分或者局部发生故障时，系统仍能正常运转，而不是整个系统瘫痪。

# 分布式事务的提交协议

分布式数据库中的全局事务由被分解为在各个场地上执行的子事务所组成。只有当各个场地上的子事务都正确执行后，全局事务才可以提交。只要有一个子事务不能提交，则全局事务应该废弃，接下来所有的子事务也应全部废弃。因此，所有子事务均可以正确提交是分布式事务提交的前提。

上述过程，通过两阶段提交协议来实现，简称2PC（2-Phase-Commit）协议。

# 分布式事务的提交协议

## 协调者和参与者

- 协调者：是协调者进程的执行方，即根代理，负责决定所有子事务的提交或废弃；
- 参与者：是代理者进程的执行方，即子代理，负责执行本地数据库操作，负责本地子事务的提交或废弃，并向协调者报告本地子事务的执行状态；
- 协调者和参与者都有本地日志，他们在进行操作前都必须将该操作记录到相应的日志文件中，以进行事务故障恢复和系统故障恢复。

# 分布式事务的提交协议

## 两阶段提交协议的基本思想

**决定阶段：**有协调者发送预提交(prepare)命令，然后等待参与者的应答。如果所有的参与者都返回“准备提交(ready)”。那么协调者做出提交决定；如果至少有一个参与者返回“准备废弃”，那么协调者做出废弃决定。

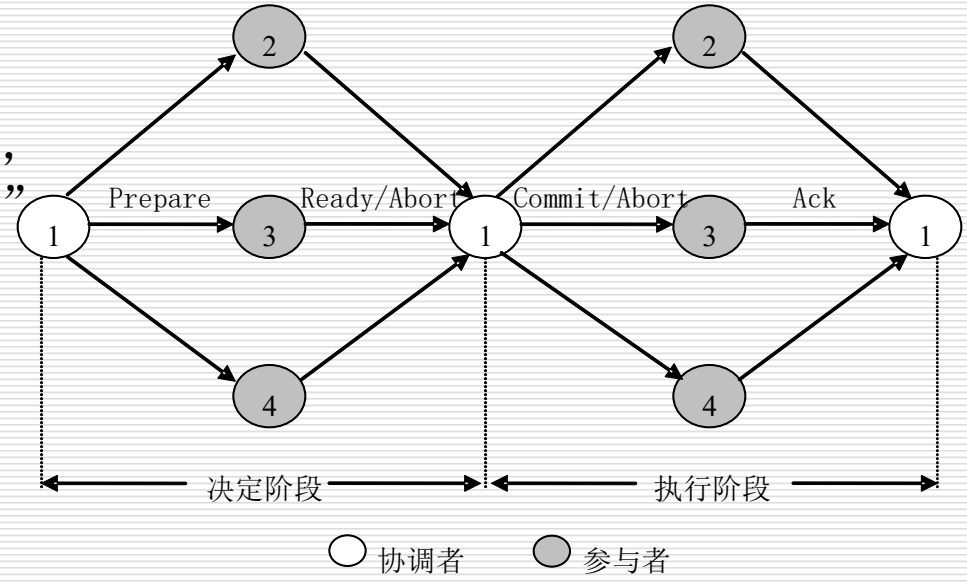
**执行阶段：**协调者把在决定阶段做出的决定发送给参与者。如果协调者向各个参与者发“提交”（Commit）命令，各个参与者执行提交；如果协调者向各个参与者发出“废弃”（Abort）命令，各个参与者执行废弃，取消对数据库的修改。无论是“提交”还是“废弃”，各参与者执行完毕后都要向协调者返回“确认”（Ack）应答，通知协调者执行结束。

# 分布式事务的提交协议

## 两阶段提交协议的基本流程

步骤1：协调者在征求各参与者的意见之前，首先要在它的日志文件中写入一条“开始提交”（Begin_commit）记录。然后，协调者向所有参与者发送“预提交”（Prepare）命令。协调者进入等待状态，等待收集各参与者的应答。

步骤2：参与者接收到“预提交”（Prepare）命令后，判断本场地子事务是否已准备好提交。若是，参与者在日志中写入“准备提交”（Ready）记录，并向协调者发送“准备提交”（Ready）应答；否则，参与者在日志中写入“准备废弃”（Abort）记录，并向协调者发送“准备废弃”（Abort）应答。发送应答后，参与者将等待协调者所做出的最终决定。

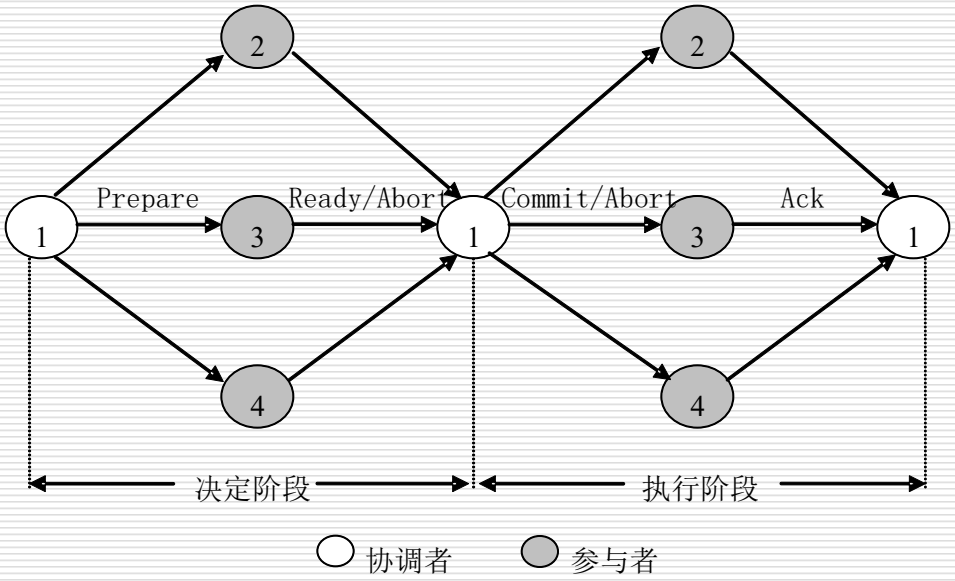


# 分布式事务的提交协议

## 两阶段提交协议的基本流程

步骤3：协调者收集各参与者发来的应答，若存在至少一个参与者发来“准备废弃”的应答，则协调者在日志中写入“决定废弃”（Abort）记录，并发送“全局废弃”（Abort）命令；否则，协调者在日志中写入“决定提交”（Commit）记录，向所有参与者发送“全局提交”（Commit）命令。此后，协调者进入等待状态，等待收集各参与者的应答信息。

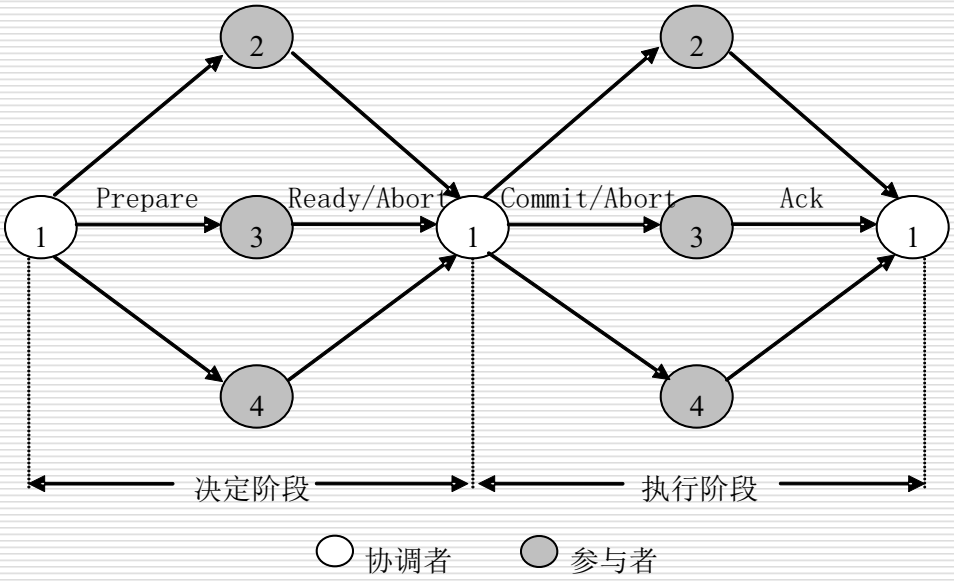
步骤4：参与者接收到协调者发来的命令，若为“全局提交”命令，则参与者在日志中写入“提交”（Commit）的记录，并对子事务实施提交；否则，参与者在日志文件中写入“废弃”（Abort）的记录，并对子事务实施废弃。实施完毕后，各个参与者要向协调者发送确认信息（Ack）。



# 分布式事务的提交协议

## □ 两阶段提交协议的基本流程

步骤5：当协调者接收到所有参与者发送的确认信息后，在日志中写入“事务结束”（End_transaction）记录，全局事务结束。

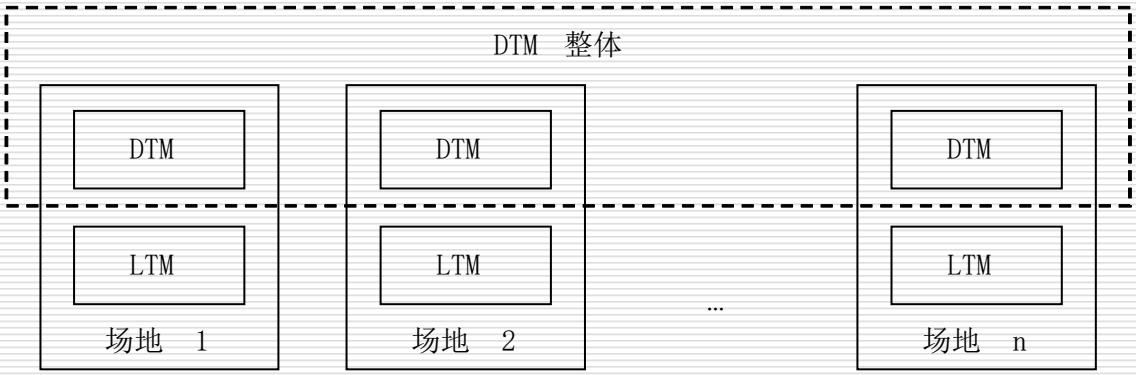


# 分布式事务管理的实现

## LTM与DTM

分布式事务管理在功能上分为两个层次：

- 局部事务管理器（LTM）
  负责本场地的子事务执行、故障恢复和并发控制；
- 分布式事务管理器（DTM）
  协同全局事务的执行、物理上多个、逻辑上是一个整体；



# 分布式事务管理的实现

## 分布式事务执行的控制模型

控制什么？LTM和DTM的协同工作。

三种控制模型：

- 主从控制模型
- 三角控制模型
- 层次控制模型

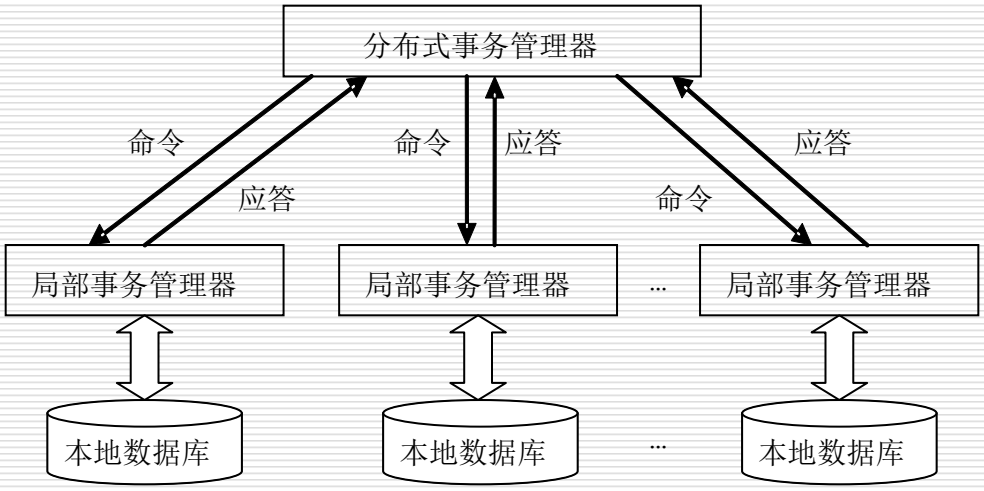
# 分布式事务管理的实现

## 分布式事务执行的控制模型

### 主从控制模型

DTM作为主控制器，LTM作为从属控制器。

一问一答方式： DTM向LTM发送命令并收集应答来实施状态监控。LTM根据DTM的命令来执行本场地的子事务，并向将DTM返回最终结果。LTM之间无直接通信（由DTM转发 ）。

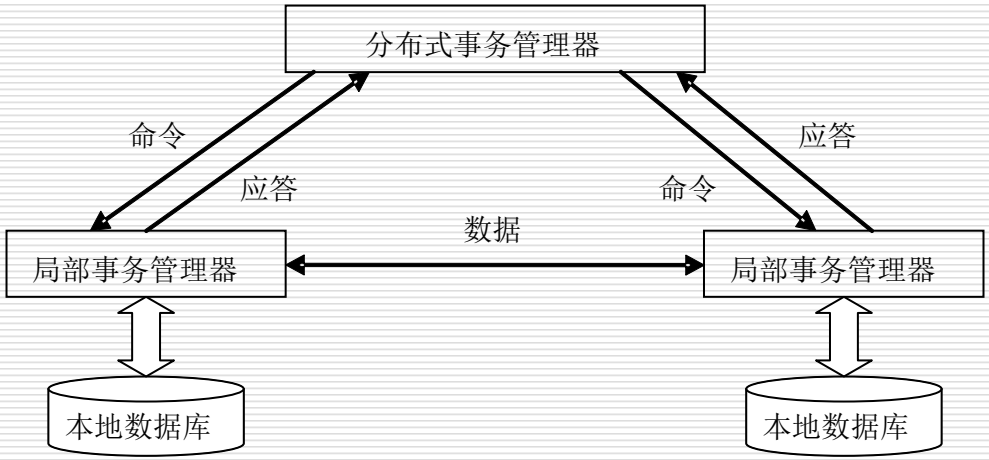


# 分布式事务管理的实现

## 分布式事务执行的控制模型

### 三角控制模型

三角控制模型与主从控制模型相类似，差别是：LTM之间可以直接通信，不需要DTM作为中介转发。因此，同主从控制模型相比，三角控制模型在一定程度上减少了通信代价，但使得DTM和LTM之间的协作控制变得更加复杂。

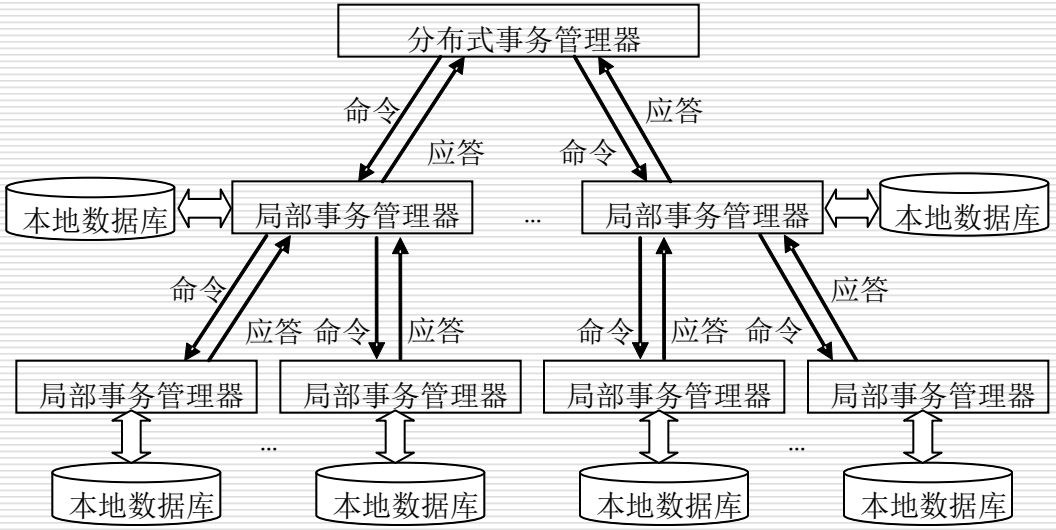


# 分布式事务管理的实现

## 分布式事务执行的控制模型

### 层次控制模型

每个LTM可以具有双重角色，除了用来管理其本地的子事务外，还可以将其负责的子事务进一步分解，衍生出一系列更下层的LTM，每个分解部分由下一层的LTM负责执行。这时，LTM自身将成为一个新的DTM，并控制更下层的LTM的执行。层次控制模型扩展性好，其层数可以随着任务量的增大而增加，但实现上更加复杂。



# 分布式事务管理的实现

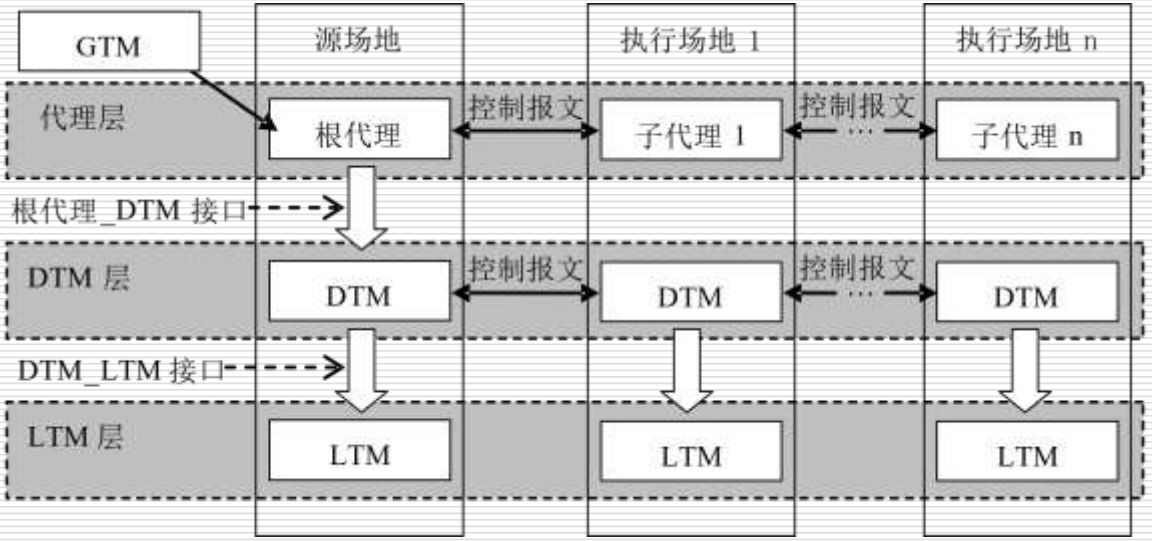
## 分布式事务管理的实现模型

实现模型的三层：

代理层：由1个根代理和若干个子代理组成，通过控制报文来协同工作；根代理包含一个GTM（全局事务管理器），负责全局事务的划分和调度。

DTM层：由各场地上的DTM组成，负责管理分布式事务，调度子事务的执行；

LTM层：由所有场地上的LTM组成，负责管理DTM交付的子事务的执行。



# 分布式事务管理的实现

## 分布式事务管理的实现模型

三层之间的通信原语：

（1）根代理_DTM接口

begin_transaction：全局事务开始；

commit：全局事务提交；

abort：全局事务废弃；

create：全局创建子代理。

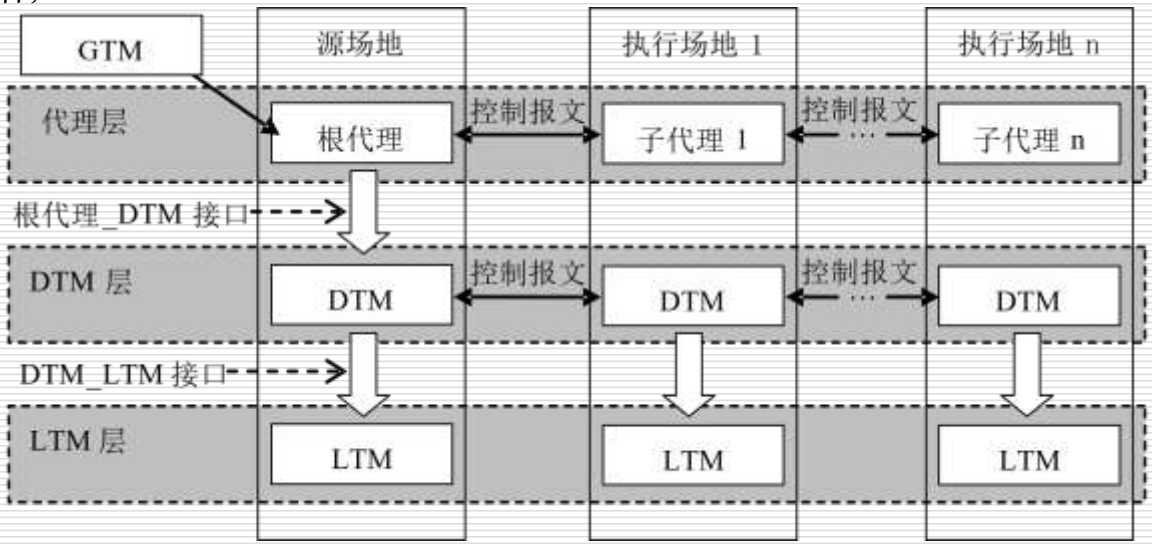
（2）DTM_LTM接口

local_begin：局部事务开始；

local_commit：局部事务提交；

local_abort：局部事务废弃；

local_create：局部创建子代理。



# 分布式事务管理的实现

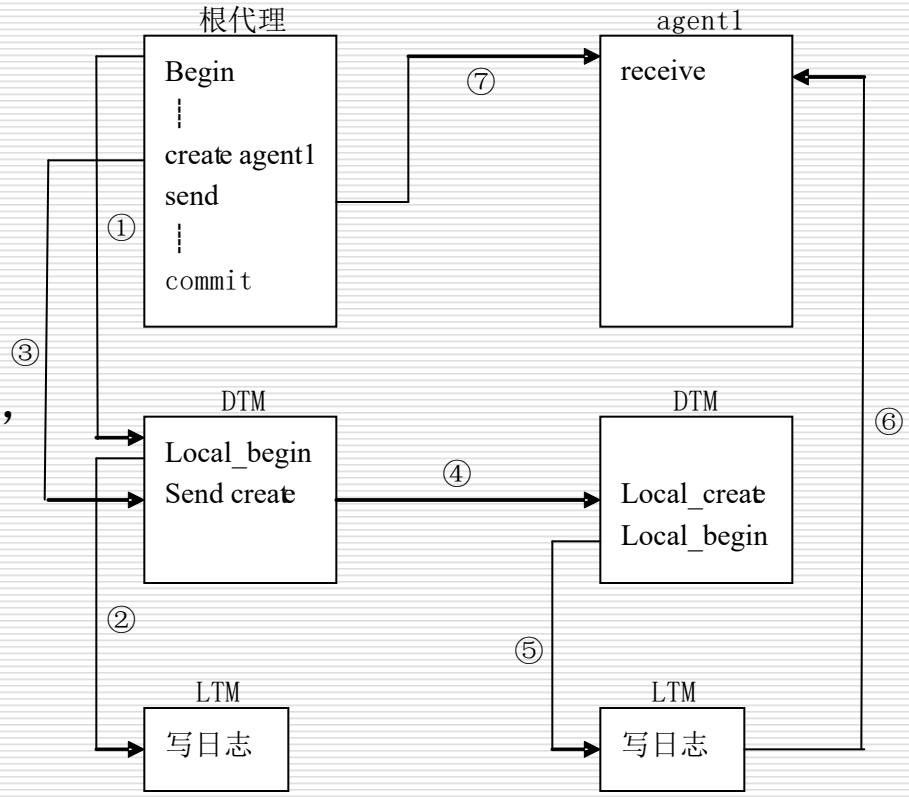
## 分布式事务管理的实现模型

执行过程：

Step①：根代理向本地DTM发送begin_transaction命令，全局事务开始；

Step②：DTM向本地LTM发送local_begin命令，启动局部事务，在局部事务执行前，先将当前局部事务信息写入日志；

Step③：根代理通过create原语创建子代理agent1；



# 分布式事务管理的实现

## 分布式事务管理的实现模型

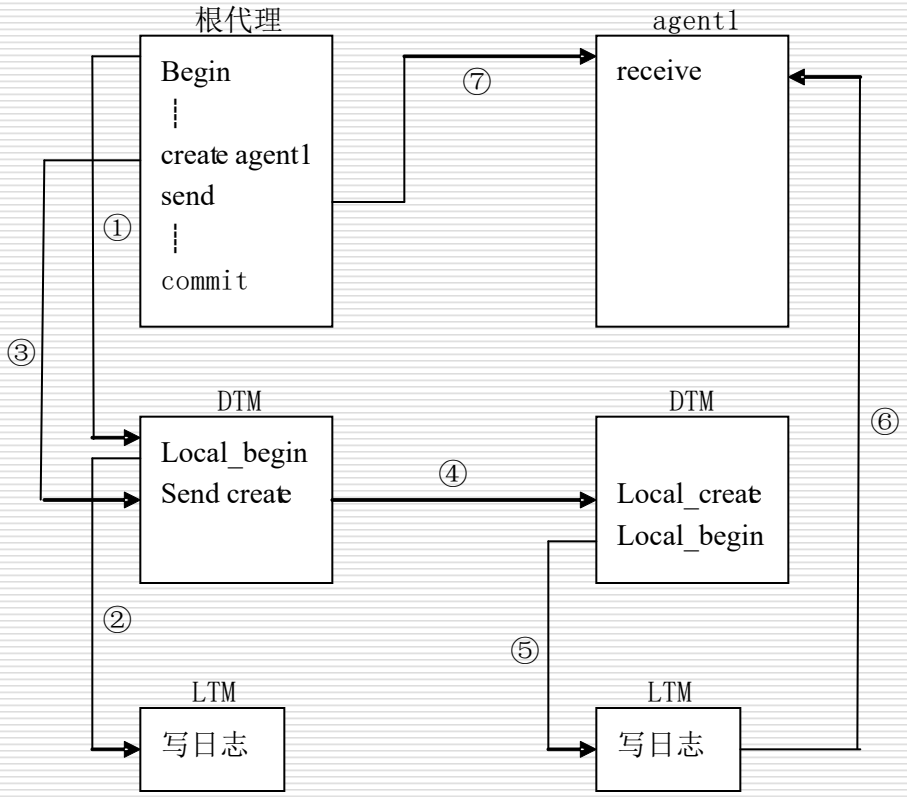
执行过程：

Step④：根代理DTM通知子代理agent1所在场地DTM，子代理agent1所在场地DTM向本地LTM发送local_begin命令，建立并启动局部事务；

Step⑤：在局部事务执行前，子代理agent1所在场地LTM将当前局部事务信息写入日志；

Step⑥：子代理agent1所在场地LTM通知并激活子代理agent1来执行子事务；

Step⑦：agent1接收参数信息。



# 两段提交协议(2PC)的实现方法

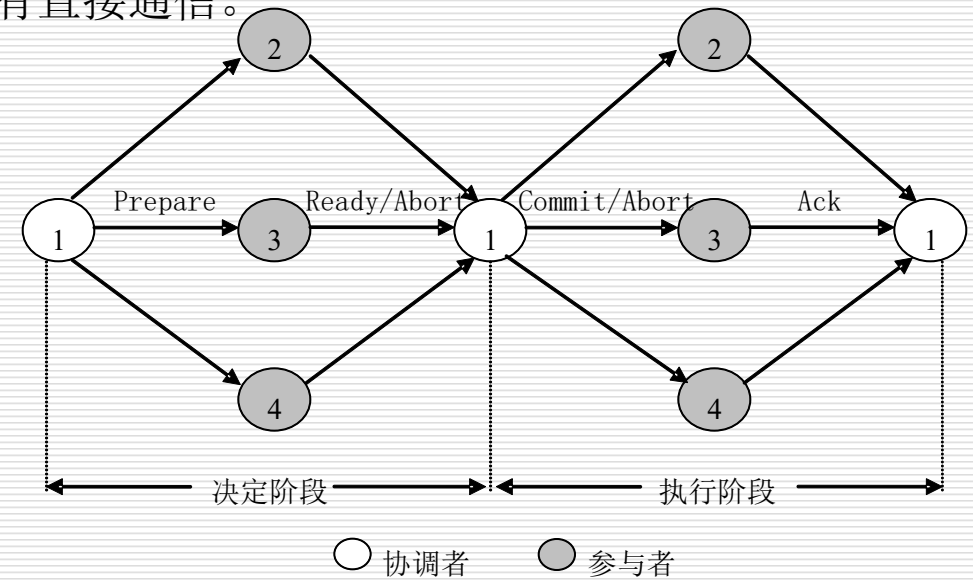
- 基本思想：分布式事务提交过程分两个阶段：
  决定阶段→执行阶段
- 实现方法：
  - 集中式2PC
  - 分布式2PC
  - 分层式2PC
  - 线形2PC

# 两段提交协议(2PC)的实现方法

## 集中式2PC

首先确定一个协调者场地，通常由事务的发起者场地（源场地）充当，完成事务提交的初始化。“集中式”主要体现在协调者在整个分布式事务提交过程中始终作为唯一的总控节点（发布命令、搜集状态信息、做出决定）。通信只发生在协调者与参与者之间，参与者与参与者之间没有直接通信。

假设参与者的数目为N，则需要传递的消息总数为4N。可见，对于集中式2PC，消息传输量随着参与者数目的增加而线性增长。假设场地间传输一个消息的平均时间为T，则集中式2PC传输消息总用时为4T。

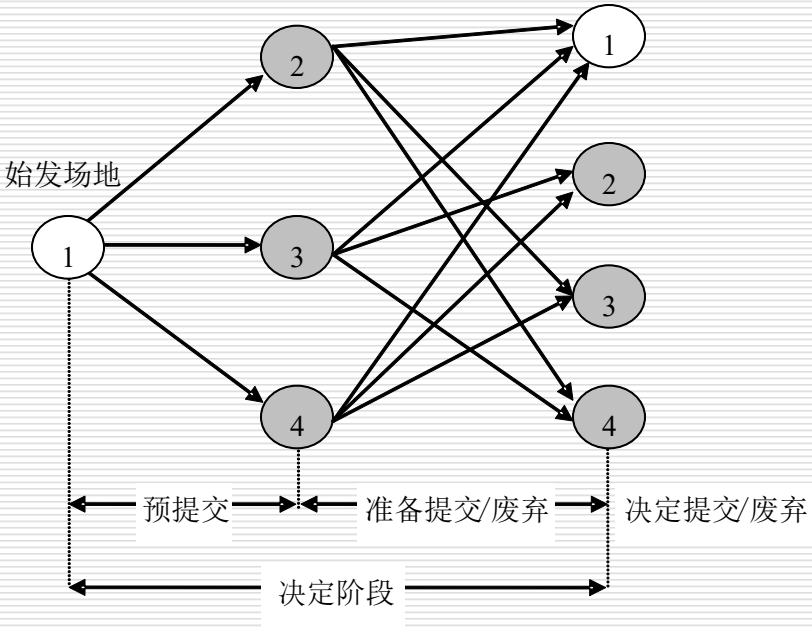


# 两段提交协议(2PC)的实现方法

## 分布式2PC

“分布式”是指事务的所有参与者同时也都是协调者，都可以决定事务的提交和废弃，提交过程是完全分布地完成。由事务的始发场地完成提交初始化工作。

始发场地初始化完毕后向所有场地广播“预提交”命令。各场地收到“预提交”命令后，做出“准备提交”或“准备废弃”的决定，并向所有场地发送应答信息。每个场地都根据接收到的所有场地的应答独立地做出决定，若存在至少一个 “准备废弃”的应答，则废弃本场地的子事务；否则，各个场地提交子事务。

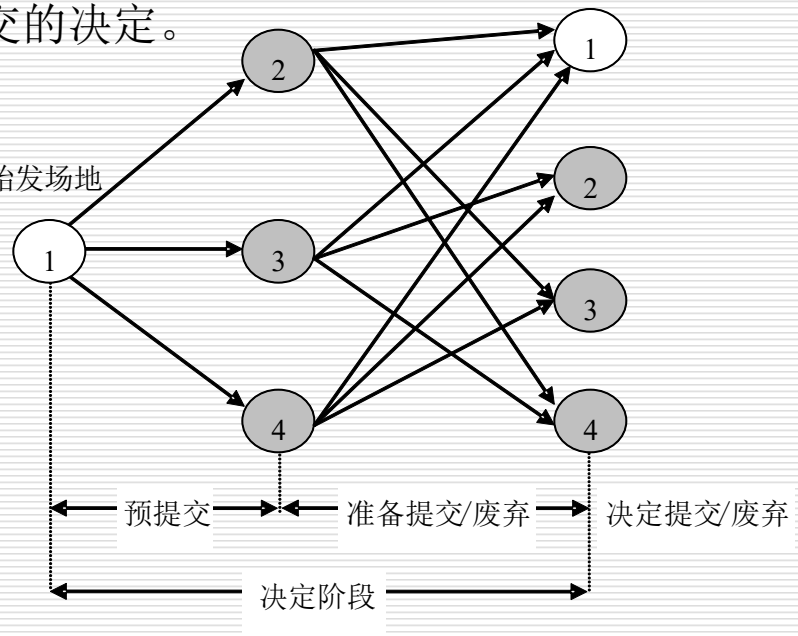


# 两段提交协议(2PC)的实现方法

## 分布式2PC

分布式2PC最大的特点是事务的提交过程只需要一个阶段，即决定阶段。所有场地都可以互相通信，使得各个场地均可以获悉其它场地的当前状态（“准备提交”或“准备废弃”），独立地做出事务是否提交的决定。

假设有N个场地（不含始发场地），则需要传递的消息总数为N×(N+1)。假设场地间传输一个消息的平均时间为T，则传输消息总用时为2T。由此可见，同集中式2PC相比，分布式2PC的响应效率高，但通信量大。

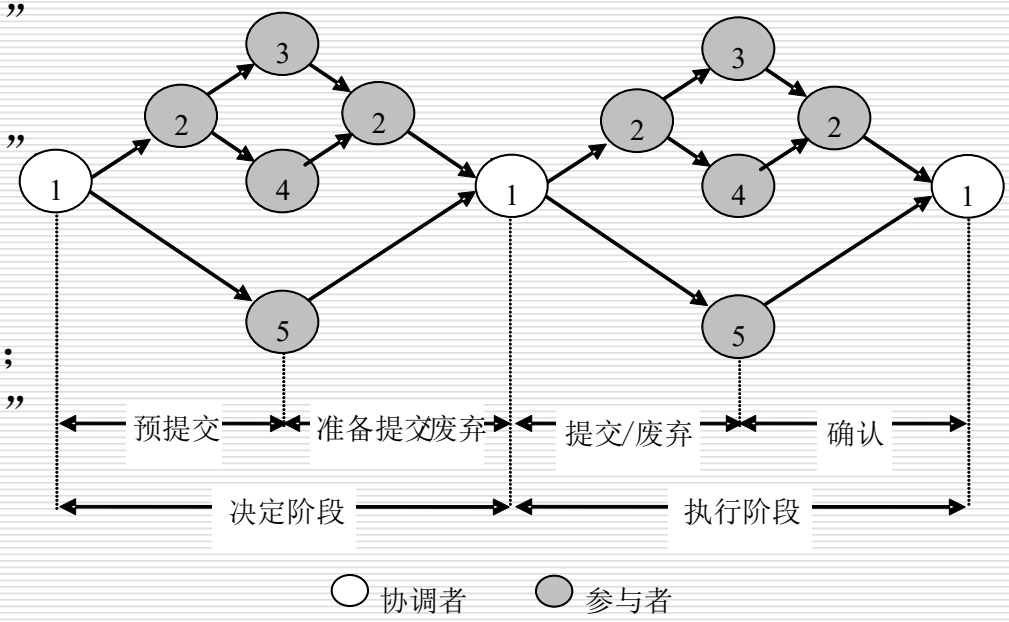


# 两段提交协议(2PC)的实现方法

## 分层式2PC

分层式2PC也称树状实现方法，协调者与参与者间的通讯结构如同一棵树。其中，协调者为根节点，参与者构成树的中间节点或叶子节点。

根节点向其下层节点下发“预提交”命令；各层节点进行转发，直到叶子节点。当叶子节点收到“预提交”命令后，根据自身状态发送应答（“准备提交”/“准备废弃”），应答信息经由中间结点到达根结点；根节点做出决定，下发“提交/废弃”命令；各层节点据此完成提交或废弃，并返回“确认”应答。

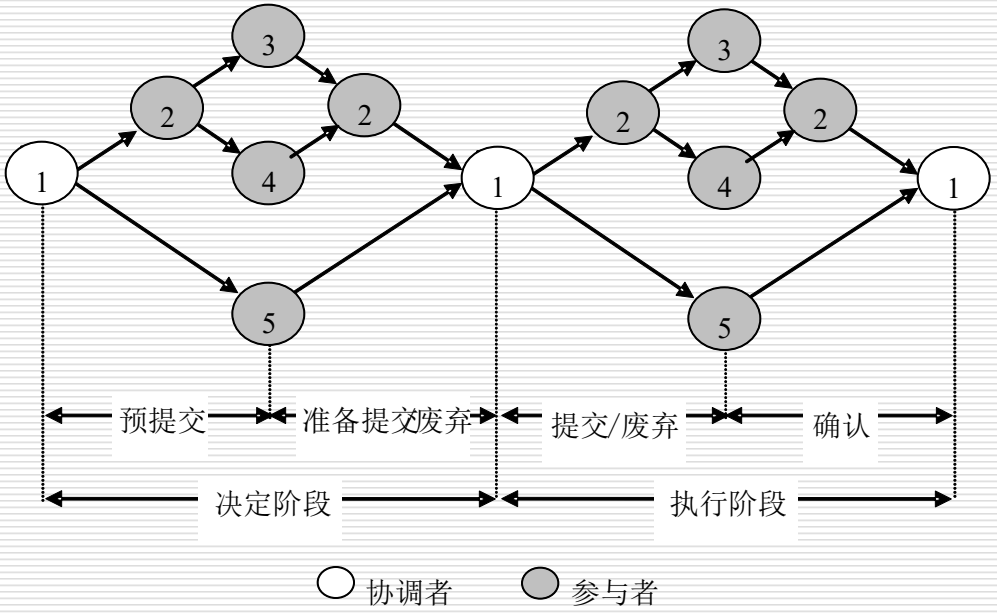


# 两段提交协议(2PC)的实现方法

## 分层式2PC

分层式2PC的特点是协调者与参与者间的通信没有采用广播方式，而是借助于树型结构将报文逐层传递。集中式2PC可以看作是分层式2PC的特例（只有两层）。

假设参与者的数目为N，则需要传递的消息总数为4N。假设场地间传输一个消息的平均时间为T，则传输消息总用时为4T~2NT。由此可见，同分布式方法相比，分层式2PC的报文传输数量少，但响应时间长。

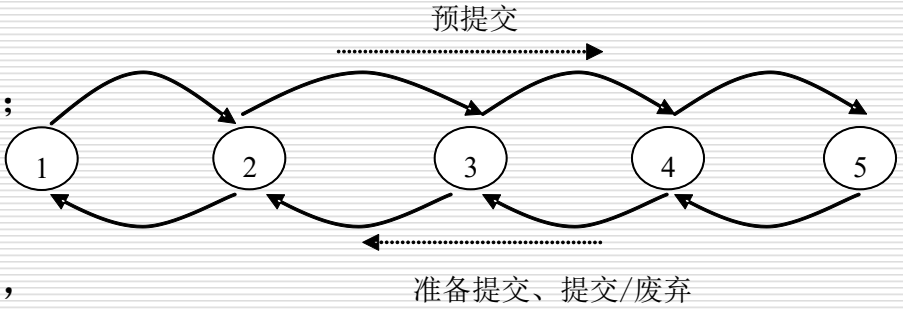


# 两段提交协议(2PC)的实现方法

## 线性2PC

在线性2PC中，允许参与者之间相互通信，由事务的始发场地构造一个线性有序的场地链表。表中第一个场地（源场地）和最后一个场地共同担任协调者，在第一个场地后，依次为第一个参与者场地，第二个参与者场地，直到第n个参与者场地。

事务的源场地进入“预提交”状态，向第一个参与者场地发送“预提交”命令；若第一个参与者场地准备好提交，则向上一个场地发“准备提交”应答，并向下一场地发“预提交”命令；以此类推，



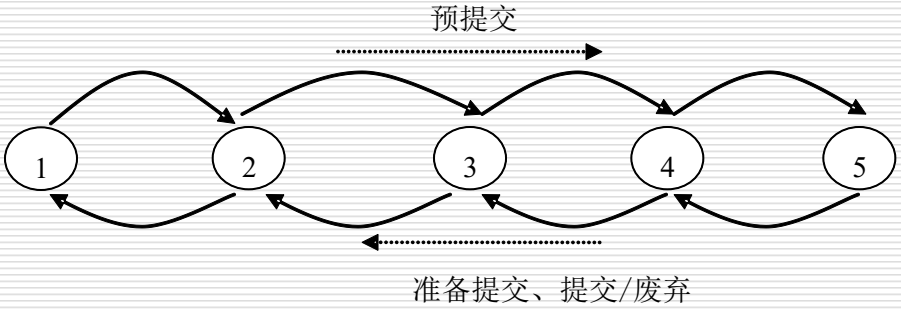

直到最后一个场地。当最后一个参与者场地收到“预提交”命令，且已准备好提交时，最后一个参与者场地充当协调者，自己首先提交，并向前一场地发提交命令，依此类推直到始发场地，事务提交完成。若任何场地收到“预提交”命令时，没准备好提交，则向前一场地发“废弃”应答，并废弃事务，依次类推直到事务始发场地，事务废弃完成。

# 两段提交协议(2PC)的实现方法

## 线性2PC

线性2PC的特点是决定阶段和执行阶段都是采用串行方式完成的。

假设参与者的数目为N，则需要传递的消息总数至多为3×(N-1)， N-1个预提交、 N-1个准备提交和N-1个提交/废弃。假设场地间传输一个消息的平均时间为T，传输消息总用时至多为2×(N-1)×T。由此可见，同其它类别的2PC相比，线性2PC的报文传输数量较少，但响应时间最长。



# 非阻塞分布式事务提交协议

## 两阶段提交协议存在的问题

如果在两段提交协议执行的过程中出现协调者故障或网络故障，那么参与者就不能及时收到协调者发出的“提交”命令，参与者处的子事务将处于等待状态。在故障恢复前，参与者子事务所占有的系统资源也不能被释放，参与者的子事务进入了阻塞状态。若参与者一直收不到协调者的命令，则子事务将始终处于阻塞状态而挂在相应的执行场地上，所占用的系统资源也不能被其它事务利用。

如何改进两段提交协议，使其成为非阻塞的分布式事务提交协议呢？

三段提交协议(3PC)，在一定程度上减少了事务阻塞的发生，提高了系统效率。

# 非阻塞分布式事务提交协议

## 三阶段提交协议的基本思想

全局事务的提交分为三个阶段：

阶段一：投票表决阶段

阶段二：准备提交阶段

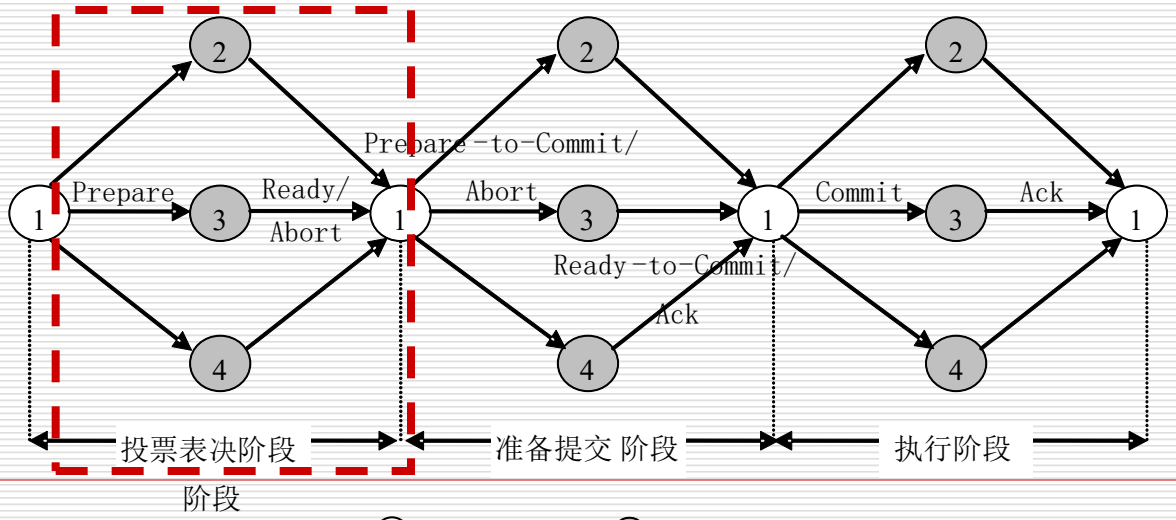
阶段三：执行阶段

# 非阻塞分布式事务提交协议

## 三阶段提交协议的基本思想

### 阶段一：投票表决阶段

由协调者向各个参与者发“预提交”（Prepare）命令，然后等待回答。每个参与者根据自己的情况进行投票，若参与者可以提交，则向协调者返回“赞成提交”（Ready）应答，否则向协调者发送“准备废弃”（Abort）应答。

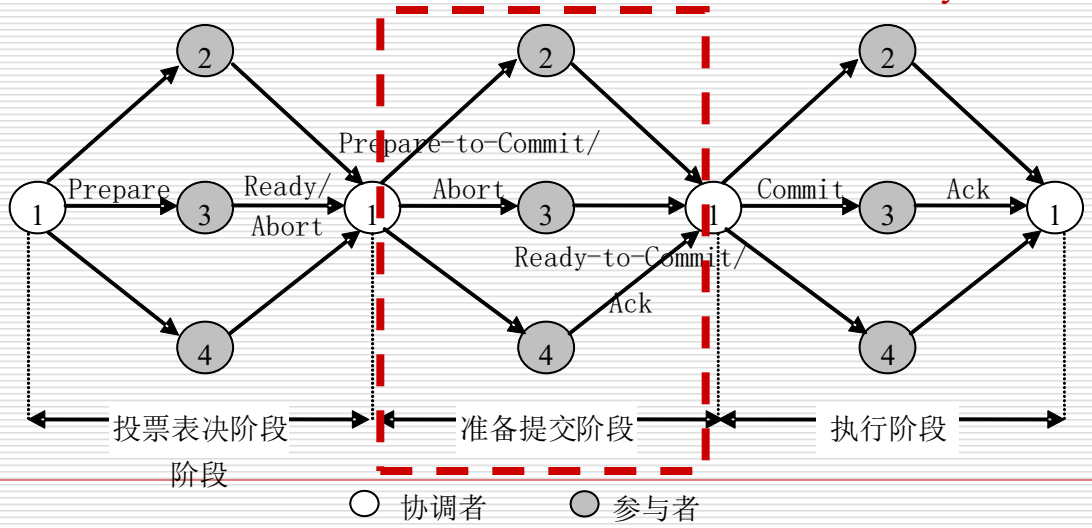


# 非阻塞分布式事务提交协议

## 三阶段提交协议的基本思想

### 阶段二：准备提交阶段

若协调者收到的应答中存在“准备废弃”（Abort）应答，则向各个参与者发“全局废弃”（Abort）命令，各个参与者执行废弃，执行完毕后向协调者发送“废弃确认”（Ack）应答；若协调者收到的应答均为“赞成提交”（Ready）应答，则向各个参与者发“准备提交”（Prepare-to-Commit）命令，然后等待回答，若参与者已准备就绪，则向协调者返回“准备就绪”（Ready-to-Commit）应答。

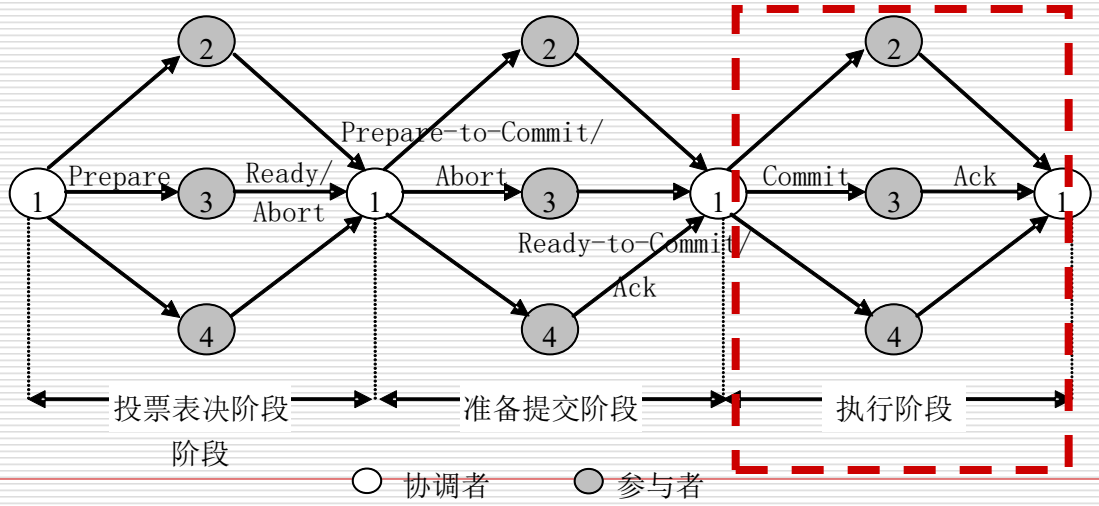


# 非阻塞分布式事务提交协议

## 三阶段提交协议的基本思想

### 阶段三：执行阶段

当协调者收到所有参与者的“准备就绪”（Ready-to-Commit）应答后，向所有参与者发送“提交”（Commit）命令，此时各个参与者已知道其它参与者均赞成提交，因此可以执行提交，提交后向协调者发送“提交确认”（Ack）应答。



# 非阻塞分布式事务提交协议

## 三阶段提交协议的基本思想

参与者的四个状态：“赞成提交”、“准备就绪”、“提交”和“废弃”

- 参与者发送完“赞成提交”应答后，处于“赞成提交”状态；
- 参与者发送完“准备就绪”应答后，处于“准备就绪”状态；
- 参与者发送完“提交确认”应答后，处于“提交”状态；
- 参与者发送完“废弃确认”应答后，处于“废弃”状态。

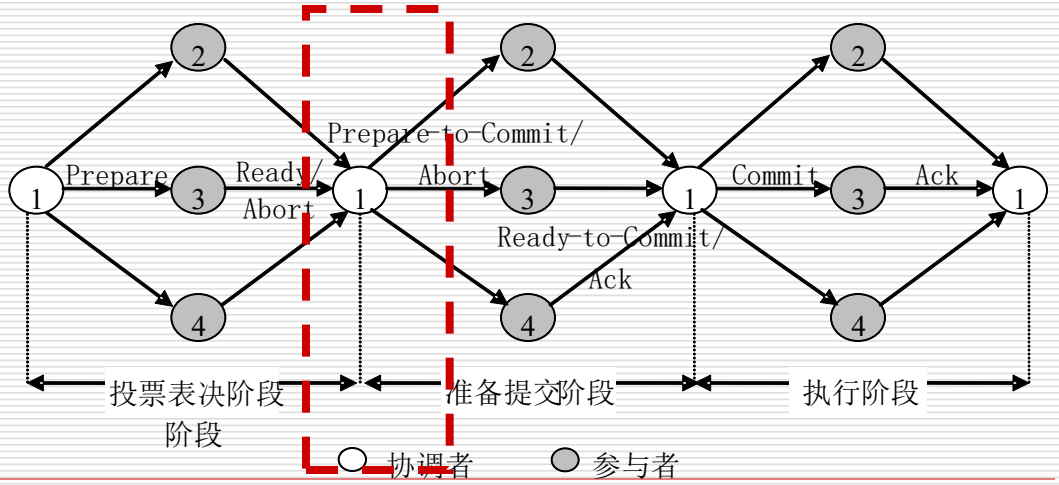
| 参与者2 / 参与者1 | 赞成提交 | 准备就绪 | 提交 | 废弃 |
|---|---|---|---|---|
| 赞成提交 | 相容 | 相容 | 不相容 | 相容 |
| 准备就绪 | 相容 | 相容 | 相容 | 不相容 |
| 提交 | 不相容 | 相容 | 相容 | 不相容 |
| 废弃 | 相容 | 不相容 | 不相容 | 相容 |

# 非阻塞分布式事务提交协议

## 三阶段提交协议的基本思想

两种故障情况下，子事务阻塞问题的解决：

参与者发送完“赞成提交”(Ready)或者“准备废弃”(Abort)的应答后，如果长时间收不到协调者发送来的“全局废弃”(Abort)或者“准备提交(Prepare-to-Commit)”命令，那么参与者启动恢复处理进程；

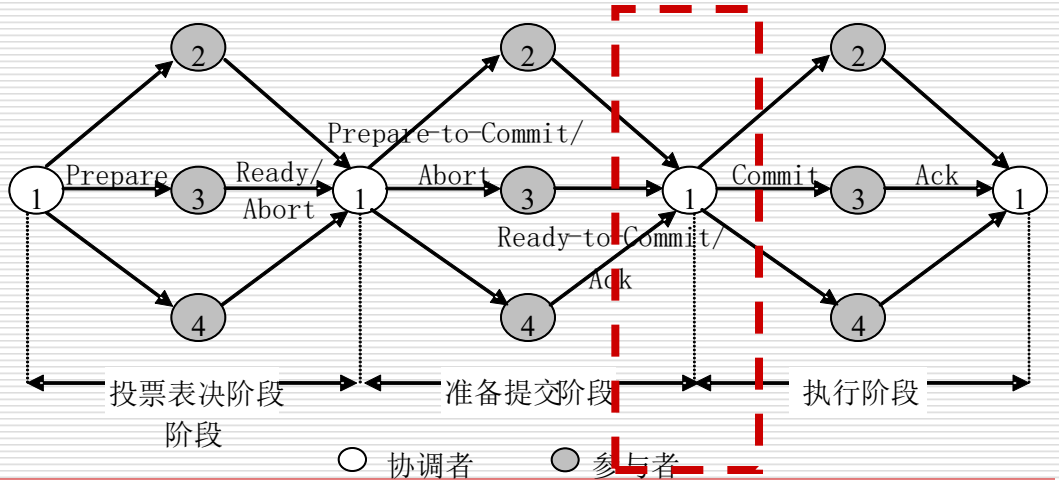


# 非阻塞分布式事务提交协议

## 三阶段提交协议的基本思想

两种故障情况下，子事务阻塞问题的解决：

参与者已发送完“准备就绪”(Ready-to-Commit)应答后，长时间没有收到协调者发来的“提交”(Commit)命令，参与者启动恢复处理过程；



# 非阻塞分布式事务提交协议

## 三阶段提交协议的基本思想

恢复处理的具体过程：

参与者进入恢复处理后，访问其它参与者的当前状态。

(赞成提交、准备就绪)

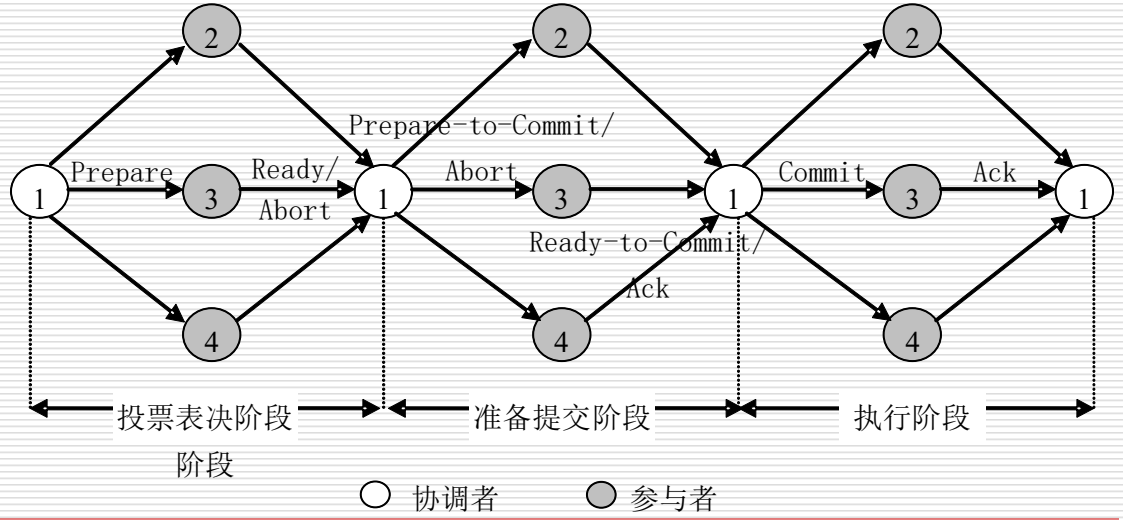
(赞成提交、提交)

(赞成提交、废弃)

(准备就绪、提交)

(准备就绪、废弃)

(提交、废弃)



# 非阻塞分布式事务提交协议

## 三阶段提交协议的基本思想

恢复处理的具体过程：

- (赞成提交、废弃)或者(赞成提交)或者(废弃)

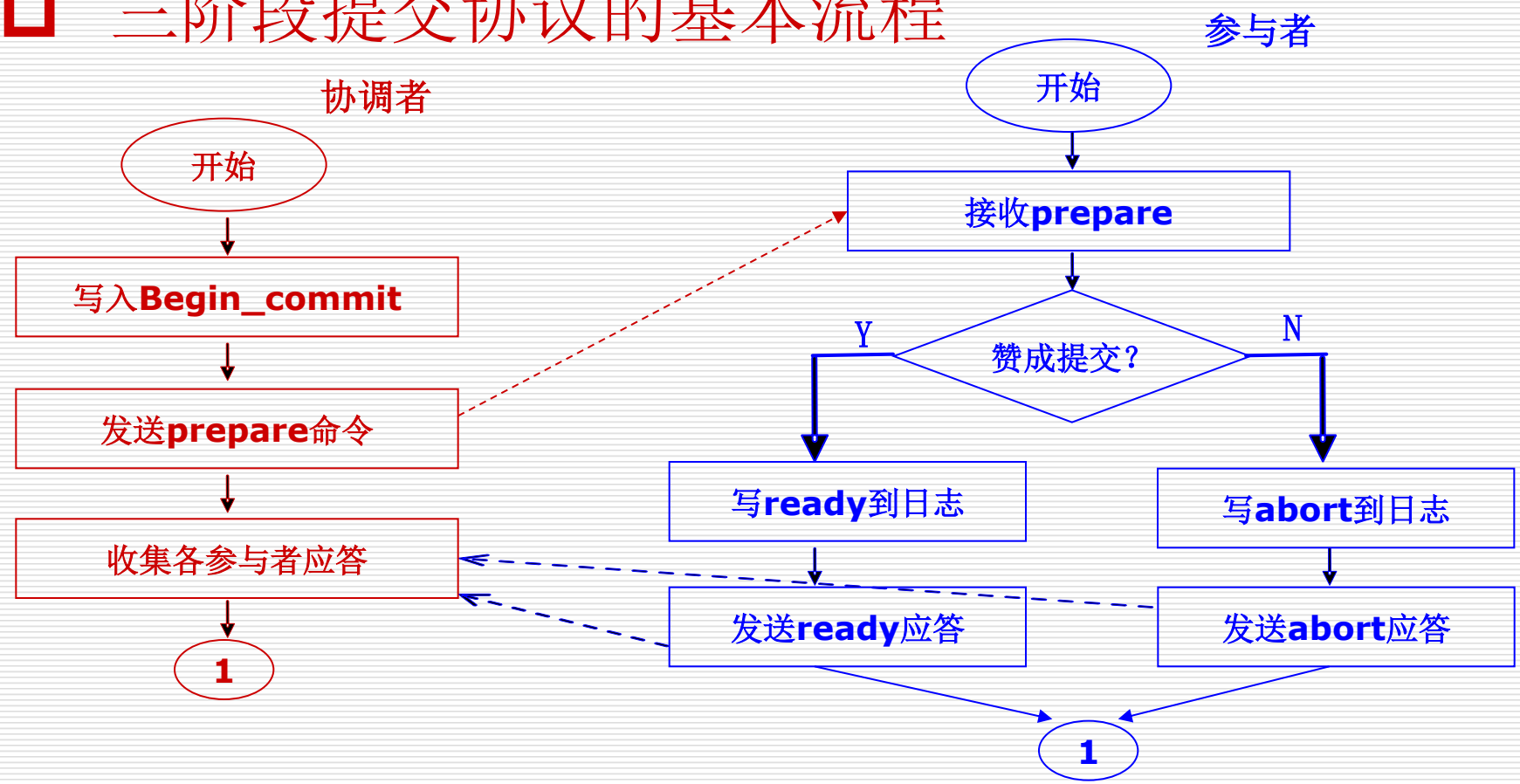
参与者进入恢复处理后，访问其它参与者的当前状态。若某些或者全部参与者处于“赞成提交”或“废弃”状态，则该参与者通知所有参与者进行废弃。

- (赞成提交、准备就绪)或者(准备就绪、提交)或者(准备就绪)

若存在某些参与者处于(赞成提交、准备就绪)或者(准备就绪、提交)状态，或者全部参与者都处于(准备就绪)状态，此时该参与者通知所有参与者提交。注意，若某参与者当前状态为“赞成提交”，则需要先将其转化为“准备就绪”状态，然后再进行提交，进而进入“提交”状态。

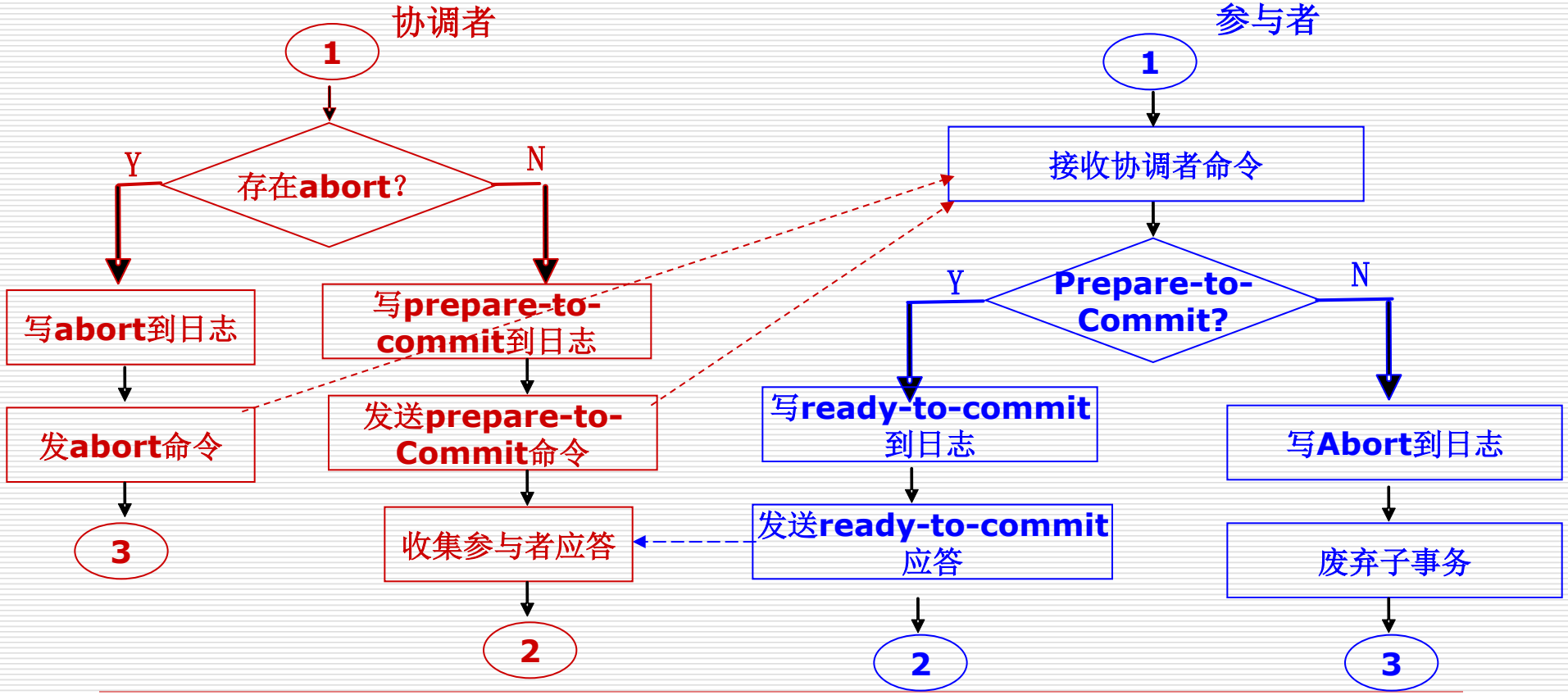
# 非阻塞分布式事务提交协议

## 三阶段提交协议的基本流程



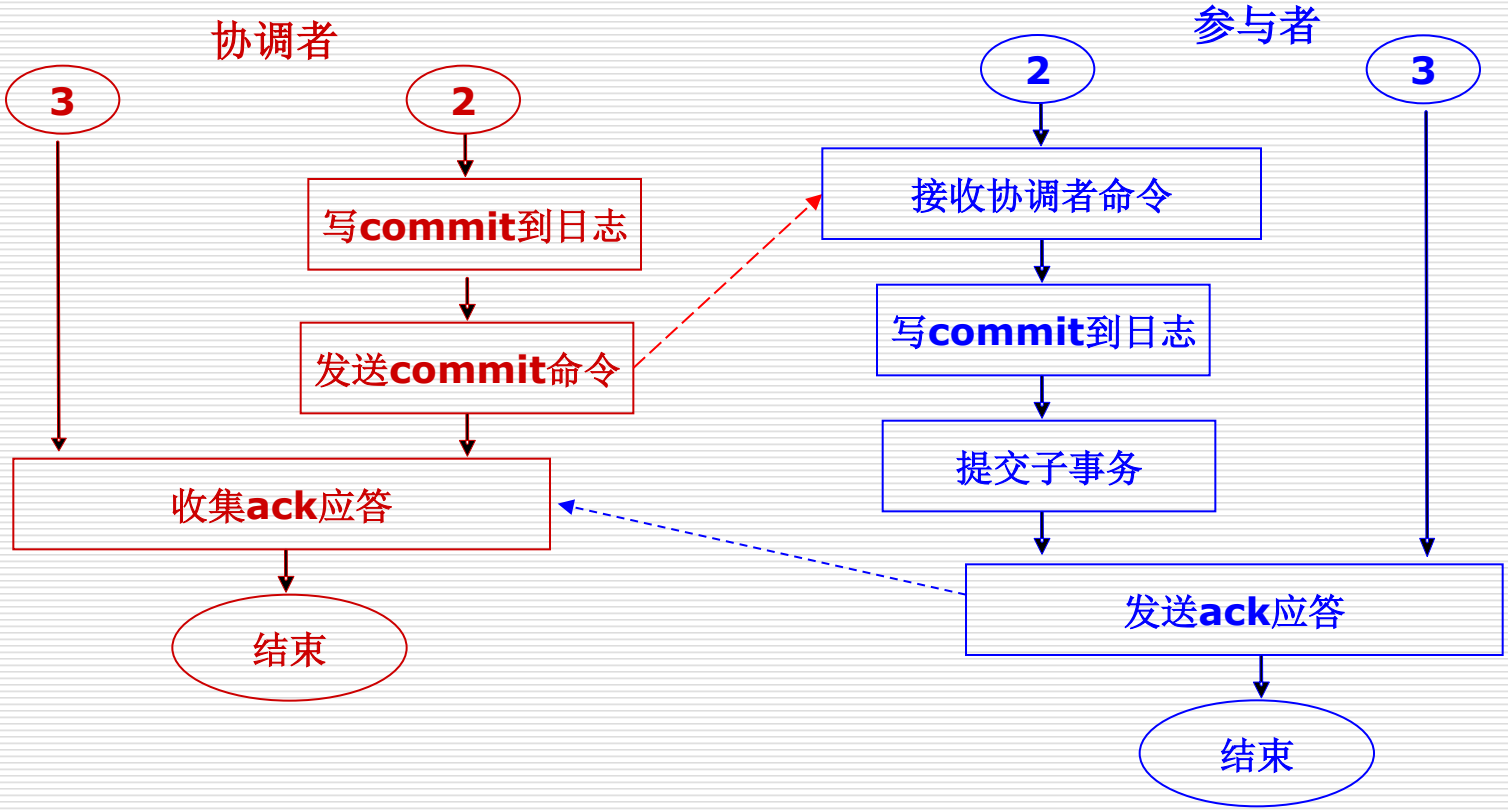
# 非阻塞分布式事务提交协议

## 三阶段提交协议的基本流程



# 非阻塞分布式事务提交协议

- 三阶段提交协议的基本流程



# 2.分布式恢复管理

- 分布式恢复概述
- 集中式数据库的故障恢复
- 分布式事务的故障恢复
- 分布式可靠性协议

# 分布式恢复概述

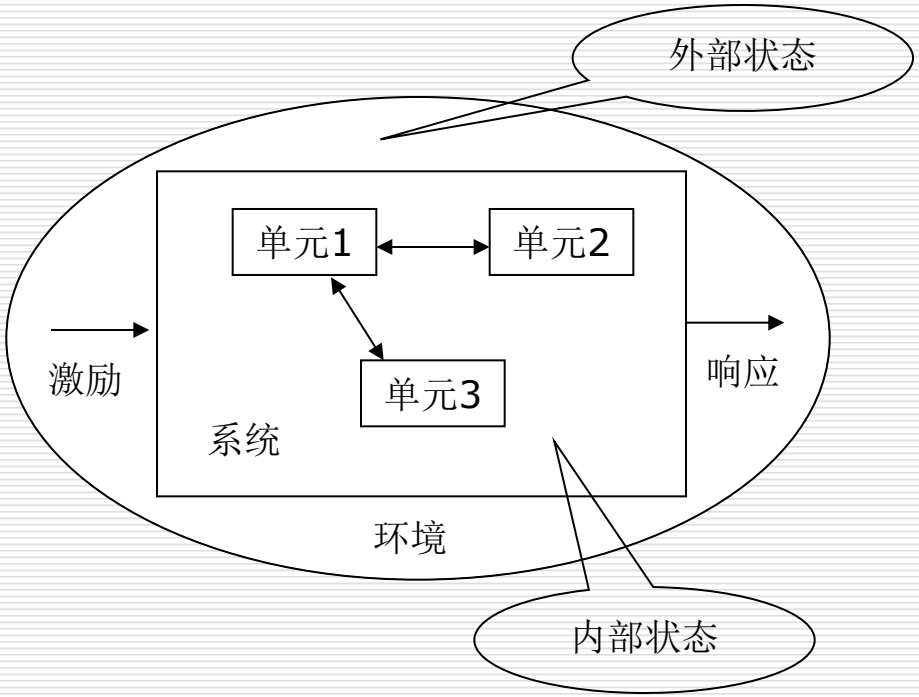
数据库系统的可恢复性和高可靠性是保证各种应用正确而可靠地运行所不可缺少的重要组成部分。

- 可恢复性，提供恢复措施，把数据库系统从故障状态恢复到一个已知的正确状态；
- 高可靠性，尽量将崩溃后数据库的不可用的时间减少到最低，并保证事务的原子性和耐久性。

# 分布式恢复概述

## 故障模型

系统的组成




故障的三种形式

- 故障（Fault）
- 错误（Error）
- 失效（Failure）

# 分布式恢复概述

## 故障模型

故障（Fault）：系统单元内部发生的错误或系统内部设计错误；
错误（Error）：系统单元内出现了不正确的状态，是故障的内在表现形式；
失效（Failure）：指系统的外部状态中所表现出来的错误；

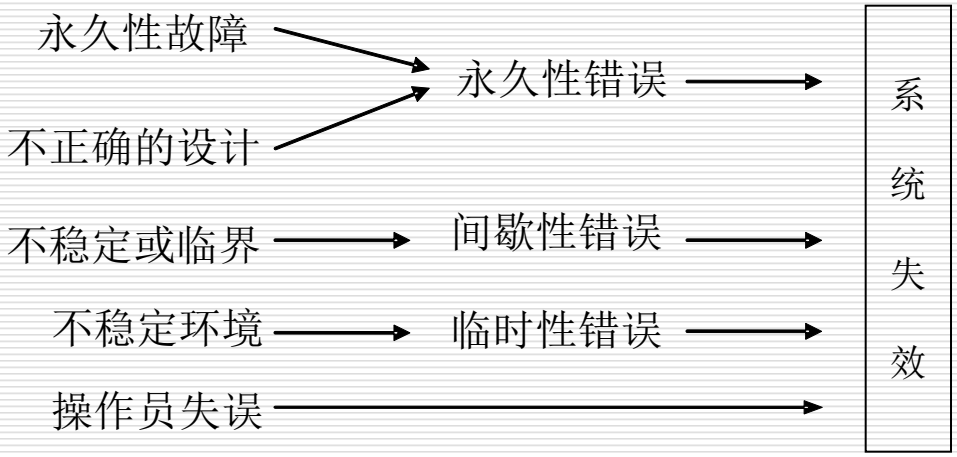



当系统单元组建不合理或系统内部设计存在不足时，将会引发系统故障，此时系统的内部状态处于错误的状态，进而使系统的外部环境受到影响，最终产生失效。

# 分布式恢复概述

## 故障模型



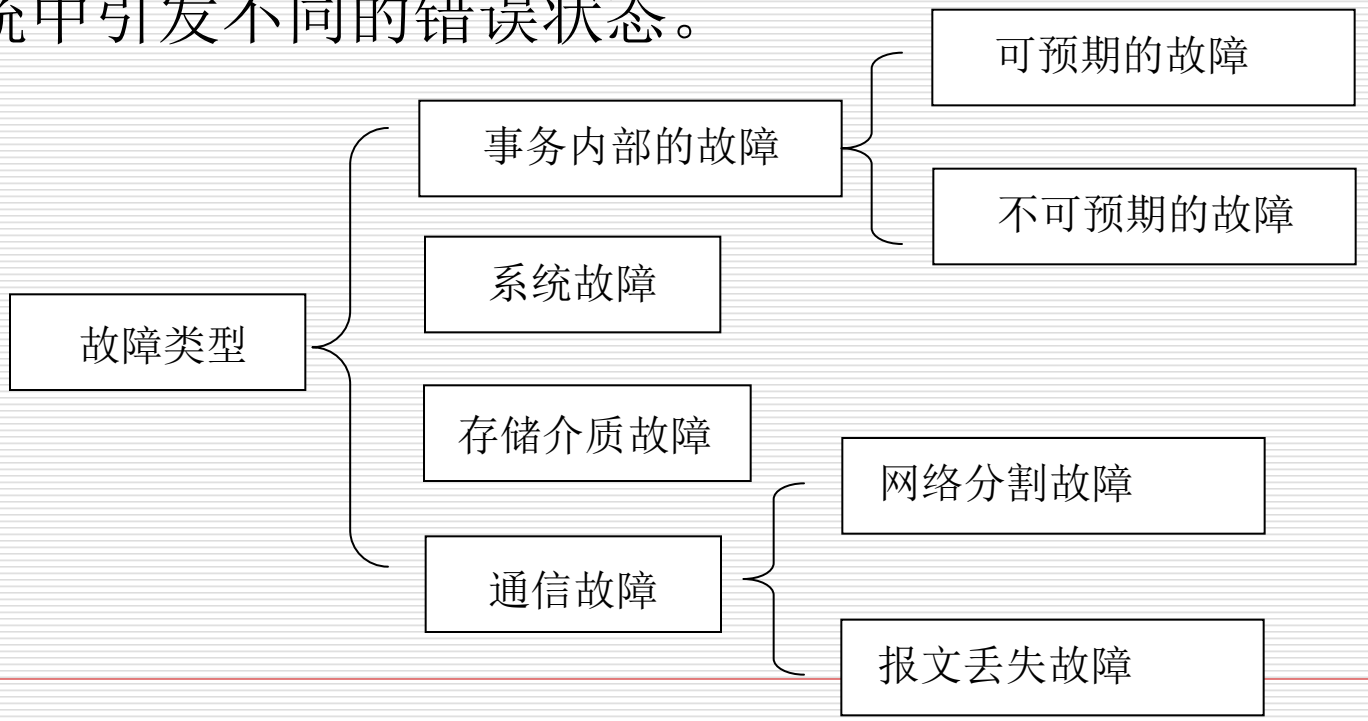




- 永久性错误：会一直持续下去，不会自动恢复，如硬盘损坏等。
- 间歇性错误：时常发生，会自动恢复，如电路接触故障、计算机运行不稳定等。
- 暂时性错误：可能发生一次便不再发生，如偶然电流干扰导致的瞬间读写错误。

# 分布式恢复概述

## □ 故障模型

数据库系统中可能发生各种各样的故障，每种故障都会在系统中引发不同的错误状态。



# 分布式恢复概述

## □ 故障模型

数据库系统中的故障通常分为4类：

（1）事务内部的故障

事务内部的故障可细分为可预期的和不可预期的。

可预期的事务故障是指故障的发生可以通过事务程序本身来检测。例如：在转帐事务中，事务主动检查贷方余额是否足够支持转帐，若发现不足，则废弃该子事务，数据库状态没有因为此事务发生变化；

不可预期的事务故障是指故障的发生不能被应用程序所检测并处理。例如：死锁、算术溢出、完整性被破坏、操作员失误等故障都是不可预期的事务故障，这类故障无法由事务程序本身所预测。

事务内部的故障大多数都是不可预期的，发生这类故障的事务将不能正常运行到其终点位置（Commit或Abort）。因此，针对这类故障，数据库恢复机制要强行废弃该事务，使数据库回滚到事务执行前的状态。

# 分布式恢复概述

## □ 故障模型

数据库系统中的故障通常分为4类：

（2）系统故障

- **表现：**使系统停止运转，必须经过重启后系统才能恢复正常。例如，CPU故障、系统死循环、缓冲区溢出、系统断电等。
- **特点：**仅使正在运行的事务受到影响，但数据库本身没有被破坏；内存中的数据全部丢失。一方面，一些尚未完成的事务的结果可能已被写入数据库中；另一方面，一些已提交的事务的结果可能还未更新到磁盘上。
- **结果：**系统故障发生后数据库可能处于不一致的状态。

对于系统故障，数据库恢复机制要在系统重启后，将所有非正常终止的事务强行废弃，同时将已提交的事务的结果重新更新到数据库中，以保证数据库的正确性。

# 分布式恢复概述

## □ 故障模型

数据库系统中的故障通常分为4类：

（3）存储介质故障

**存储介质故障**是指存储数据的磁盘等硬件设备发生的故障。例如，磁盘坏损、磁头碰撞、瞬时强磁场干扰等均为存储介质故障。

**特点：**不仅使正在运行的所有事务受到影响，而且数据库本身也被破坏。

**结果：**同前两种故障相比，存储介质故障是一种较严重的故障类型。

对于存储介质故障，数据库恢复机制要定期地对数据库进行转储，借助于备份数据库和日志文件来进行故障恢复。

# 分布式恢复概述

## 故障模型

数据库系统中的故障通常分为4类：

（4）通信故障

前三个故障都是单场地故障、也是集中式数据库中的故障。对于分布式数据库而言，还要进行各个场地的通信，因此还会产生通信故障。

通信故障可细分为网络分割故障和报文丢失故障。

- 网络分割是指通讯网络中一部分场地和另一部分场地之间完全失去联系。
- 报文丢失是指报文本身错误或在传送过程中丢失而导致数据不正确。

对于通信故障，数据库恢复机制要针对故障的表现形式进行判断，分析故障产生原因并进行相应的处理，如重发报文等。

# 分布式恢复概述

## 故障模型

事务内部故障

系统故障

通信故障

存储介质故障

软故障—临时性的、间歇性的，导致软失效。

硬故障—永久性的、不能自行恢复、致命的，导致硬失效。

# 分布式恢复概述

## 恢复模型

在故障恢复过程中，数据库恢复管理器依据数据库日志文件（log）对数据库事务进行恢复操作。

- 日志文件的格式和内容
- 反做(undo)和重做(redo)恢复策略
- 故障恢复模型

# 分布式恢复概述

## □ 恢复模型

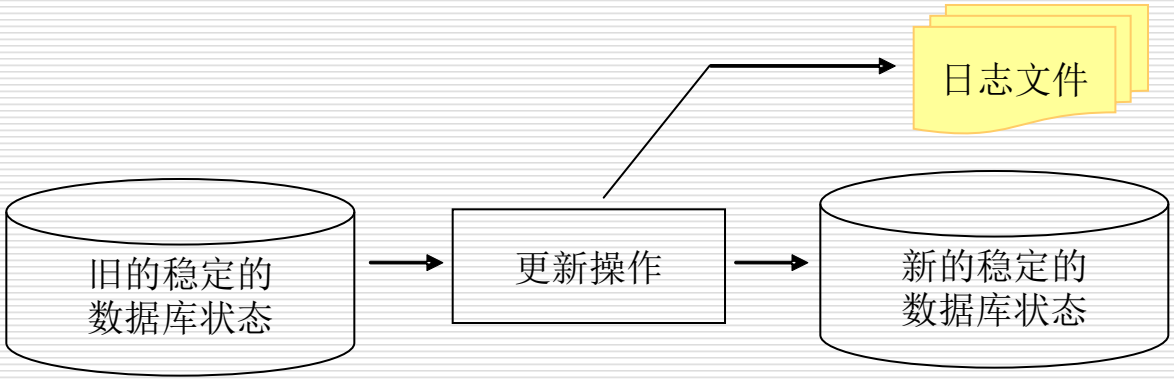
### ◆ 数据库的日志文件

内容：记录事务的执行信息。

作用：① 用来进行数据库的恢复；

② 数据库使用情况统计和分析的数据源。

有多重要？先写日志原则(WAL, Write Ahead Log)。



# 分布式恢复概述

## 恢复模型

### 数据库的日志文件

在日志中，系统的运行情况通常以运行记录的形式存放。日志记录可分为：

- 数据日志记录
- 命令日志记录
- 检查点日志记录

# 分布式恢复概述

## 恢复模型

### 数据库的日志文件

数据日志记录包括：

- 事务标识符（标明是哪个事务）
- 操作类型（插入、删除或修改）
- 操作的数据项
- 数据项的旧值（前像，BeforeImage）
- 数据项的新值（后像，AfterImage）

# 分布式恢复概述

## 恢复模型

### 数据库的日志文件

命令日志记录包括：

- 事务标识符（标明是哪个事务）；
- 命令（如：Begin、Abort或Commit）。

# 分布式恢复概述

## □ 恢复模型

### ◆ 数据库的日志文件

#### 检查点记录

检查点是在日志中周期设定的操作标志，目的是减少系统故障后的恢复的工作量。在检查点上，需要完成的操作包括：

**首先，**将日志缓冲区中的内容写入外存中的日志；

**然后，**在外存日志中登记一个检查点记录；

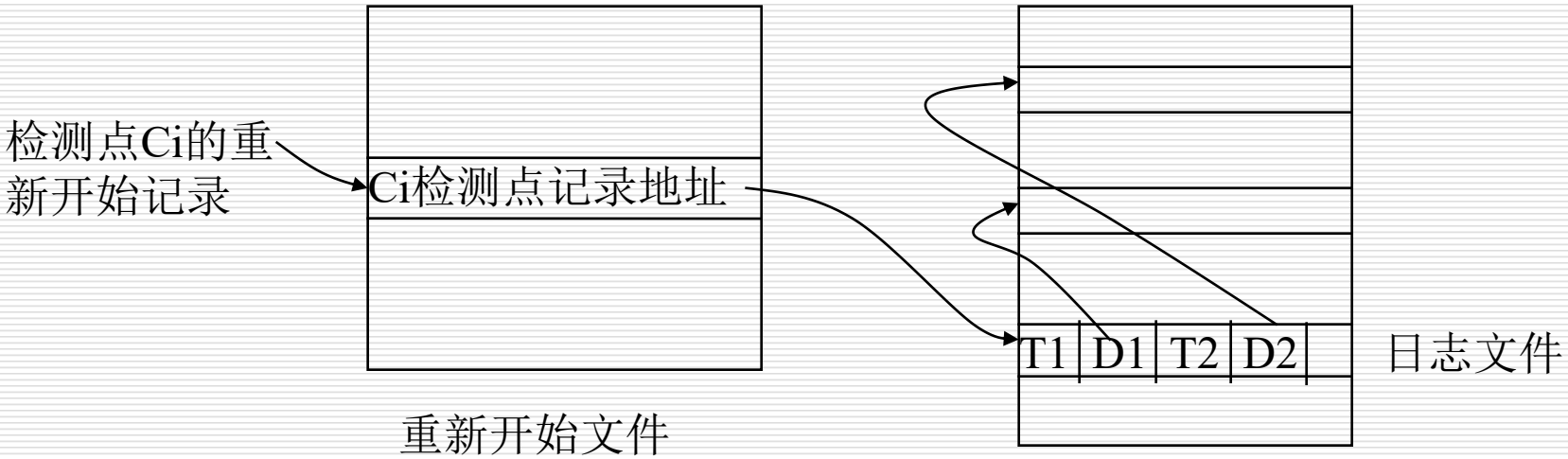
**接下来，**将数据库缓冲区的内容写入外存数据库；

**最后，**把外存日志中检查点的地址写入重启动文件。

上述工作的目的就是把检查点之前的工作持久化。

# 分布式恢复概述

- 检测点记录的内容
  - 建立检测点时刻所有正在执行的事务清单
  - 这些事务最近一个日志记录的地址

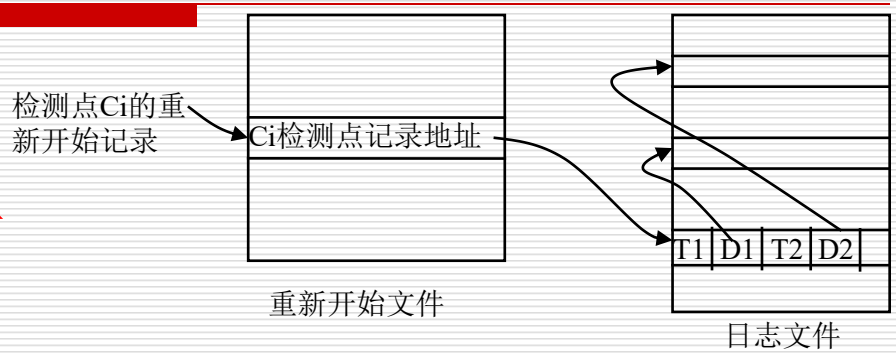


# 分布式恢复概述

## 恢复模型

### 数据库的日志文件

检查点记录



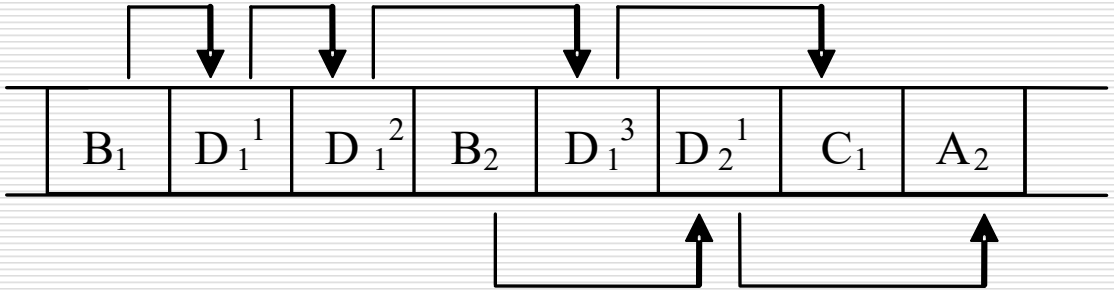

当系统出现故障时，只需要进行如下恢复处理： 在重启动文件中找到最近的一次检查点记录在日志中的位置，然后在日志中找到最近的一个检查点。对最近的检查点以后的提交操作进行恢复处理；对最近检查点没提交的活动事务的操作进行恢复处理；对检查点以后没有提交的事务的操作进行恢复处理。

可见，基于检查点的恢复处理可有效减少恢复的工作量（只需要恢复最近一个检查点之后的数据库操作）。

# 分布式恢复概述

## 恢复模型

### 数据库的日志文件




其中，$D_i^j$表示事务i对数据D的第j步操作，$B_i$表示事务i开始执行(begin)，$C_i$和$A_i$分别表示事务i的提交(commit)与废弃(abort)。

$T_1$的执行过程为：$B_1 \rightarrow D_1^1 \rightarrow D_1^2 \rightarrow D_1^3 \rightarrow C_1$

$T_2$的执行过程为：$B_2 \rightarrow D_2^1 \rightarrow A_2$

# 分布式恢复概述

## 恢复模型

### 反做(undo)和重做(redo)恢复策略

**什么是反做？**

- 反做（undo）也称撤消、回滚（rollback），是将一个数据项的值恢复到其修改之前的值，即取消一个事务所完成的操作结果。
- 当一个事务尚没提交时，如果缓冲区管理器允许该事务修改过的数据写到外存数据库，一旦此事务出现故障需废弃时，就需对被这个事务修改过的数据项进行反做，即根据日志文件将其恢复到前像。反做的目的是保持数据库的原子性。

反做幂等律：undo（undo（...T））） = undo（T）



机械工业出版社
CHINA MACHINE PRESS

# 分布式恢复概述

## □ 恢复模型

### ◆ 反做(undo)和重做(redo)恢复策略

**什么是重做？**

- 重做（redo）也称为前滚操作（rollforward），是将一个数据项的值恢复到其修改后的值，即恢复一个事务的操作结果。
- 当一个事务提交时，如果缓冲区管理器允许该事务修改过的数据不立刻写到外存数据库，一旦此事务出现故障，需对被这个事务修改过的数据项进行重做（redo），即根据日志文件将其恢复到后像。重做（redo）的目的是保持数据库的持久性。

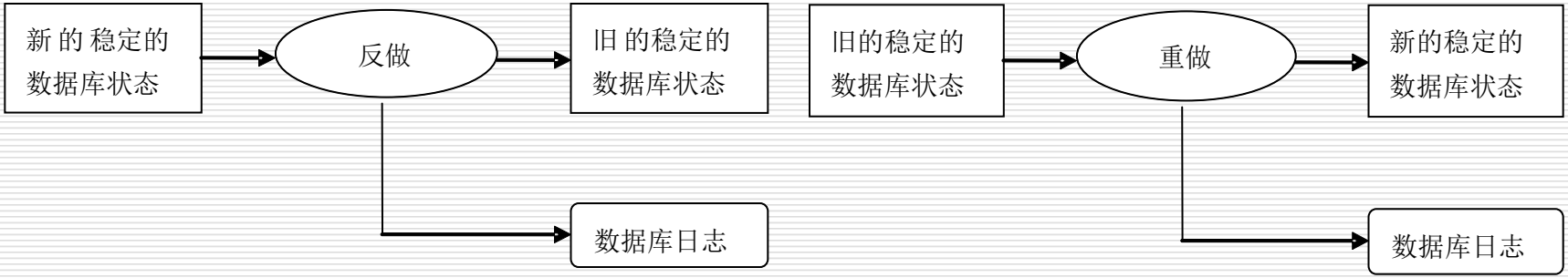
重做幂等律：redo（redo（...T））） = redo（T）

# 分布式恢复概述

## 软/硬故障的恢复模型

### 软故障的恢复模型

当软故障发生时，造成数据库不一致状态的原因包括：一、一些未完成事务对数据库的更新已写入外存数据库；二、一些已提交事务对数据库的更新还没来得及写入外存数据库。因此，需要的恢复操作有undo和redo。



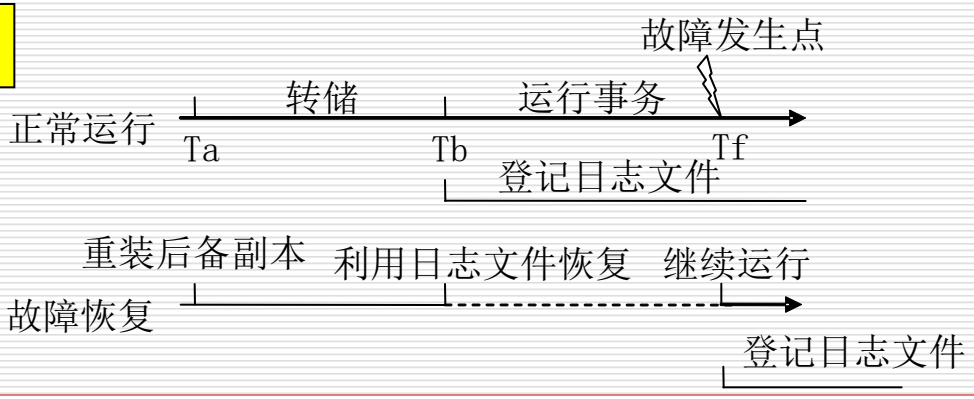
# 分布式恢复概述

## 软/硬故障的恢复模型

### 硬故障的恢复模型

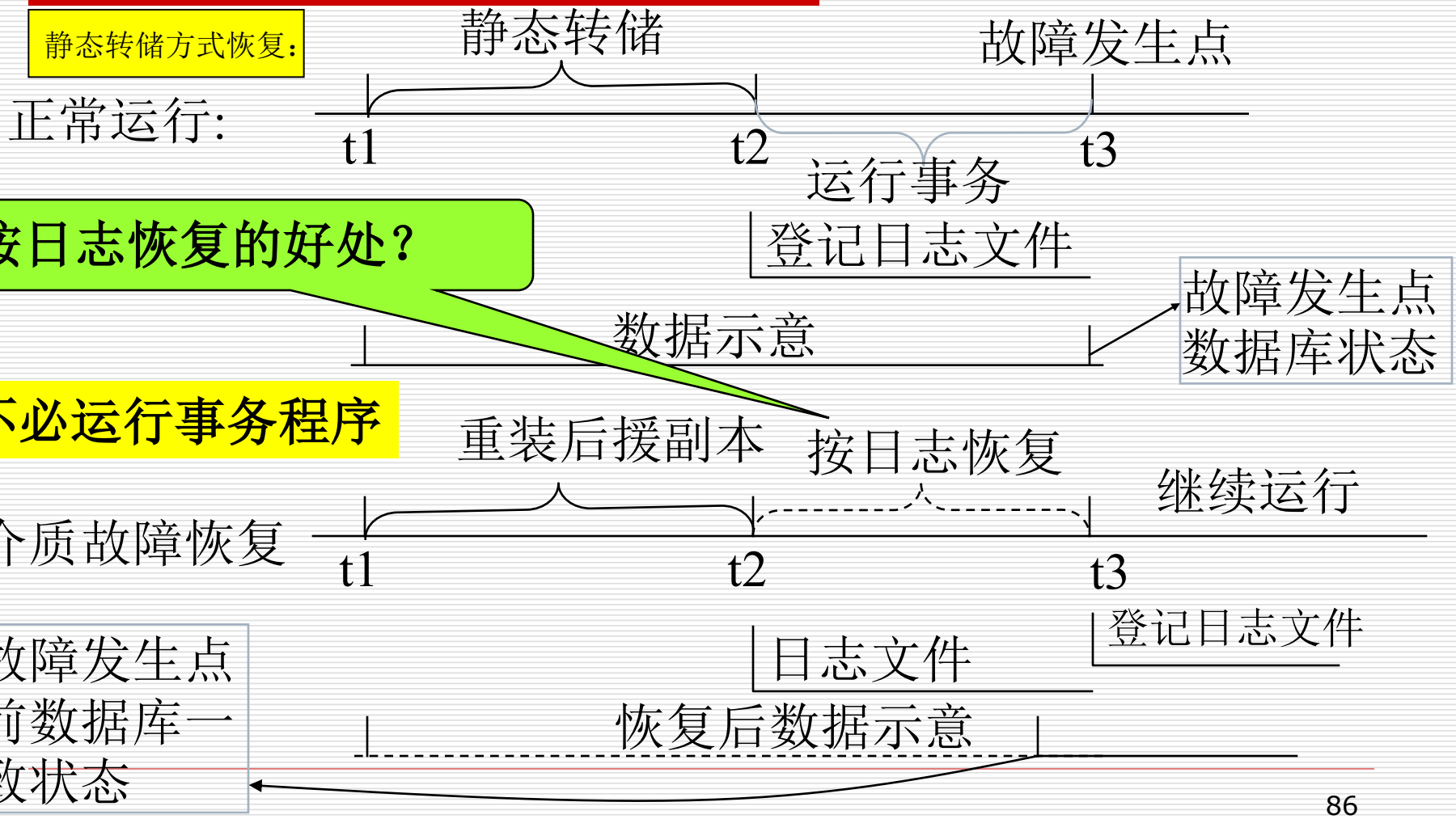
硬故障的主要恢复措施是进行数据转储和建立日志文件。首先，DBA要定期地将数据库转储到其它磁盘上，形成一系列备份数据库（也称“后备副本”）。接下来，利用日志文件重新运行转储以后的所有更新事务，使数据库再进一步地恢复到故障发生时的状态。

静态转储方式恢复：



# 分布式恢复概述

静态转储方式恢复：




按日志恢复的好处？

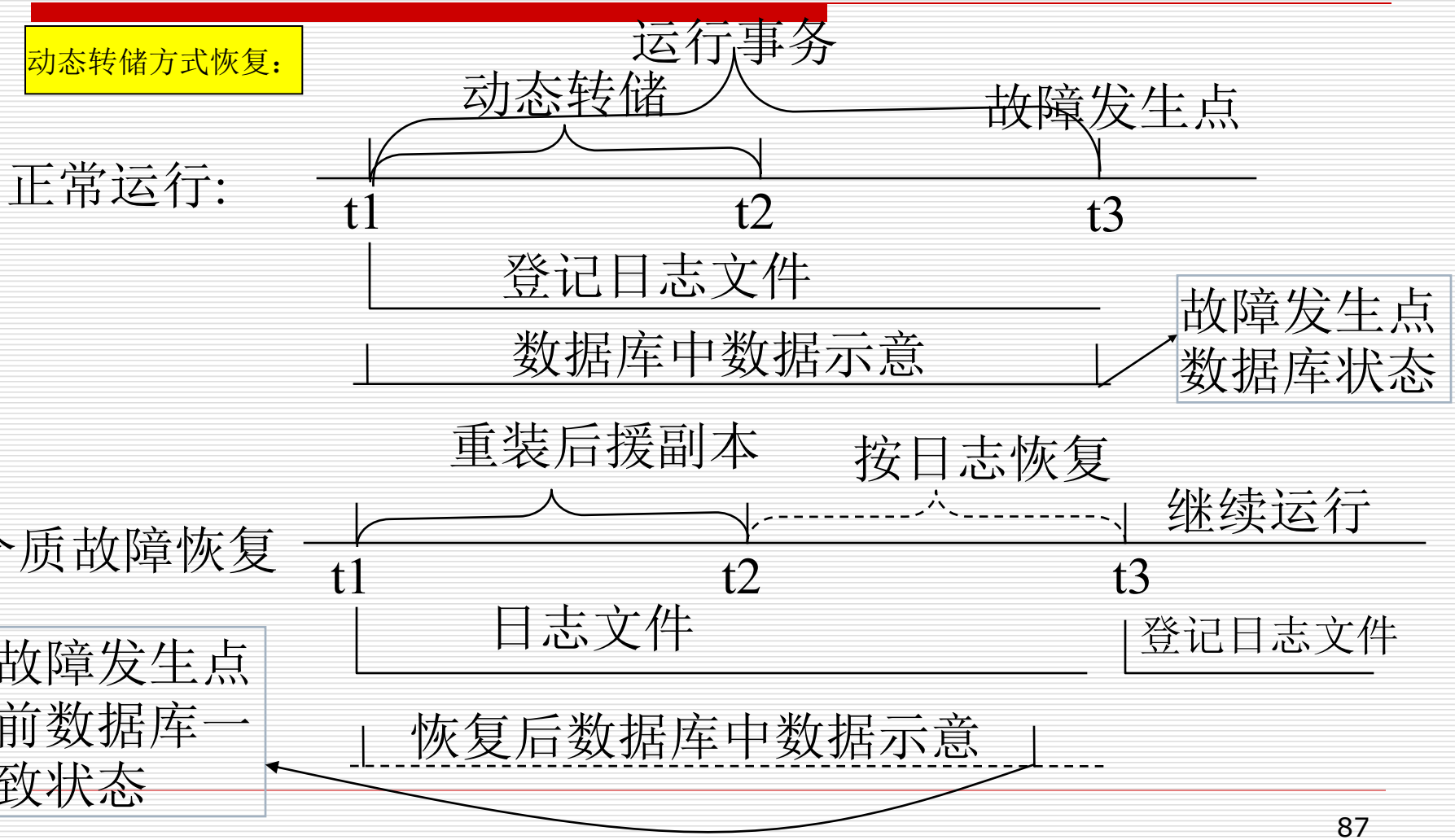
不必运行事务程序



机械工业出版社
CHINA MACHINE PRESS

# 分布式恢复概述

动态转储方式恢复：



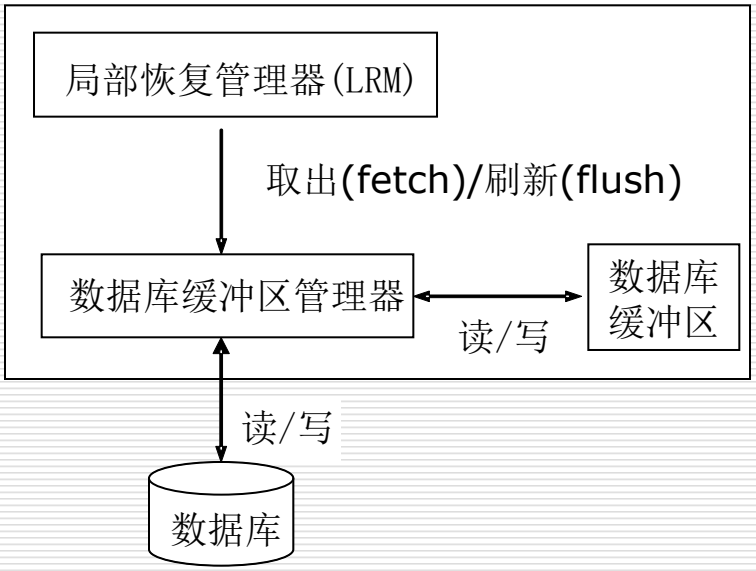
# 集中式数据库的故障恢复

## 局部恢复系统的体系结构

数据库存储在永久性的外存设备上。

数据库缓冲区用来存放最近执行的事务所使用的数据。数据库缓冲区被放置在具有挥发性的内存中，以页为单位来缓存数据。

数据库缓冲区管理器负责读写数据库及缓冲区中的数据。局部恢复管理器与缓冲区管理器之间存在两个交互接口：读取数据页（fetch）和刷新数据页（flush）。

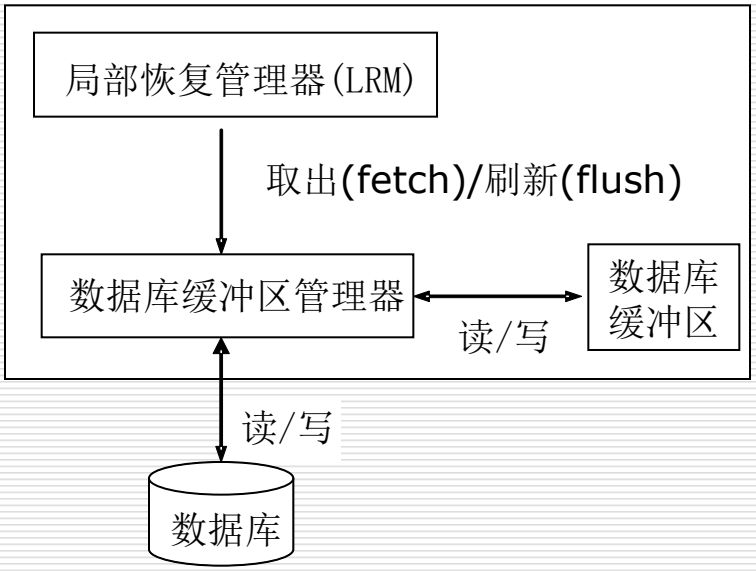


# 集中式数据库的故障恢复

## 局部恢复系统的体系结构

局部恢复管理器刷新数据库缓冲区数据页的过程：

- 首先，由局部恢复管理器发出“刷新”（flush）命令并指明它想要刷新的数据页；
- 然后，缓冲区管理器在当前缓冲区中查找该数据页，若存在，则将该页数据写回到数据库中；若不存在，还要将该页数据从数据库读取到缓冲区中，将其更改后写回到数据库。



# 集中式数据库的故障恢复

## 数据更新策略

(1) 数据的更新策略

数据库数据的更新通常采用两种更新方法，即原地更新和异地更新。

- 原地更新，更新操作直接修改数据库缓冲区中的旧值。
- 异地更新，更新操作将数据项新值存放在与旧值不同的位置上。

异地更新的两种方法：影子页面和差分文件。

影子页面：更新数据时，不改变旧存储页面，而是建立影子页面，将新值存于新建的影子页面上。旧页面用于故障恢复。

差分文件：由于更新操作等价于先删除旧值，再插入新值。因此，差分文件(F)由只读部分(FR)加上插入部分($DF^+$)和删除部分($DF^-$)组成，即$F = (FR \cup DF^+) - (DF^-)$。

# 集中式数据库的故障恢复

## 数据更新策略

(2) 缓冲区的更新策略

缓冲区更新策略有固定/非固定(fix/non_fix)和刷新/非刷新(flush/no_flush) 。

- 固定/非固定(fix/non_fix)：事务在执行过程中，缓冲区管理器是否要等LRM下发命令，才将缓冲区中修改的内容写到外存数据库；
- 刷新/非刷新(flush/no_flush)：在事务执行结束后，LRM是否强制缓冲区管理器将当前缓冲区中已修改数据写回外存数据库；

四种缓冲区更新策略：

- fix/flush
- fix/no_flush
- non_fix/flush
- non_fix/no_flush

# 集中式数据库的故障恢复

- 针对不同的缓冲区更新策略的恢复方法

(1) fix/flush

--特点：事务执行过程中，缓冲区页面固定，废弃事务的修改不会被写入外存；事务结束后，LRM强制将缓冲区中的事务修改写回到外存数据库。

--处理过程：① 事务结束后，LRM向缓冲区管理器发flush命令，将更新数据刷新到外存；② LRM向缓冲区管理器发unfix命令释放所有被固定的页面；③ LRM向日志文件中写事务结束记录。

--恢复操作：不需做任何恢复操作。

# 集中式数据库的故障恢复

## 针对不同的缓冲区更新策略的恢复方法

(2) fix/no_flush

--特点：事务执行过程中，缓冲区页面固定，废弃事务的修改不会被写入外存；事务结束后，LRM不强制将缓冲区中的事务修改写回到外存数据库，已经提交的事务可能没有把修改刷新到外存数据库；

--处理过程：①LRM向日志文件中写事务结束记录；②LRM向缓冲区管理器发unfix命令释放所有被固定的页面。

--恢复操作：已提交的事务进行重做(redo)处理 。

# 集中式数据库的故障恢复

- 针对不同的缓冲区更新策略的恢复方法

(3) no_fix/flush

--特点：事务执行过程中，缓冲区页面不固定，废弃事务的修改可能被写入外存；事务结束后，LRM强制将缓冲区中的事务修改写回到外存数据库，已经提交的事务一定把修改刷新到了外存数据库；

--处理过程：①LRM向缓冲区管理器发flush命令，将更新数据刷新到外存；②LRM向日志文件中写事务结束记录。

--恢复操作：对未提交事务全部做反做(undo)处理。

# 集中式数据库的故障恢复

- 针对不同的缓冲区更新策略的恢复方法

(4) no_fix/no_flush

--特点：事务执行过程中，缓冲区页面不固定，废弃事务的修改可能被写入外存；事务结束后，LRM不强制将缓冲区中的事务修改写回到外存数据库，已经提交的事务不一定把修改刷新到了外存数据库；

--处理过程：LRM向日志文件中写事务结束记录。

--恢复操作：见下页。

# 集中式数据库的故障恢复

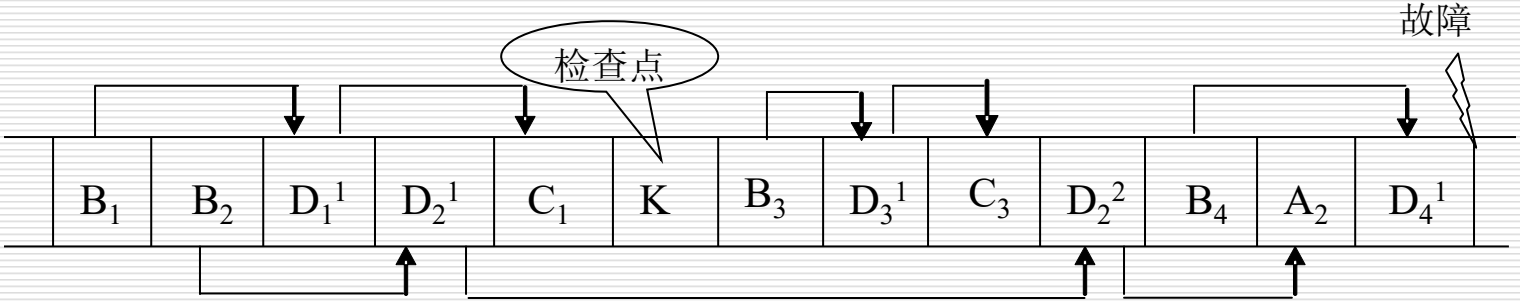
- 针对不同的缓冲区更新策略的恢复方法

(4) no_fix/no_flush

--恢复操作：①LRM首先从重启动文件中取得最近的检查点的地址，然后建立两个表：重做表和反做表。初始化两表，重做表初态为空；反做表初态为检查点上活动事务。②确定重做事务和反做事务：在日志中从检查点向日志结尾扫描。如果发现有新开始（B记录）的事务，放入undo表；如果发现有提交（C记录）的事务，从undo表中删除，写入redo表。③反做undo表中的所有事务，重做redo表中的所有事务。

# 集中式数据库的故障恢复

## 针对不同的缓冲区更新策略的恢复方法




要求：根据日志文件确定redo表和undo表。

步骤1：初始化redo表和undo表。在检查点上没有结束的事务T2为活动事务，放入undo表中。

步骤2：从检测点开始向扫描日志末尾扫描， 如有新开始的事务，放入undo表；如有提交事务，从undo表中删除，放入redo表，直到日志文件结束。

步骤3：反做undo事务表中的事务{T2,T4}，重做redo事务表{T3}；

初始化：redo表={}，undo表={T2}

开始扫描，遇到$B_3$：
redo表={}，undo表={T2} ∪ {T3}

继续扫描，遇到$C_3$：
redo表={} ∪ {T3}，
undo表={T2 ,T3} - {T3}={T2}

继续扫描，遇到$B_4$：
redo表= {T3}，undo表={T2} ∪{T4}

继续扫描，遇到$A_2$：
redo表= {T3}， undo表={T2,T4}

# 分布式事务的故障恢复

在分布式数据库系统中，除了可能出现集中式数据库系统中的故障外，还可能出现分布式数据库系统特有的故障。

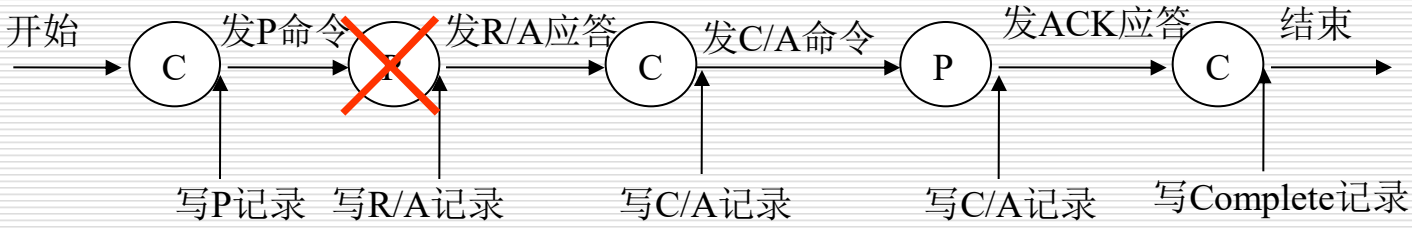
- 发生在各场地内部的故障称为场地故障。
- 发生在场地间通信过程中的故障称为通信故障。

# 分布式事务的故障恢复

## 两阶段提交协议对故障的恢复

*C：协调者*
*P：参与者*

### (1)场地故障




**场地故障情况1：**在参与者场地，若参与者在写R/A记录（*准备提交/放弃*）之前出错。

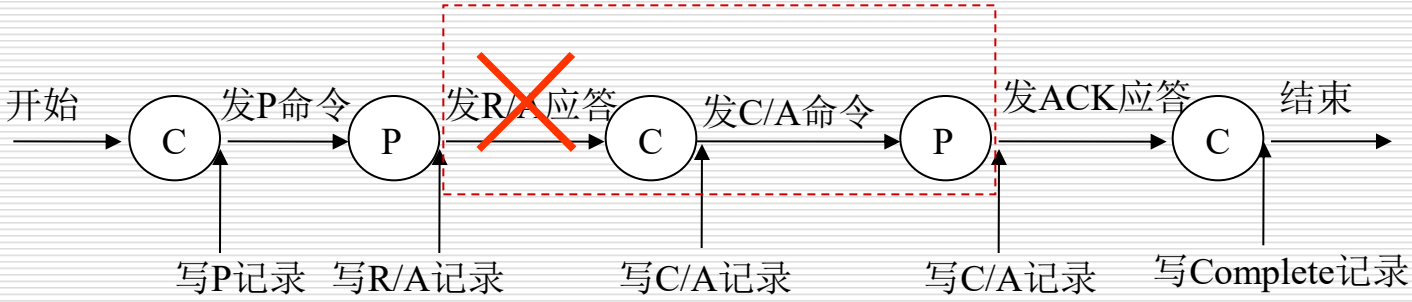
**现象：**协调者发完P命令（*预提交*）后在规定时间内收不到应答信息。

**恢复策略：**协调者做“废弃”处理，即缺省认为收到A应答。

# 分布式事务的故障恢复

## 两阶段提交协议对故障的恢复

### (1)场地故障




**场地故障情况2：**在参与者场地，若参与者在写R/A记录之后，写C/A记录之前出错。

**讨论①：**出错时参与者已写R/A记录，但未发送R/A应答。

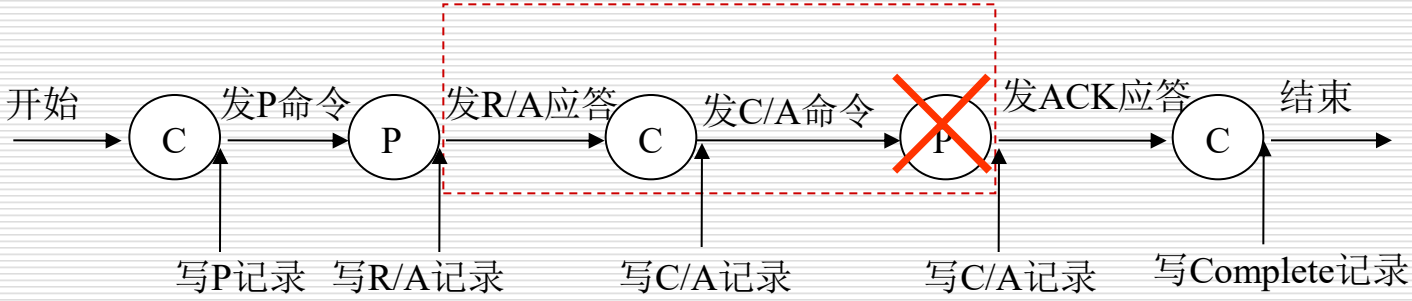
**现象：**由于协调者没有收集到所有参与者返回的R/A应答，因此处于等待状态。

**恢复策略：**故障参与者恢复后， 写A记录，发送A应答。

# 分布式事务的故障恢复

## 两阶段提交协议对故障的恢复

### (1)场地故障




**场地故障情况2：**在参与者场地，若参与者在写R/A记录之后，写C/A记录之前出错。

**讨论②：**出错时参与者已经将R/A应答返回给协调者 。协调者发送了C/A命令，等待参与者返回ACK应答。

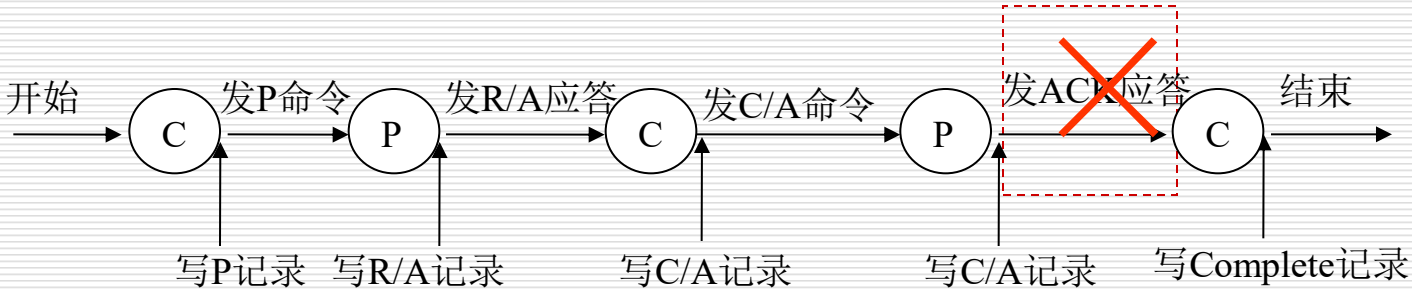
**现象：**由于协调者没有收集到所有参与者返回的ACK应答，因此处于等待状态。

**恢复策略：**协调者给故障参与者重发C/A命令。

# 分布式事务的故障恢复

## 两阶段提交协议对故障的恢复

### (1)场地故障




**场地故障情况3：**在参与者场地，若参与者在写C/A记录之后出错。

**讨论①：**在参与者场地，若参与者在写C/A记录之后出错，但没有返回ACK应答。

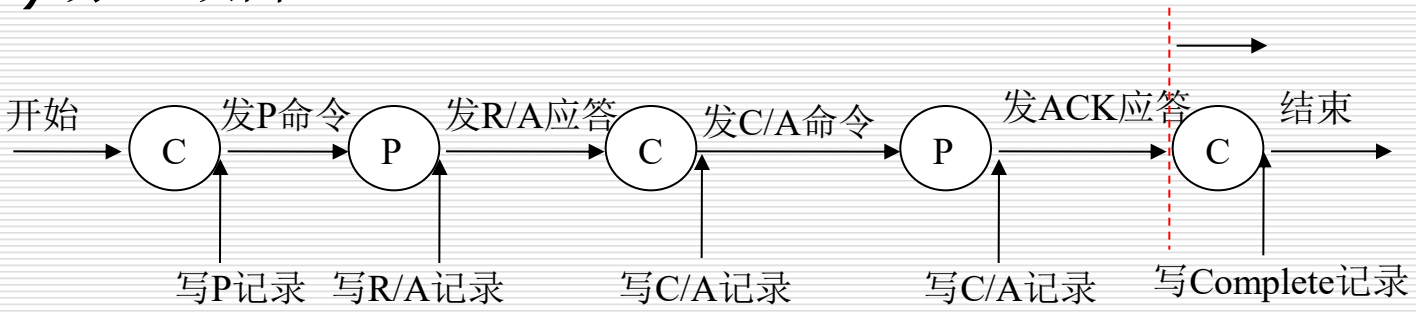
**现象：**由于协调者没有收集到所有参与者返回的ACK应答，因此处于等待状态。

**恢复策略：**协调者给故障参与者重发C/A命令。

# 分布式事务的故障恢复

## 两阶段提交协议对故障的恢复

### (1)场地故障




**场地故障情况3：**在参与者场地，若参与者在写C/A记录之后出错。

**讨论②：**出错时参与者已发送ACK应答。

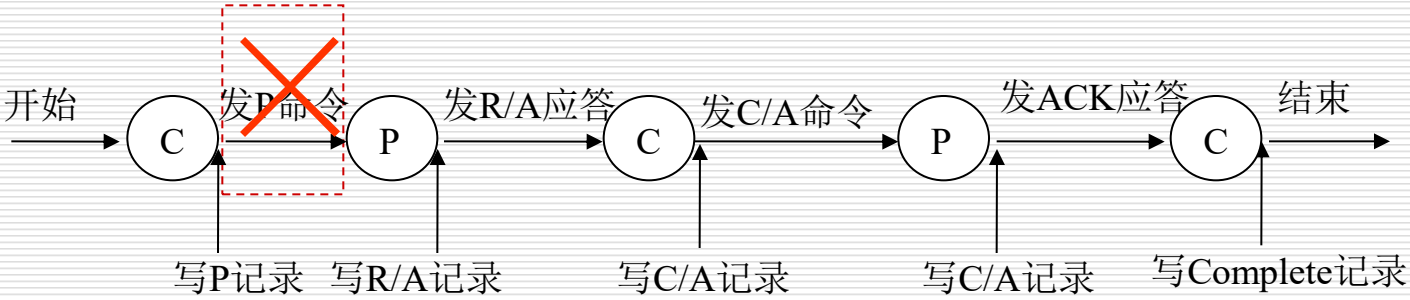
**现象：**协调者收集到所有参与者返回的ACK应答，事务正常结束。

**恢复策略：**不需要任何恢复处理。

# 分布式事务的故障恢复

## 两阶段提交协议对故障的恢复

### (1)场地故障




**场地故障情况4：**在协调者场地，协调者在写P记录之后，写C/A记录之前出错。

**讨论①：**出错时协调者未将P命令发送给参与者。

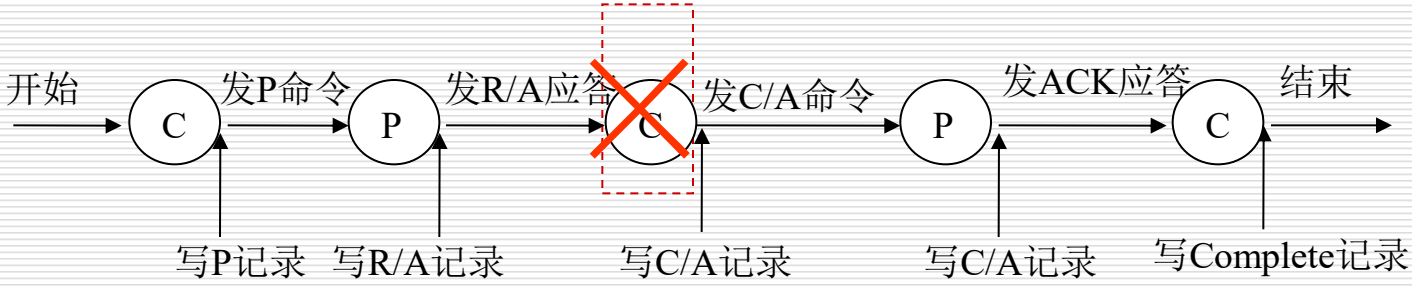
**现象：**无

**恢复策略：**协调者重新启动后，重新执行两段提交协议。

# 分布式事务的故障恢复

## 两阶段提交协议对故障的恢复

### (1)场地故障




**场地故障情况4：**在协调者场地，协调者在写P记录之后，写C/A记录之前出错。

**讨论②：**出错时协调者已将P命令发送给参与者。

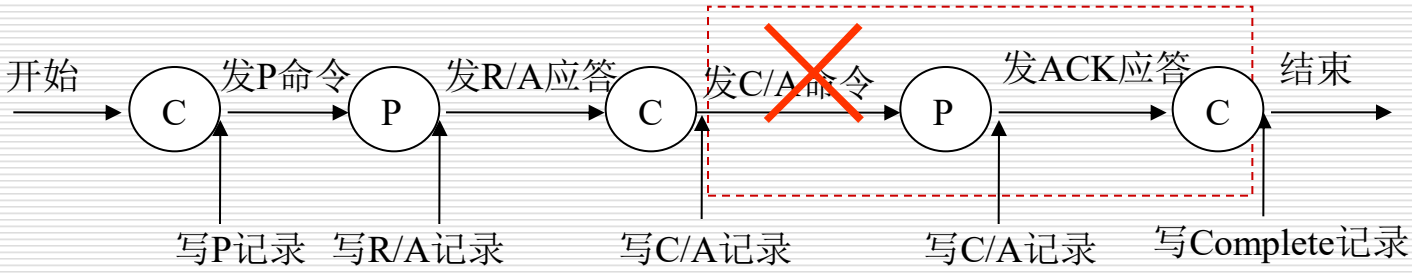
**现象：**此时，参与者已经发送了R/A应答，处于等待状态，等待协调者的C/A命令。

**恢复策略：**参与者发现超时，启动两阶段提交协议的终结协议。

# 分布式事务的故障恢复

## 两阶段提交协议对故障的恢复

### (1)场地故障




**场地故障情况5：**在协调者场地，协调者在写C/A记录之后，写Complete记录之前出错。

**讨论①：**出错时协调者未将C/A命令发送给参与者。

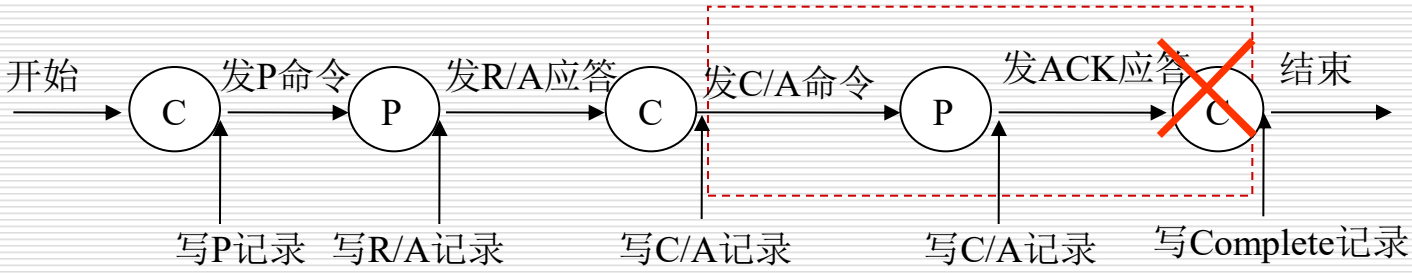
**现象：**此时，参与者已经发送了R/A应答，处于等待状态，等待协调者的C/A命令。

**恢复策略：**参与者发现超时，启动两阶段提交协议的终结协议。

# 分布式事务的故障恢复

## 两阶段提交协议对故障的恢复

### (1)场地故障




**场地故障情况5：**在协调者场地，协调者在写C/A记录之后，写Complete记录之前出错。

**讨论②：**出错时协调者已将C/A命令发送给参与者。

**现象：**此时，协调者在故障过程中可能错过了对参与者ACK应答的接收。

**恢复策略：**协调者重新启动后，给所有参与者重发其C/A命令。

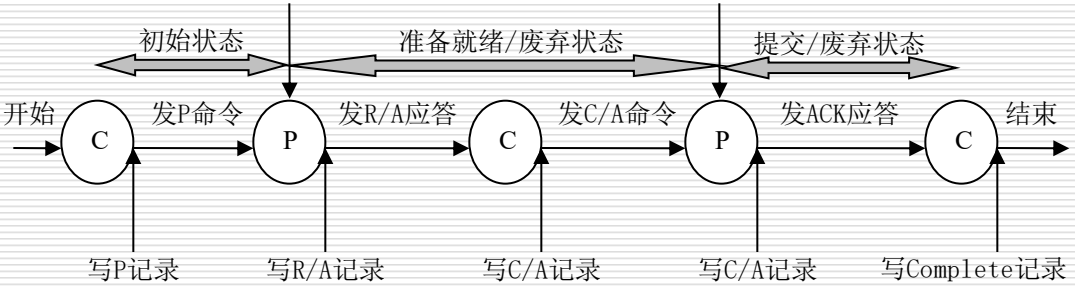
# 分布式事务的故障恢复

## 两阶段提交协议对故障的恢复

### (1)场地故障

两阶段提交协议的终结协议的核心思想是：参与者通过访问其它参与者的当前状态来推断协调者的上一个决定，从而确定终结类型（全局提交还是全局废弃）。

- 初始状态
- 准备就绪状态
- 废弃状态
- 提交状态




**注意：**只有当参与者在成功发送完消息后才完成状态的转换。另外，不存在比其余参与者多于一次状态转换的参与者，即不存在“初始”和“提交”并存的情况。

# 分布式事务的故障恢复

## 两阶段提交协议对故障的恢复

(1)场地故障

两阶段提交协议的终结协议的内容：

步骤1：选择参与者$P_T$作为新的协调者。

步骤2： $P_T$向所有参与者发送“访问状态”命令，各参与者返回自身的状态。

步骤3： $P_T$ 根据各参与者当前的状态做出决定：

（1）若部分参与者处于“初始”状态，部分参与者处于“准备就绪” 状态，则$P_T$ 发送Abort命令；

（2）若所有参与者均处于“准备就绪”状态，则$P_T$ 发送Commit命令；

（3）若至少一个参与者处于 “提交”状态，则$P_T$发送Commit命令；

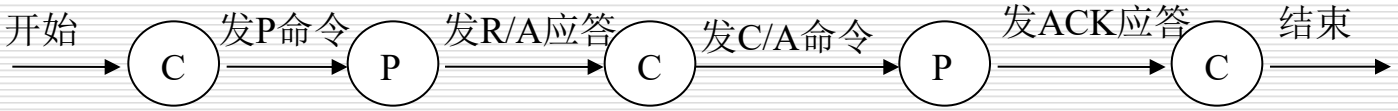
（4）若至少一个参与者处于“废弃”状态，则$P_T$发送Abort命令；

# 分布式事务的故障恢复

## 两阶段提交协议对故障的恢复

(2)通信故障

通信故障有两种：报文丢失和网络分割。




**丢失P报文：**此时，参与者未收到协调者发送的P命令，故不予应答，协调者处于等待R/A应答状态。协调者发现超时，做“废弃”处理。

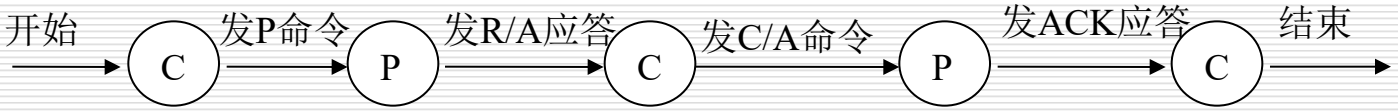
**丢失R/A应答报文：**此时，协调者没有收集到所有参与者返回的应答，默认做“废弃”处理。

# 分布式事务的故障恢复

## 两阶段提交协议对故障的恢复

(2)通信故障

通信故障有两种：报文丢失和网络分割。




**丢失C/A命令报文：**此时，参与者没有及时收到来自协调者的命令，因而处于等待状态。如果参与者发现等待超时，可以申请协调者重新发送C/A命令，或者启动两阶段提交协议的终结协议。

**丢失ACK应答报文：**此时，协调者没有收集到全部的ACK应答而处于等待状态。协调者如果发现超时，向参与者重发C/A命令，要求参与者给予应答。

# 分布式可靠性协议

- 可靠性与可用性

可靠性（Reliability）：在给定环境条件下和规定的时间内，数据库系统符合其行为规范的概率。强调正确性。

可用性（Availability）：在给定时刻t上，数据库系统正常运行的概率。强调可运行的能力。

# 分布式可靠性协议

## 可靠性与可用性

可靠性和可用性往往相互矛盾。例如，数据a的副本分别存放在n个场地上，某事务采用两段提交协议来更新a。当协调者已决定提交，但在给各个参与者发送提交命令时发生通信故障。此时：

- 若要保证系统的可靠性，则要求各个参与者保持等待而不允许其它事务使用a及其副本，以免造成数据的不一致，这就降低了系统的可用性；
- 若要保证系统的可用性，则允许其它事务使用a及其副本，但降低了系统的可靠性；

# 分布式可靠性协议

## 可靠性与可用性

可靠性的度量方法：

在某时间段内系统发生失败的次数服从泊松分布，即在[0，t]的时间段内系统发生k次失败的概率为：

$$\Pr\{k \text{ failures in time } [0,t]\} = \frac{e^{-m(t)}[m(t)]^k}{k!},$$

$$where\ m(t) = \int_0^t z(x)dx.$$

进一步地，在[0，t]的整个时间段内系统发生0次失败的概率。

$$R(t) = \Pr\{0 \text{ failures in time } [0,t]\} = e^{-m(t)}$$



机械工业出版社
CHINA MACHINE PRESS

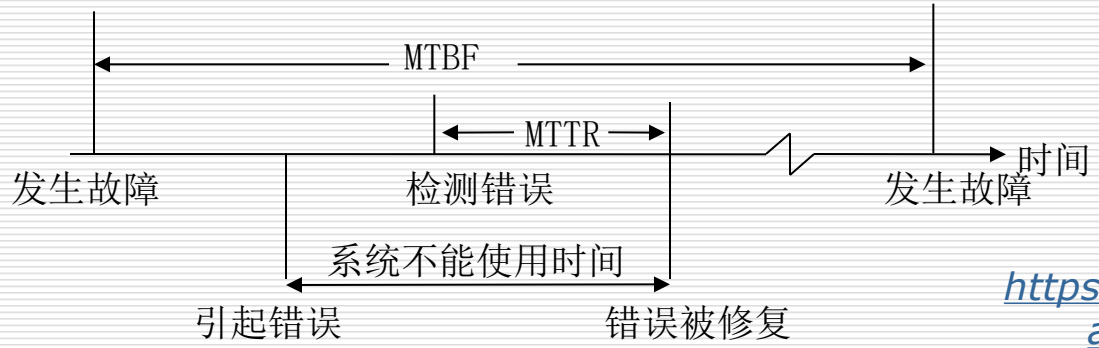
# 分布式可靠性协议

## 可靠性与可用性

可用性的度量方法：

平均故障间隔时间（MTBF）：MTBF是在可以自我修复的系统中相继的两次失败之间的期望时间。

平均故障修复时间（MTTR）：MTTR是修复一个失败系统所需要的期望时间，该值与系统的修复概率相关。




$$A = \frac{MTBF}{MTBF+MTTR}$$

相关易混淆概念的解释
https://blog.csdn.net/yunhua_lee/article/details/121674703

# 3.分布式并发控制

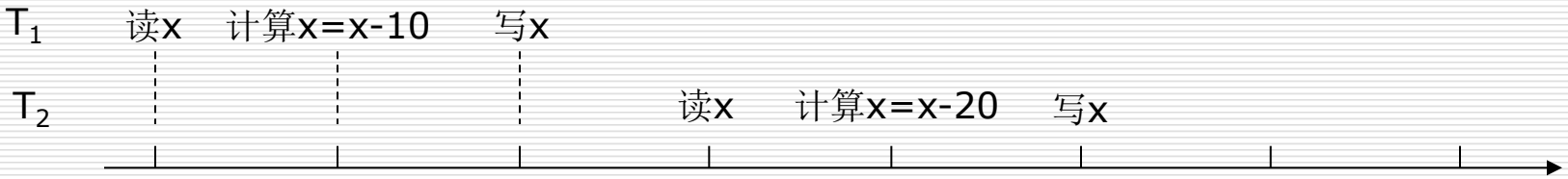
- 基本概念
- 并发控制理论基础
- 基于锁的并发控制方法
- 两段封锁协议
- 分布式数据库并发控制方法
- 分布式死锁管理

# 基本概念

- 简单地讲，并发就是多个事务的同时执行，并发能够提高系统的效率，但也可能会带来三种错误（丢失修改、不能重复读、读脏数据）。
- 并发控制的主要目的是保证事务的一致性和隔离性，最终保证数据的一致性。
- 当分布事务并发执行时，并发控制既要实现分布事务的可串行性，又要保持事务具有良好的并发度，以保证系统具有良好的性能。

# 基本概念

## 并发控制问题



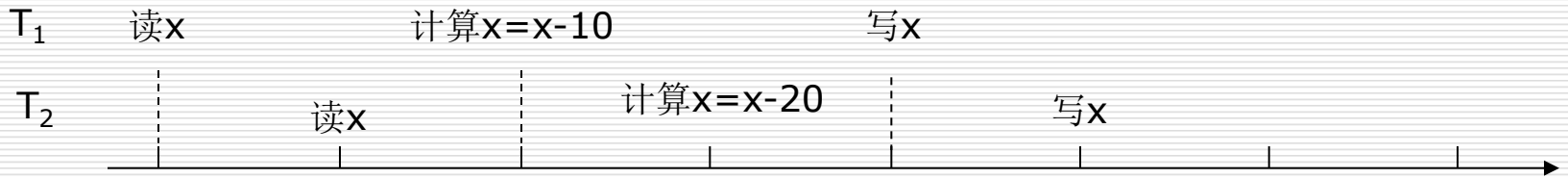

T1：R1（x），O1（x），W1（x）；
T2：R2（x），O2（x），W2（x）；

若串行执行（T1→T2），x初始值为100，则操作序列为：
R1（x），O1（x），W1（x），R2（x），O2（x），W2（x），
执行结果x=70。

# 基本概念

## 并发控制问题

(1) 丢失修改错误

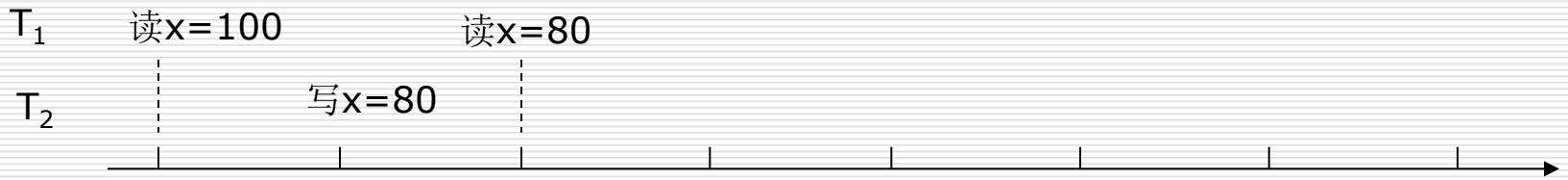



执行结果x=80

# 基本概念

## 并发控制问题

(2) 不可重复读错误

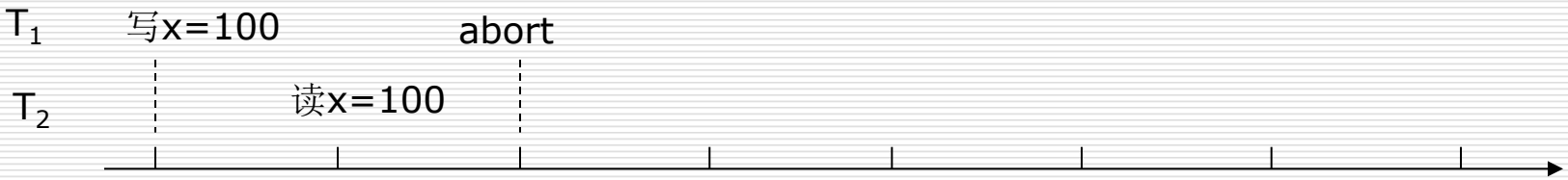



$T_1$两次读取同一个数据，读取的值不同

# 基本概念

## 并发控制问题

(3) 读脏数据错误




$T_2$读取了$T_1$废弃的数据x

# 基本概念

- 并发控制问题

  并发控制就是利用正确的方式调度事务中所涉及的并发操作序列，避免造成数据的不一致性；防止一个事务的执行受到其它事务的干扰，保证事务并发执行的可串行性。

# 并发控制理论基础

## 事务执行过程的形式化描述

通常以串行化理论来检验并发控制方法的正确性。

- 依据串行化理论，在数据库上运行的一个事务的所有操作，按其性质分为读和写两类。
- 读集与写集：一个事务$T_i$所读取数据项的集合，称为$T_i$的读集，所写的数据项的集合，称为$T_i$的写集，分别记为$R(T_i)$和$W(T_i)$。
- 例如：设有事务$T_1$，完成的操作如下：$T_1$：x=x+1；y=y+1；则$T_1$可表示为：$T_1$ :$R_1(x)$ $W_1(x)$ $R_1(y)$ $W_1(y)$。

  读/写集分别是：

  $R(T_1)=\{x，y\}$

  $W(T_1)=\{x，y\}$

# 并发控制理论基础

## 事务执行过程的形式化描述

- 历程： 在一个数据库上，各个事务所执行的操作组成的序列，称为事务的历程，记为H，有时也称为调度，记录了各事务的操作顺序。
- 串行历程与并发历程：对于一个历程上的任何两个事务：$T_i$和$T_j$，如果$T_i$的最后一个操作在$T_j$的第一个操作之前完成，或反之，$T_j$的最后一个操作在$T_i$第一个操作之前完成，称这样的历程为串行历程。否则，称为并发历程。
- 期望：事务历程中的各个事务是并发的，但同时它们的执行结果又等价于一个串行历程，即事务并发历程是可串行化的。

# 并发控制理论基础

- 事务执行过程的形式化描述

例如：

有事务$T_1$和$T_2$：

$T_1$：$R_1(x)$ $R_1(y)$ $W_1(x)$ $W_1(y)$

$T_2$：$R_2(x)$ $W_2(y)$

历程$H_1$和$H_2$，分别为：

$H_1$：$R_1(x)$ $R_1(y)$ $W_1(x)$ $W_1(y)$ $R_2(x)$ $W_2(y)$

$H_2$：$R_1(x)$ $R_2(x)$ $R_1(y)$ $W_1(x)$ $W_1(y)$ $W_2(y)$

可见，$H_1$为串行历程，$H_2$为并行历程。

# 并发控制理论基础

- 集中式数据库的可串行化问题
  - 无论在集中式数据库系统中，还是在分布式数据库系统中，并发调度都要解决并发事务对数据库的冲突操作问题，使冲突操作串行执行，非冲突操作并发执行
  - 在分布式数据库系统中，事务是由分解为各个场地上的子事务的执行实现的。因此，分布式事务之间的冲突操作，就转化为了同一场地上的子事务之间的冲突操作，分布式事务的可串行性调度也转化为了子事务的可串行性调度问题

# 并发控制理论基础

- 集中式数据库的可串行化问题
  - 可串行化：在集中式数据库系统中的一个历程H，如果等价于一个串行历程，则称历程H是可串行化的。由可串行化历程H所决定的事务执行顺序，记为SR(H)
  - 事务的执行顺序：分别属于两个事务的两个操作$O_i$和$O_j$，如果它们操作同一个数据项，且至少其中一个操作为写操作，则$O_i$和$O_j$这两个操作是冲突的。例如：$R_1(x)$ 和$W_2(x)$，$W_1(x)$和 $W_2(x)$ 。
  - 在一个串行历程中，用符号“<”表示先于关系，对分别属于$T_i$和$T_j$的两个冲突操作$O_i$和$O_j$，若存在$O_i<O_j$，则$T_i<T_j$



机械工业出版社
CHINA MACHINE PRESS

# 并发控制理论基础

- 集中式数据库的可串行化问题
  - 历程等价的判别方法

**定理1：**任意两个历程$H_1$和$H_2$等价的充要条件是

- 在$H_1$和$H_2$中，每个读操作读出的数据是由相同的写操作完成的；
- 在$H_1$和$H_2$中，每个数据项上最后的写操作是相同的

- **定理1的引理：**对于两个历程$H_1$和$H_2$，如果每一对冲突操作$O_i$和$O_j$，在$H_1$中有$O_i<O_j$，在$H_2$中也有$O_i<O_j$，则$H_1$和$H_2$是等价的

# 并发控制理论基础

## 集中式数据库的可串行化问题

### 历程等价的判别方法

例：$H_1$：$R_1(x)$ $R_1(y)$ $W_1(x)$ $W_1(y)$ $R_2(x)$ $W_2(x)$

$H_2$：$R_1(x)$ $R_1(y)$ $W_1(x)$ $R_2(x)$ $W_1(y)$ $W_2(x)$

$H_3$：$R_1(x)$ $R_1(y)$ $R_2(x)$ $W_1(x)$ $W_1(y)$ $W_2(x)$

$H_1$是串行历程。判断$H_2$和$H_3$是否与$H_1$等价（是可串行化的历程）。

H1的冲突操作：

$R_1(x) < W_2(x)$

$W_1(x) < W_2(x)$

$W_1(x) < R_2(x)$

H2的冲突操作：

$R_1(x) < W_2(x)$ √

$W_1(x) < R_2(x)$

$W_1(x) < W_2(x)$

H3的冲突操作：

$R_1(x) < W_2(x)$ ×

$R_2(x) < W_1(x)$

$W_1(x) < W_2(x)$

# 并发控制理论基础

## 分布式事务的可串行化问题

- 局部历程：在分布式事务执行过程中，场地$S_i$上的所有子事务的操作的执行序列，称为局部历程，用$H(S_i)$表示。
- **分布式事务可串行化判定定理：**对于n个分布式事务$T_1$，$T_2$，…，$T_n$在m个场地上$S_1$，$S_2$，…，$S_m$上的并发执行序列，记为E。如果E是可串行化的，则必须满足以下条件：
  - 每个场地$S_i$上的局部历程$H(S_i)$是可串行化的；
  - 存在E的一个总序，使得在总序中，如果有$T_i<T_j$，则在各局部历程中必须有$T_i<T_j$。

# 并发控制理论基础

## 分布式事务的可串行化问题

**分布式事务可串行化判定定理的引理：**

设$T_1$，$T_2$，…，$T_n$是n个分布式事务，E是这组事务在m个场地上的并发执行序列，$H(S_1)$，$H(S_2)$，…，$H(S_m)$是在这些场地上事务的局部历程，如果E是可串行化的，则必须存在一个总序，使得$T_i$和$T_j$中的任意两个冲突操作$O_i$和$O_j$，如果在$H(S_1)$，$H(S_2)$，…，$H(S_m)$中有$O_i<O_j$，当且仅当在总序中也有$T_i<T_j$。

# 基于锁的并发控制方法

- 基本思想：事务在对某一数据项操作之前，必须先申请对该数据项加锁，申请成功后才可以对该数据项进行操作。如果该数据项已被其它事务加了不相容的锁，那么后申请使用数据的事务必须等待，直到该数据项被解锁为止。
- 锁分两种类型：排它锁(exclusive)和共享锁(shared)
- 锁的相容性：

| | 读锁 | 写锁 |
|---|---|---|
| 读锁 | 共享 | 排斥 |
| 写锁 | 排斥 | 排斥 |

# 基于锁的并发控制方法

- 封锁规则
  - 事务在执行过程中需对其访问的数据项进行加锁，访问结束要及时释放其对数据项加的锁，以便供其它事务访问，保证多个事务正确地并发执行。
  - 具体封锁规则为：
    - 事务T在对数据项A进行读/写操作之前，必须对数据项A施加读/写锁，访问后立即释放已申请的锁；
    - 如果事务T申请不到希望的锁，事务T需等待，直到申请到所需要的锁之后，方可继续执行。

# 基于锁的并发控制方法

- 锁的粒度
  - 封锁数据对象的单位称锁的粒度，指被封锁的数据对象的大小。锁的粒度也称锁的大小。系统根据自己的实际情况确定锁的粒度，锁的粒度可以是关系的属性（或字段）、关系的元组（或记录）、关系（也称文件）或整个数据库等。
  - 系统的并发度与锁粒度成反比。锁粒度越大，系统的开销越小，但降低了并发度。
- 锁的种类
  - 显式锁，用户加封锁命令实现
  - 隐式锁，系统自动加锁实现

| 粒度 | 开销 | 并发度 |
|---|---|---|
| 小 | 大 | 高 |
| 大 | 小 | 低 |

# 两段封锁协议

- 两段封锁协议(2PL)是数据库系统中解决并发控制的重要方法之一，保证事务的可串行性调度。
- 2PL的实现思想是将事务中的加锁操作和解锁操作分两阶段完成，要求并发执行的多个事务要在对数据操作之前进行加锁，且每个事务中的所有加锁操作要在解锁操作以前完成。
- 两段封锁协议分为：
  - 基本的两段封锁协议
  - 严格的两段封锁协议

# 两段封锁协议

## 基本的两段封锁协议

基本的两段封锁协议分加锁阶段和解锁阶段。

阶段1：加锁阶段

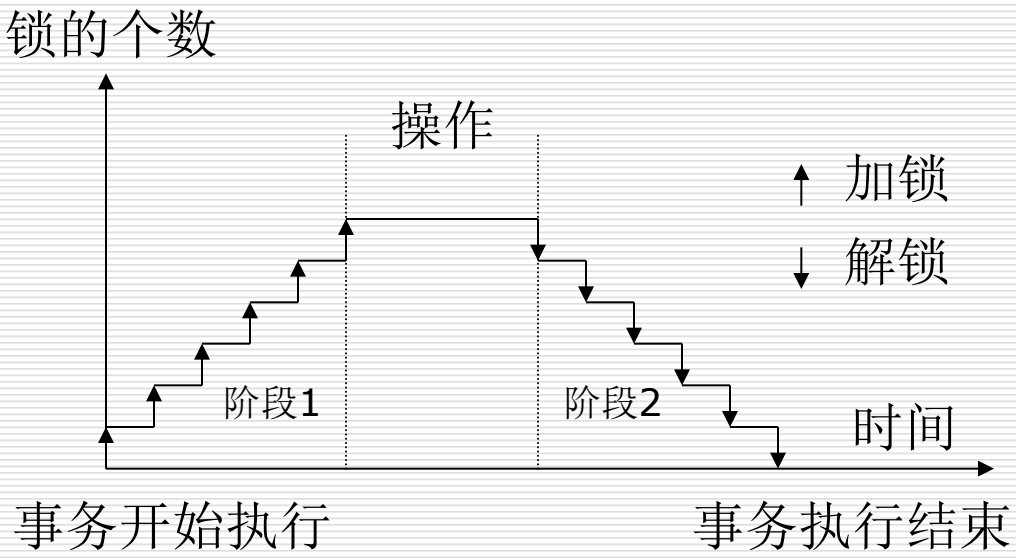
- 事务在读写一个数据项之前，必须对其加锁；
- 如果该数据项被其它使用者已加上不相容的锁，则必须等待。

阶段2：解锁阶段

事务在释放锁之后，不允许再申请其它锁；

# 两段封锁协议

- 基本的两段封锁协议

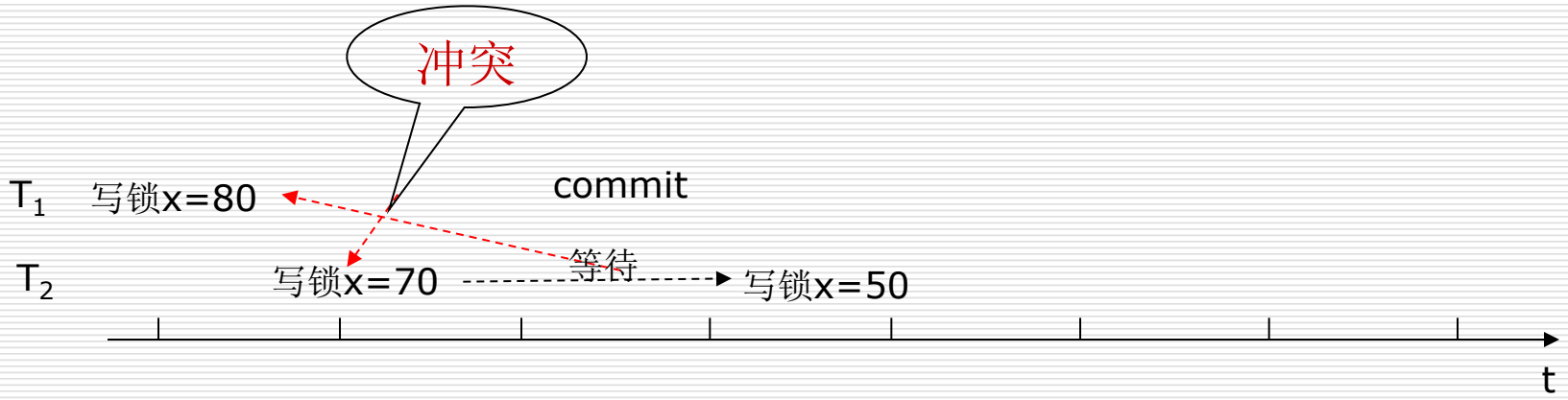



两阶段封锁协议是否可以解决并发事务带来的3个错误？

# 两段封锁协议

## 基本的两段封锁协议

丢失修改错误

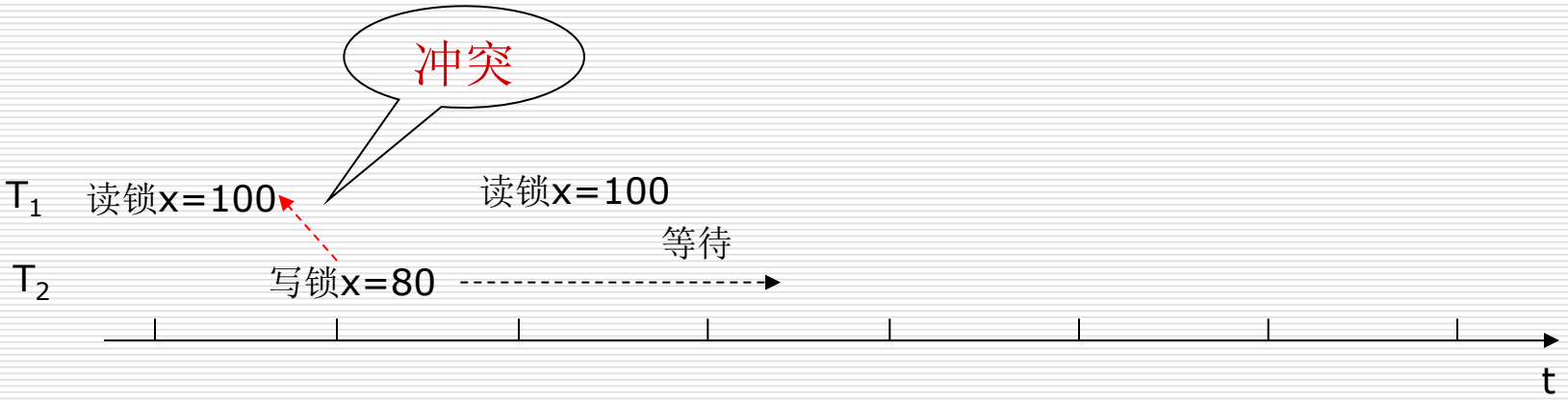



X=100:当事务$T_2$：x-30申请写锁时，不能申请成功，必须等待，当事务$T_1$：x-20提交数据项x，放锁后，$T_2$才能申请到锁，读到x=80，结果为50，避免了丢失修改错误。

# 两段封锁协议

## 基本的两段封锁协议

不能重复读错误

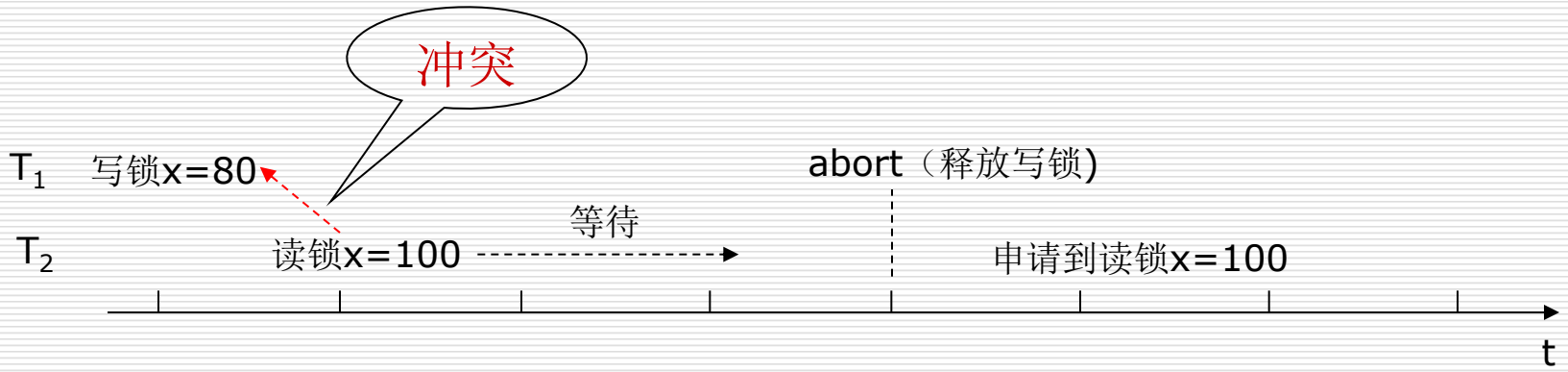



当事务T2申请写锁时，不能申请成功，必须等待，当事务T1再次读数据项x时，读取的x值与第一次读到的值相同，避免了不能重复读错误。

# 两段封锁协议

## 基本的两段封锁协议

读脏数据错误



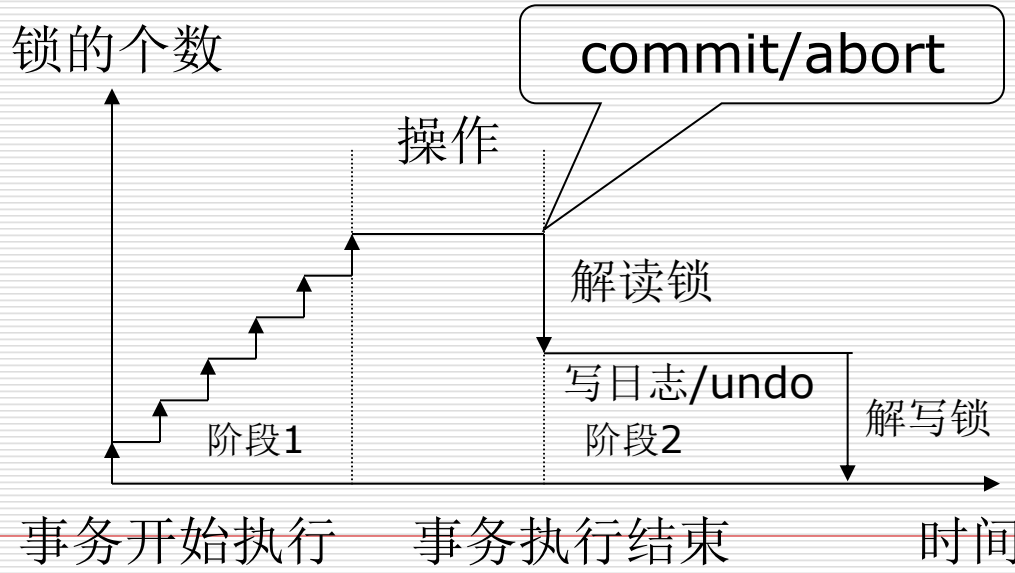

事务T1申请写锁后，写x=80。此时事务T2申请读锁失败，必须等待。当事务T1产生故障中断时，废弃事务T1的执行，即执行undo，使x恢复为原值100，并释放对x加的写锁。此时事务T2申请到读锁，读取x值，x值为原值100，而不是事务T1的脏数据80。

# 两段封锁协议

## 严格的两段封锁协议

严格的两段封锁协议与基本的两段封锁协议内容基本上是一致的，只是解锁时刻不同，是在事务结束时才启动解锁，保证了事务所更新数据的永久性。采用两段封锁协议（2PL）的事务执行过程为：

begin_transaction→加锁→操作→ commit/abort→解锁。




(1)对commit的处理

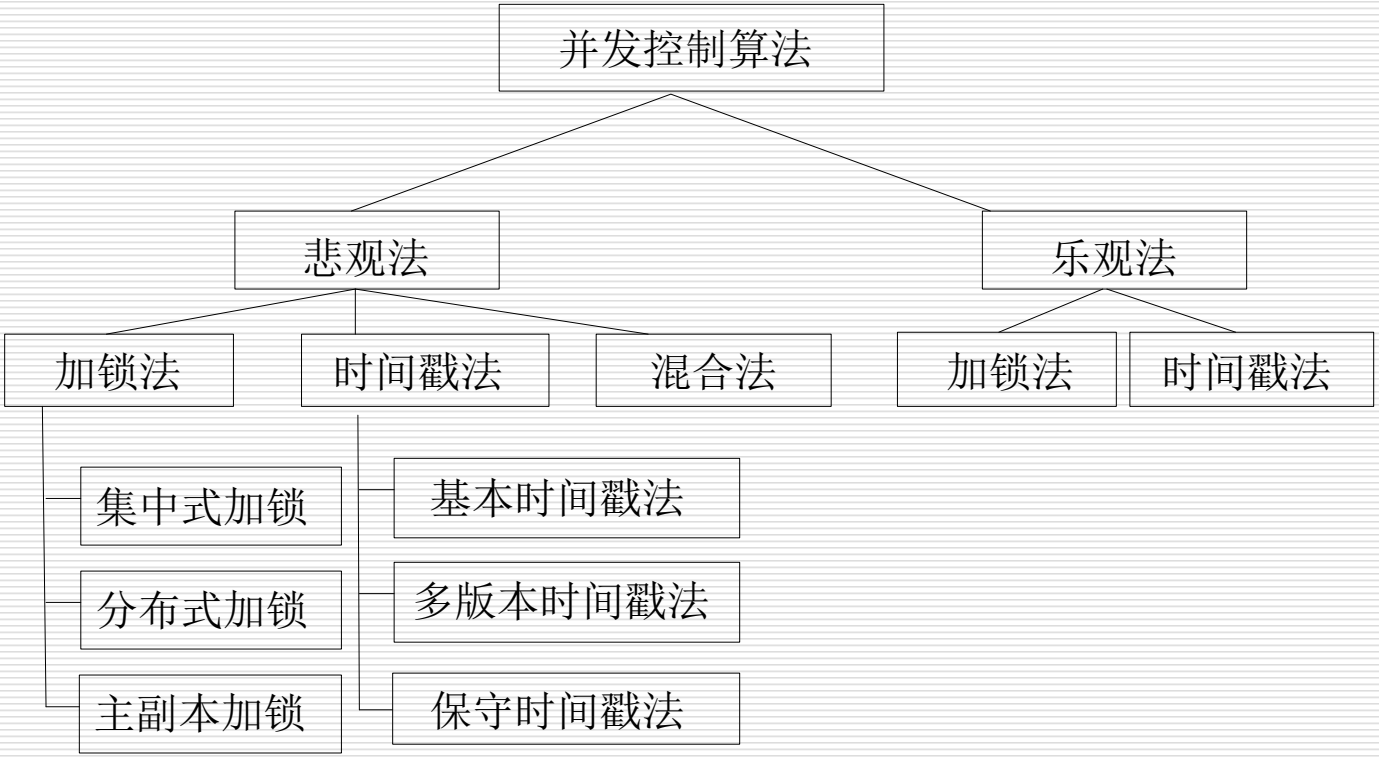
- 释放读锁；
- 写日志；
- 释放写锁；

(2)对abort的处理

- 释放读锁；
- undo处理；
- 释放写锁；

# 分布式数据库的并发控制

基于锁的并发控制方法、时间戳方法、乐观方法等

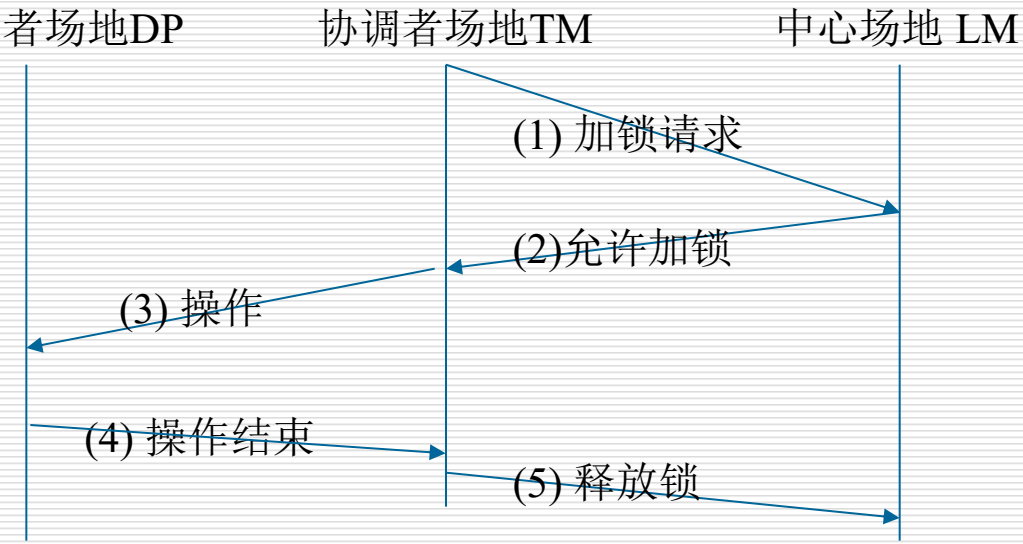


# 分布式数据库的并发控制

## 基于锁的并发控制方法的实现

### (1) 集中式的实现方法

集中式实现方法是在分布库中设立一个2PL调度器，所有封锁请求均由该调度器完成，每个场地的事务管理器都和该调度器通信。




优点：实现简单，方便管理；

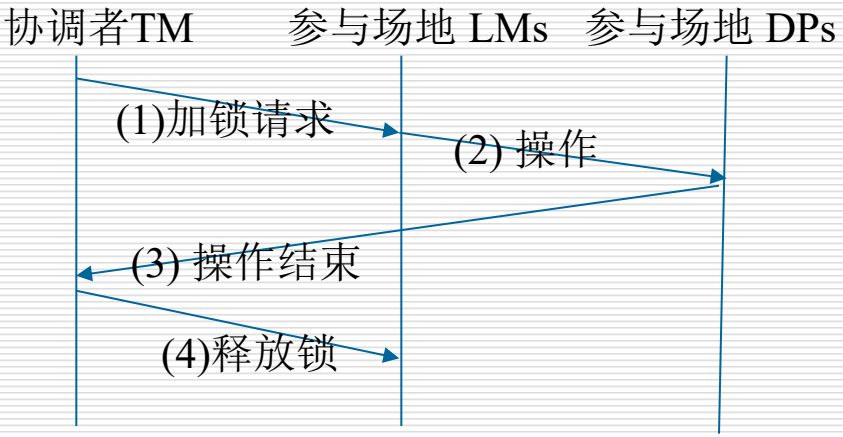
缺点：中心场地负载重，单点失效；

# 分布式数据库的并发控制

## 基于锁的并发控制方法的实现

### (2) 分布式的实现方法

分布式实现方法是在每个场地上都有一个2PL调度器，负责处理本场地上的封锁请求。




优点：避免了集中式封锁方法的缺点；

缺点：实现复杂，锁管理复杂；

# 分布式数据库的并发控制

## 基于锁的并发控制方法的实现

### (3) 对复制数据的封锁方法

- 基于特定副本的封锁方法
  - 主副本法：规定一个数据项的多个副本中的一个作为主副本，所有对该数据项的封锁请求都施加在主副本上。
  - 主场地法：规定保存副本的某个场地为主场地，所有封锁请求都施加在主场地上的数据项副本上。
  - 带有后备场地的主场地法
- 基于投票的封锁方法
  - 读写全法：事务对数据项加读锁时，只封锁其中一个副本；若加写锁，必须封锁所有副本。
  - 多数副本法是：无论读锁还是写锁申请，必须封锁数据项一半以上的副本。申请成功后，若为读锁，读取一个副本的值；若为写锁，需向所有副本发送新值。

# 分布式数据库的并发控制

- 基于时间戳的并发控制方法的实现

  基于锁的并发控制是通过事务的互斥来维护并发事务的可串行化的；

  基于时间戳的方法是通过事务的优先级来实现并发事务的可串行化。

  事务的优先级通过时间戳来实现，事务管理器为每个事务$T_i$在其产生时都设置时间戳，$ts(T_i)$。

# 分布式数据库的并发控制

## 基于时间戳的并发控制方法的实现

### (1) 基本概念

- 时间戳(TimeStamp)是基于事务启动时间点，由系统赋予该事务的全局唯一标识。同一事务管理器产生的时间戳是单调增加的，可以通过时间戳来区分事务。
- 设置时间戳的方法主要有以下几种：一种是使用全局单调递增计数器，但是全局的计数器比较难维护。
- 一般每个场地都有自己的单调递增的计数器,为了保证唯一性，时间戳是一个二元组<本地计数器值，场地id>，要保持各局部时钟的同步，则才能实现全局有序。

# 分布式数据库的并发控制

## 基于时间戳的并发控制方法的实现

### (1) 基本概念

通常，遵循如下规则设置时间戳：

① 每个事务在启动场地赋予一个全局唯一标识（时间戳）；

② 事务的每个读操作或写操作都带有本事务的时间戳；

③ 数据库中的每个数据项都记录有对其进行读操作和写操作的最大时间戳。设数据项为X，X的读/写操作的最大时间戳分别标记为rts(X)和wts(X)；

④ 如果事务被重新启动，则其被赋予新的时间戳。

# 分布式数据库的并发控制

## 基于时间戳的并发控制方法的实现

### (1) 基本概念

- 基于时间戳的并发控制方法的思想：给每个事务赋予一个唯一时间戳，根据时间戳对事务的操作执行顺序进行排序，则事务按时间戳顺序串行执行。如果发生冲突，撤销一个事务并重新启动，同时为重新启动事务赋予新的时间戳。
- 时间戳排序(TimeStamp Ordering)规则：分别属于事务$T_i$和事务$T_k$的两个冲突操作$O_i^j$和$O_k^l$，$O_i^j$在$O_k^l$之前执行当且仅当$ts(T_i)<ts(T_k)$，称$T_i$是较老的事务，$T_k$是较新的事务。

# 分布式数据库的并发控制

## 基于时间戳的并发控制方法的实现

### (1) 基本概念

- 基于时间戳的事务并发控制的调度方法

①设ts是对数据X进行读操作的时间戳，若ts<rts(X)，则拒绝该操作，重新启动该事务并赋予新的时间戳；否则执行读操作，rts(X)=ts。

②设ts是对数据X进行写操作的时间戳，若ts<rts(X)或ts<wts(X)，则拒绝该操作，重新启动该事务并赋予新的时间戳；否则，执行写操作，wts(X)=ts

- 基于时间戳的并发控制方法不会出现死锁，任何事务不会被阻塞。若某个事务的某个操作不能执行，则事务重新启动。但是，基于时间戳的并发控制方法避免死锁是以重启动为代价的。

# 分布式数据库的并发控制

- 基于时间戳的并发控制方法的实现

(2) 基本的时间戳排序(Basic TO)算法

事务管理器TM对每个事务设置时间戳并且将其附着在事务所包含的每个数据库操作上。

调度器SC负责跟踪读写时间戳并进行序列检查。

# 分布式数据库的并发控制

## 基于时间戳的并发控制方法的实现

### (2) 基本的时间戳排序(Basic TO)算法

while(1){ ----------事务管理器(TM)端算法----------

监听各个场地的消息;

如果是数据库操作信息，那么判断操作类型：

--开始事务，则将场地集合清空，并设置事务时间戳ts(T);

--读，选择执行场地加入场地集合，并将操作信息和事务时间戳发送给目标场地调度器;

--写，存储数据的场地加入场地集合，并将操作类型和事务时间戳发送给目标场地调度器;

--commit/abort，则将操作发送给场地的调度器;

如果是调度器信息，那么判断信息内容：

--拒绝，则将"Abort T"添加到消息中，发送给场地中的调度器，重新设置时间戳;

--数据处理信息，则根据不同处理类型执行不同操作：

---读，则返回给应用读的结果;

---写，则通知用户写完成;

---commit/abort，则通知应用程序提交/废弃完成;

}

# 分布式数据库的并发控制

## 基于时间戳的并发控制方法的实现

### (2) 基本的时间戳排序(Basic TO)算法

While(1){----------调度器(SC)端算法----------

监听TM发送的数据库操作消息；

初始化参数，保存读写时间戳，判断消息内容，执行相应的操作：

---读操作，则比较时间戳，如果ts(T) > rts(x)，则将数据操作信息发送给本场地DP，并且将rts(x)修改为ts(T)；如果 ts(T)< rts(x)则拒绝该事务的操作，并将拒绝消息发送给TM；

---写操作，若(ts(T) > rts(x)&&ts(T) > wts(x))，则发送处理操作给本场地DP，并且将rts(x)和wts(x)都修改为ts(T)；否则，拒绝行该操作，并通知TM。

---提交，则发送数据操作给本场地DP；

---废弃，则对于所有被事务T访问过的数据恢复rts(X)和wts(X)为其初始值，发送数据操作给本场地DP 。

}

# 分布式数据库的并发控制

- 基于时间戳的并发控制方法的实现

(3) 保守的时间戳排序(TO)算法

- 基本的TO算法不会产生死锁，但会带来过多的重启动，导致系统性能下降。
- 保守的TO算法的思想：通过减少事务重启的数量减少系统的开销。
- 基本TO算法试图当接收到一个操作时就立即执行该操作，而保守算法是希望尽可能延迟每个操作，直到保证调度器中没有时间戳更小的操作。如果这个条件可以保证的话，那么调度器将不会拒绝操作，但是这种延迟可能会带来死锁。

# 分布式数据库的并发控制

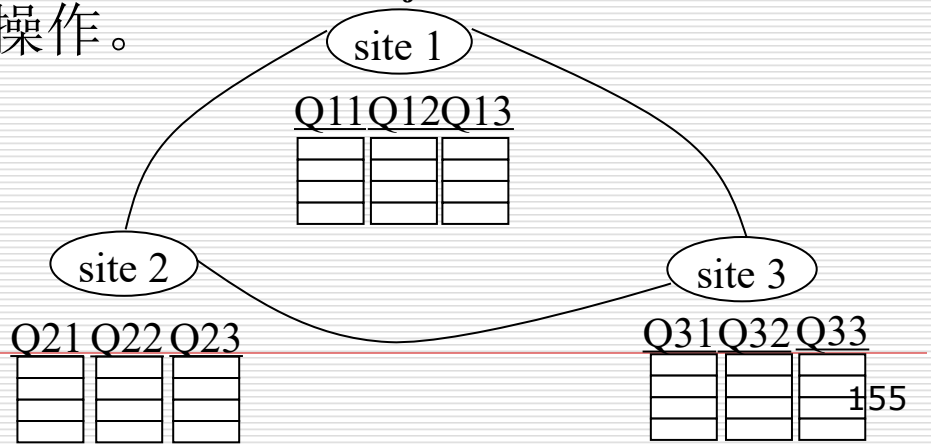
- 基于时间戳的并发控制方法的实现

(3) 保守的时间戳排序(TO)算法

基本思想：每个事务的操作都被缓存起来并建立起有序的队列，然后按照顺序执行操作。

具体做法：在场地site i，每个调度器i有一个队列$\mathbf{Q_{ij}}$，用于对应场地j的TM，简称$TM_j$。一个来自于$TM_j$的操作按照时间戳递增的顺序到达site i，并放置于$\mathbf{Q_{ij}}$中。调度器i按照这个顺序执行所有队列的操作。

例子：存在3个场地，每个场地上的调度器都为所有场地的TM建立一个事务操作的队列。



# 分布式数据库的并发控制

## 基于时间戳的并发控制方法的实现

### (3) 保守的时间戳排序(TO)算法

对于读操作R, 若在Site i有缓冲的写，使得

$$TS(R) > TS(W)$$

即申请的读新于已有的写，则R被缓冲，直到所有这样的写完成。

对于写操作W, 若在Site i有缓冲的读，使得

$$TS(W) > TS(R)$$

或有缓冲的写W′，使得

$$TS(W) > TS(W')$$

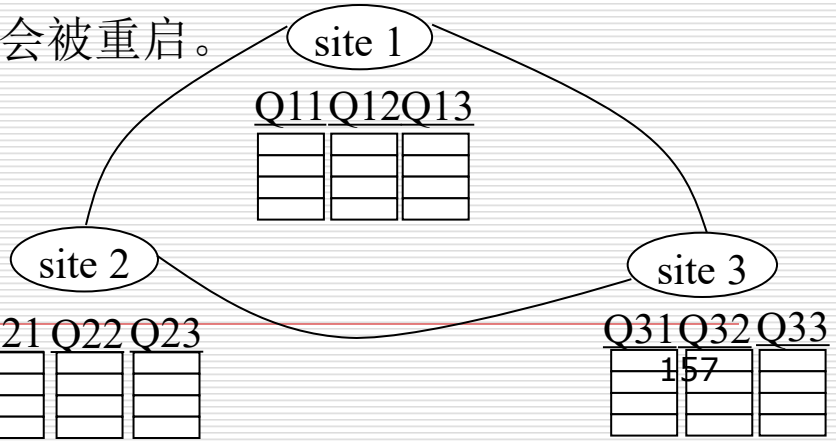
则W被缓冲，直到所有这些操作被执行完毕。

# 分布式数据库的并发控制

## 基于时间戳的并发控制方法的实现

### (3) 保守的时间戳排序(TO)算法

保守时间戳算法可以减少大量的重启，但是当队列是空的时候，还是会存在重启的现象。例如：若场地i上对场地j的队列$Q_{ij}$是空的，场地i的调度器将选择一个最小时间戳的操作发送给数据处理器，可是，场地j可能已经发送给i一个带有更小时间戳的操作，当操作到达场地i的时候，将会被拒绝，来自场地j的事务会被重启。



# 分布式数据库的并发控制

- 基于时间戳的并发控制方法的实现

(4) 多版本时间戳算法

- 减少重启动的另外一种方法，具体做法：

  修改x上时间戳，对数据项x

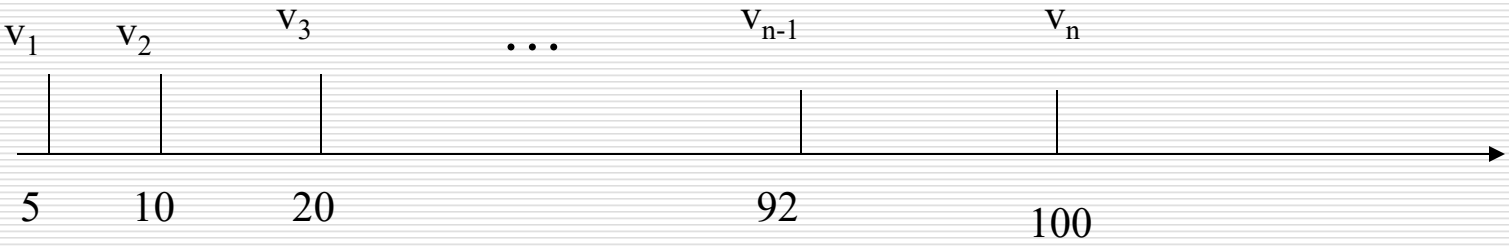
  - RTM(x) 有一组值，记录了所有$T_i$对x的读时，即 RTM($T_i$, x)
  - WTM(x) 有一组值，记录了所有$T_i$对x的写时，新值也被记录，即WTM($T_i$ ,x, $v_i$)

# 分布式数据库的并发控制

- 基于时间戳的并发控制方法的实现

(4) 多版本时间戳算法

- 当要对x读时，读操作永不拒绝。假定读操作的时间戳是TS，于是在WTM($T_i$, x, $v_i$)中找所有小于TS的{WTM($T_i$, x, $v_i$)}中的最大 WTM($T_j$, x, $v_j$)，将$v_j$给读的事务。
- 例如：



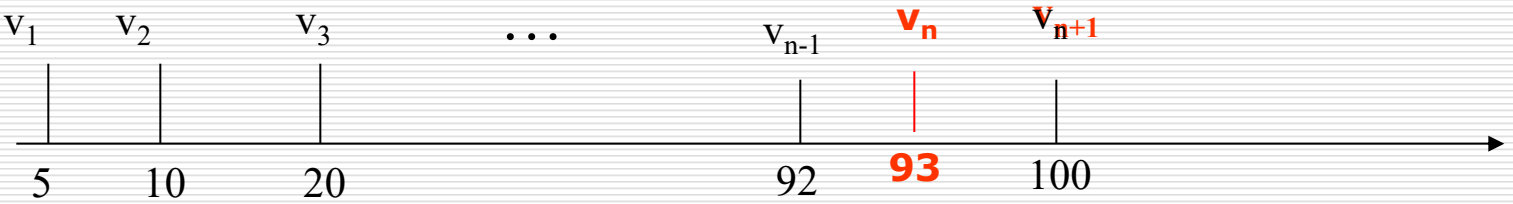

若读 TS($R_i$)=95， 则读 <92, $v_{n-1}$> 的值。

# 分布式数据库的并发控制

## 基于时间戳的并发控制方法的实现

### (4) 多版本时间戳算法

- 当要对x写时，一般不拒绝。只记录下新的WTM($TS(T_k)$, x, $v_k$) 即可。若存在一个拥有比$TS(T_k)$大的时间戳的事务(更加新的事务)已经对x完成了读，那么拒绝 $TS(T_k)$的写。
- 例如：




$TS(R_i)=95$, 读走 $<92, V_{n-1}>$。此时，若写$TS(W_k)=93$请求写，那么拒绝$T_k$的写。

否则，Ti事务应该读走$<93,V_n>$，也就是说刚才$T_i$读错了数据。

# 分布式数据库的并发控制

- 基于时间戳的并发控制方法的实现

(4) 多版本时间戳算法

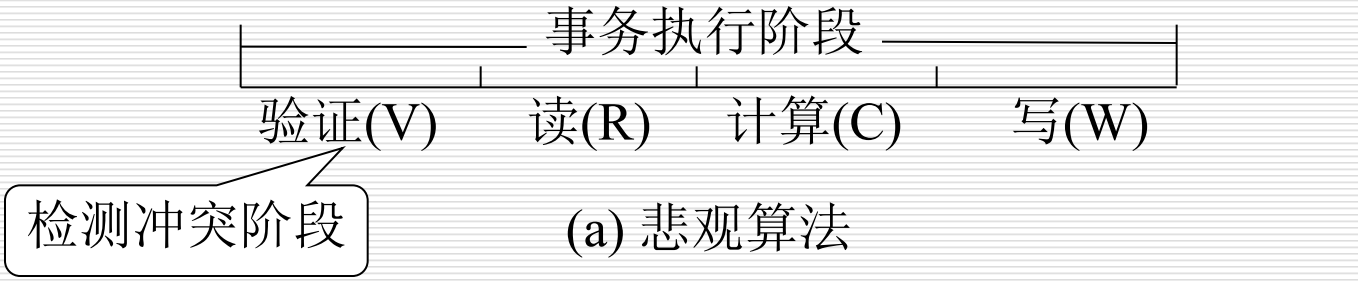
- 多版本TO算法是一种以牺牲存储空间来换取尽量少的事务重启。
- 如果采用多版本TO的并发控制方法，为了节省空间，应经常对数据库的多个版本进行清理。
- 当分布式DBMS确定不再接收需要访问被清除版本的事务时，执行清除操作。

# 分布式数据库的并发控制

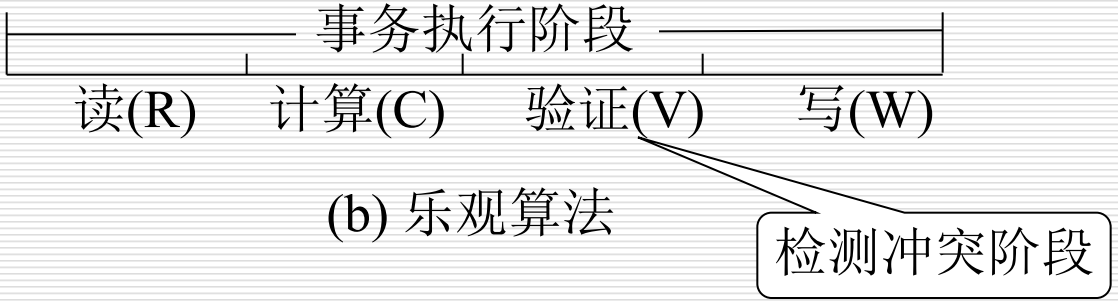
- 乐观的并发控制算法
  - 悲观算法假设冲突经常发生，而乐观算法等到写阶段开始时，才进行冲突验证，也就是说延迟验证阶段直到执行写操作之前。
  - 提交给乐观调度器的操作不会被延迟，每个事务的读、计算和写操作可以自由处理，不需要更新实际的数据库。
  - 每个事务在初始时都在本地更新副本，验证阶段检查这些更新是否能够维护数据库的一致性，如果结果是肯定的，就把这些更新写到实际的数据库中。否则，该事务就废弃并被重启。

# 分布式数据库的并发控制

## 乐观的并发控制算法




(a) 悲观算法




(b) 乐观算法

# 分布式数据库的并发控制

- 乐观的并发控制算法
  - 乐观控制算法的规则：
    - 时间戳只和事务相关，而与数据项无关。
    - 时间戳不是在初始化时而是在开始验证时赋给事务，因为时间戳只有在验证阶段才需要，过早赋值可能会导致不必要的事务拒绝操作。
    - 每个事务$T_i$又被分为许多子事务，每个子事务都可以在许多场地上执行。

# 分布式数据库的并发控制

## 乐观的并发控制算法

乐观控制算法的规则：

令$T_{ij}$为在场地j上执行的$T_i$的子事务，每个场地上子事务按一定执行序列执行，直到验证阶段，时间戳被赋给事务并拷贝给它的所有子事务，则$T_{ij}$的本地验证规则为(这些规则是互斥的)：

- **规则1:** 如果所有事务$T_k$，其中$ts(T_k)<ts(T_{ij})$，在$T_{ij}$开始读操作前完成了它们的写操作阶段，那么验证成功。

  实际上是多事务的串行执行。

- **规则2:** 如果任何事务$T_k$，满足$ts(T_k)<ts(T_{ij})$，在$T_{ij}$读阶段完成$T_k$写操作，当$WS(T_k) \cap RS(T_{ij}) = \Phi$时，验证成功。

  并发执行$T_{ij}$的读和$T_k$的写，但是$T_k$的写不影响$T_{ij}$的读。

- **规则3:** 如果任何事务$T_k$，满足$ts(T_k)<ts(T_{ij})$，在$T_{ij}$完成读阶段之前完成$T_k$读操作，当$WS(T_k) \cap RS(T_{ij}) = \Phi$ 并且 $WS(T_k) \cap WS(T_{ij}) = \Phi$时，验证成功。

  $T_k$先完成读，然后并发执行$T_{ij}$和$T_k$，如果$T_k$的写不影响$T_{ij}$的读且$T_{ij}$和$T_k$的写集交集为空，那么也不互相影响。

# 分布式死锁管理

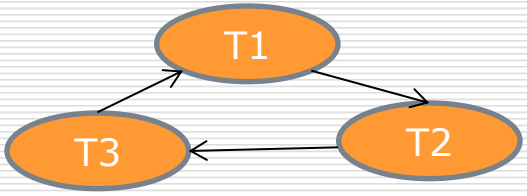
- 在使用锁的并发控制中，可能导致死锁的产生。产生死锁原因：
  - 因为在多个事务并发执行的情况下，每个正在执行的事务已对其拥有的资源进行加锁。
  - 已经占有了部分资源的事务可能会因为其他事务对其需要的资源已经加了不相容的锁而处于等待状态。
  - 多个彼此相互等待对方释放自己需要的资源的事务就会产生死锁。
- 因此，在分布式数据库管理系统中必须设计合适的处理死锁的算法。

# 分布式死锁管理

## 死锁等待图

在数据库系统中通常采用等待图方法分析死锁。等待图(Wait-For Graph，WFG)是一个表示事务之间等待关系的有向图，图中节点表示系统中的并发事务，从$T_i$指向$T_j$的有向边表示事务$T_i$等待事务$T_j$释放其对某些资源所加的锁。

当且仅当等待图中的边存在回路，即出现死锁。



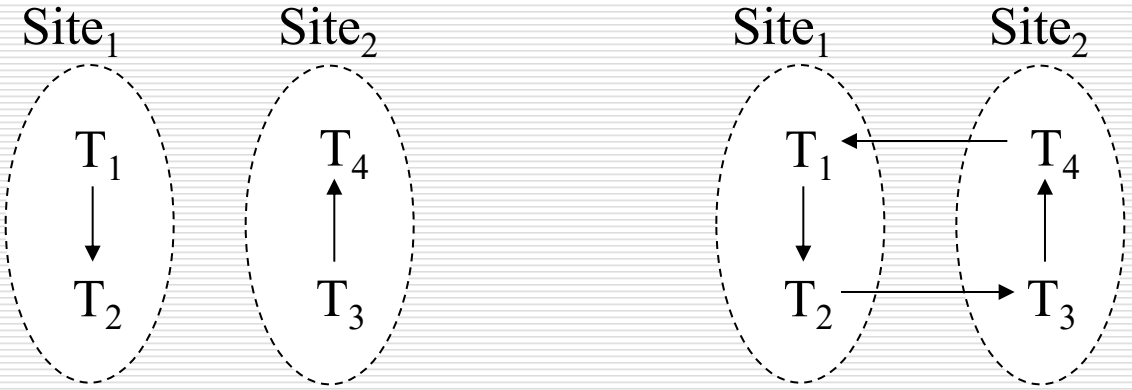
# 分布式死锁管理

## 死锁等待图

- 等待图的方法也同样适用于分布式数据库系统中。不过其检测过程比在集中式数据库更复杂，因为导致死锁产生的两个事务很可能不在同一场地上，称为全局死锁。
- 在分布式数据库的死锁检测中，除了要检测各场地事务的局部等待图(LWFG)是否存在回路，还要检测所有场地事务之间的全局等待图(GWFG)是否存在回路。
- 全局等待图由局部等待图组成，首先要得到局部等待图，然后由所有的局部等待图联合而成全局等待图。

# 分布式死锁管理

## 死锁等待图




(a)局部等待图　　(b)全局等待图

场地Site1和场地Site2分别有事务$T_1$、$T_2$、$T_3$、$T_4$。在分布式死锁检测中，首先要对每个场地的事务进行等待图的绘制。从(a)可知，在局部站点中并未出现回路，还需进一步判断各站点之间的事务是否存在等待的状态。将(a)中的局部等待图按照事务间的等待关系联合起来，形成图(b)。可以看到，事务$T_1$、$T_2$、$T_3$、$T_4$在全局等待图中构成了一个回路，因而产生了死锁。

# 分布式死锁管理

## □ 死锁的检测

### (1) 集中式死锁检测

- 选择一个场地执行整个系统的死锁检测程序(也称为集中死锁检测器)。
- 每个场地的锁管理器都要向该死锁检测器传输其LWFG，从而逐渐构成GWFG。如果在GWFG中发现回路，则认为存在死锁并采取相应的处理措施。
- 不足：频繁的LWFG传输会带来大量的通信代价，因此，各个场地的锁管理器只需传输其等待图中变化部分的信息，以减少通信量。

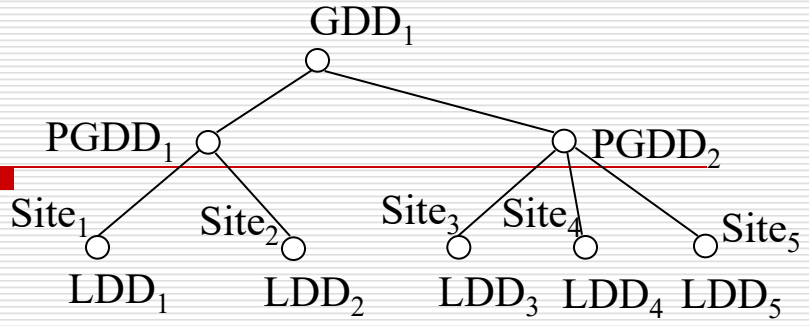
# 分布式死锁管理

## □ 死锁的检测

### (1) 集中式死锁检测

- 优点：实现简单
- 缺点：
  - 瓶颈、单点失效。
  - 难于确定各个场地向检测器传输信息的时间间隔。间隔小，可减小死锁检测延迟，但增加数据传输代价；间隔大，可减少数据传输量，但影响死锁检测及时性。
  - 通信代价大。各个场地周期地向死锁检测器场地发送消息，然而真正导致死锁产生的事务可能只涉及到少数几个场地。

# 分布式死锁管理




## □ 死锁的检测

### (2) 层次死锁检测

- 是对集中式死锁检测方法的改进，目的是减少通信量。
- 层次死锁的检测方法是建立一个死锁检测层次树，每个站点的局部死锁检测程序(LDD)作为一个叶子节点，中间层节点是部分全局死锁检测程序(PGDD)，根节点是全局死锁检测程序(GDD)。
- 每个场地首先在本地用等待图检测死锁并做出相应的处理，然后将局部等待图发送给上层节点。每个非叶子都至少包括两个下层节点，从底层至上层，部分全局死锁检测程序可以检测其所包括的下层节点的事务是否产生死锁，直到最顶层的根节点，由根节点死锁检测程序判断全局事务是否产生死锁。

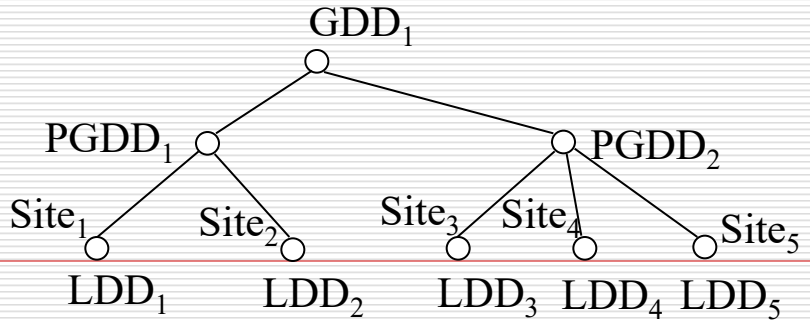
# 分布式死锁管理

- 死锁的检测

## (2) 层次死锁检测

- 例如：

场地1至场地5的死锁检测由$LDD_1$至$LDD_5$完成。PGDD1负责检测场地1与场地2中的事务是否构成死锁，PGDD2负责检测场地3、场地4与场地5中的事务是否构成死锁，GDD1负责检测全局死锁。



# 分布式死锁管理

## □ 死锁的检测

### (2) 层次死锁检测

- 与单纯的集中式死锁检测方法相比，层次死锁检测方法减少了数据传输量，是使用集中式2PL并发控制算法系统中常选择的死锁检测方法。
- 层次死锁检测方法的运行性能与层次的选择至关重要，在进行场地组划分时，应考虑站点之间的物理距离以及站点上容纳的数据的相关度等。
- PGDD数目过大会增加运行成本，过少将失去其优越性。

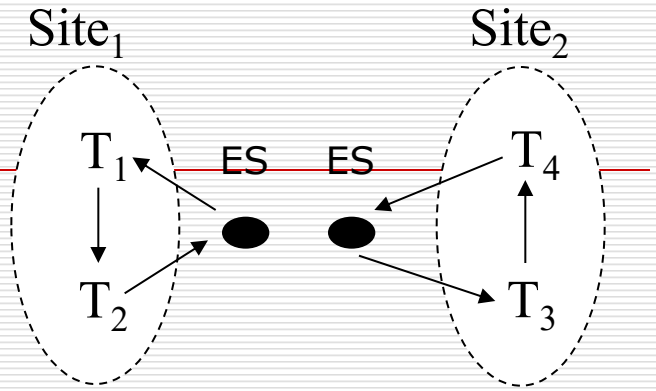
# 分布式死锁管理

## □ 死锁的检测

### (3) 分布式死锁检测

- 检测由各个场地共同完成，每个场地都是对等的，不区分全局死锁和局部死锁的检测程序，都承担检测全局死锁的任务。
- 在检测过程中，局部WFG与其它场地的WFG相互通信，测试是否存在回路。

# 分布式死锁管理




## □ 死锁的检测

### (3) 分布式死锁检测

例如，路径下推(Path-pushing)算法，主要思想是：

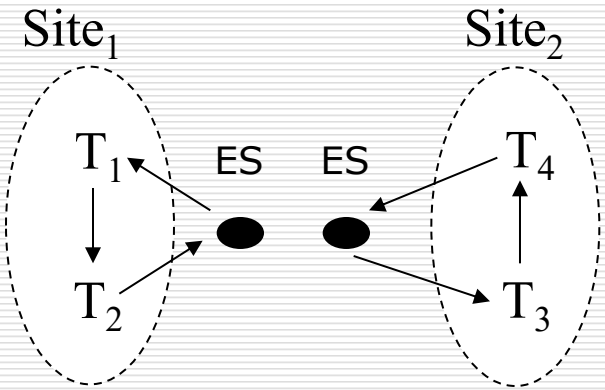
- 每个场地都接受来自于其它场地的潜在的死锁回路，并将这些边加入到本地的LWFG中。
- 等待其它场地事务的WFG中的边，并与远程节点的LWFG的边相连接，从而判断是否存在死锁。
- 如果回路不包括外部边，则为局部死锁，可以被局部处理；如果回路中包括外部边，则存在潜在的分布式死锁，必须将该回路信息通知给其它场地的死锁检测程序。

# 分布式死锁管理

## □ 死锁的检测

### (3) 分布式死锁检测

- 例如：$T_1 \rightarrow T_2 \rightarrow Es$和$T_3 \rightarrow T_4 \rightarrow Es$两个回路中都包括了外部结点(External site, Es)，可以判断存在潜在的死锁回路，并且两个场地都可以检测到可能的分布式死锁。
  - 每个场地都能检测出死锁回路，应该选择合适的操作场地以减少死锁检测代价。
  - 思想：以沿着死锁回路的正向或者反向传输信息，得到信息的场地更新它的LWFG并检查死锁。
  - 如图，场地1发送它的潜在死锁信息给场地2，场地2也发送潜在死锁信息给场地1。两个场地上都将检测死锁，实际上只需要一个场地检测即可。



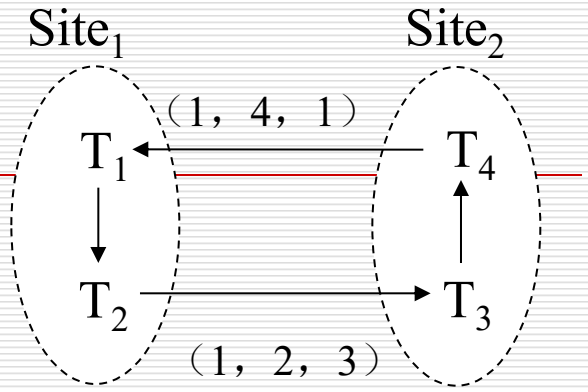
# 分布式死锁管理

- 死锁的检测

(3) **分布式死锁检测**

边追踪(Edge-Chasing)算法：沿着资源请求图上的等待边方向传递信息，并由各节点根据目前收集到的信息，判断是否出现了死锁，算法使用了LWFG来检测局部死锁，并且利用探测信息确认是否存在全局死锁。

# 分布式死锁管理




## □ 死锁的检测

### (3) 分布式死锁检测

例如：场地1上的两个事务形成依赖关系$T_1 \rightarrow T_2$，而$T_2$依赖于远程站点上的事务$T_3$，判断是否处于死锁：

- 发送探测消息(1,2,3)给场地2上的事务$T_3$的管理器。
- 如果$T_3$处于非阻塞状态，则丢弃该探测消息；如果$T_3$也处于阻塞状态，在场地2上，事务$T_3$和事务$T_4$具有依赖关系$T_3 \rightarrow T_4$，则探测消息更新为(1,3,4)。
- $T_4$依赖于远程场地1的事务$T_1$，因而形成探测信息(1,4,1)发送给事务$T_1$的管理器。此时，$T_1$场地的事务管理器发现，探测消息中出现了回路，产生死锁。

# 分布式死锁管理

## 死锁的预防与避免

*Ti是请求资源的事务*
*Tj是拥有资源的事务*

(1) 死锁的预防

顺序封锁法。预先对数据对象规定一个封锁顺序，所有事务都按照这个顺序实行封锁，避免在等待图中出现回路。

(2) 死锁的解决

使用事务的时间戳来解决死锁，基本思路是：杀死死锁等待图中的某个事务，破坏环路。典型的算法有两种：

- 等待-死亡(WAIT-DIE)

等待-死亡是非抢占算法，规则如下：

- 如果ts(Ti) < ts(Tj)，Ti等待；
- 否则，撤销Ti，并保持Ti原有时间戳重新启动。

等待-死亡算法倾向于新的事务，抑制老的事务。

- 负伤-等待(wound-wait)

负伤-等待是抢占算法，规则如下：

- 如果ts(Ti) < ts(Tj)，撤销Tj，并且将加锁的权利赋予Ti；
- 否则，Ti等待。

负伤-等待规则倾向于较老的事务，但是Tj可以保持原来的时间戳重启。

# 考试重点

## 并行计算 & 分布式系统

- 并行计算机结构：结构模型、访存模型
- 互联网络：典型网络的节点度、网络直径、对剖宽度，存储转发选路 **vs** 切通（虫孔）选路
- 性能评测：加速比性能定律对比
- 并行算法：通讯模型、**PCAM**设计阶段
- 分布式通信：通信模式对比、典型消息中间件
- 分布式存储：键值数据分片类型、一致性哈希算法、数据复制类型、 **Quorum**协议的一致性保证
- 分布式计算：计算模式、**LLM**并行方式、参数同步架构

# 考试重点

## 分布式数据库

- 分片设计：分片类型、正确性判断、分片树
- 查询优化：全局SQL语句、关系代数表达式、查询树、优化后查询树、分片查询树（转换过程）
- 存取优化：计算全连接/半连接优化等策略的传输代价、执行场地情况
- 分布式事务：描述分布式事务执行过程
- 恢复管理：根据日志信息确定事务执行过程和undo/redo事务集
- 并发控制：判断历程可串行情况